OSU Mitre 2026

Generated by Doxygen 1.14.0

1 Overview 1
1.1 Contents . . . 1

2 High Level Design 3
2.0.1 Pin management . . . 3
2.1 Groups . . . 4
2.2 File Storage . . . 4
2.2.1 File Encryption . . . 4
2.2.2 File Verification . . . 5
2.3 File Transfer . . . 5

3 Miscellaneous Details 7
3.1 Stalls . . . 7
3.2 UART Errors . . . 7
3.3 Random Data . . . 7
3.4 File System Layout . . . 7

4 Class Index 9
4.1 Class List . . . 9

5 File Index 11
5.1 File List . . . 11

6 Class Documentation 13
6.1 async_uart_ctx Struct Reference . . . 13
6.1.1 Detailed Description . . . 14
6.1.2 Member Data Documentation . . . 14
6.1.2.1 buffer . . . 14
6.1.2.2 bytes_transfered . . . 14
6.1.2.3 dma_channel . . . 14
6.1.2.4 first_chunk_offset . . . 14
6.1.2.5 stop . . . 15
6.1.2.6 stopped . . . 15
6.1.2.7 total_bytes . . . 15
6.1.2.8 transfer_complete . . . 15
6.1.2.9 uart . . . 15
6.2 BufferNode Struct Reference . . . 15
6.2.1 Detailed Description . . . 16
6.2.2 Member Data Documentation . . . 16
6.2.2.1 buffer . . . 16
6.2.2.2 length . . . 16
6.2.2.3 next . . . 16
6.3 file_address_table_t Struct Reference . . . 17
6.3.1 Detailed Description . . . 17

6.3.2 Member Data Documentation . . . 17
6.3.2.1 entries . . . 17
6.4 file_fat_entry_t Struct Reference . . . 18
6.4.1 Detailed Description . . . 18
6.4.2 Member Data Documentation . . . 18
6.4.2.1 address . . . 18
6.4.2.2 length . . . 19
6.4.2.3 padding . . . 19
6.4.2.4 uuid . . . 19
6.5 file_metadata_signed_t Struct Reference . . . 19
6.5.1 Detailed Description . . . 20
6.5.2 Member Data Documentation . . . 20
6.5.2.1 metadata . . . 20
6.5.2.2 writer_signature . . . 20
6.6 file_metadata_t Struct Reference . . . 20
6.6.1 Detailed Description . . . 21
6.6.2 Member Data Documentation . . . 21
6.6.2.1 encryption_public_key . . . 21
6.6.2.2 file_signature . . . 22
6.6.2.3 file_size . . . 22
6.6.2.4 group_id . . . 22
6.6.2.5 name . . . 22
6.6.2.6 uuid . . . 22
6.7 file_slot_entry_t Struct Reference . . . 23
6.7.1 Detailed Description . . . 24
6.7.2 Member Data Documentation . . . 24
6.7.2.1 encrypted_file . . . 24
6.7.2.2 signed_metadata . . . 24
6.8 group_permissions_t Struct Reference . . . 24
6.8.1 Detailed Description . . . 25
6.8.2 Member Data Documentation . . . 25
6.8.2.1 read . . . 25
6.8.2.2 transfer . . . 25
6.8.2.3 write . . . 25
6.9 group_private_t Struct Reference . . . 26
6.9.1 Detailed Description . . . 26
6.9.2 Member Data Documentation . . . 27
6.9.2.1 permissions . . . 27
6.9.2.2 read_key . . . 27
6.9.2.3 transfer_key . . . 27
6.9.2.4 write_key . . . 27
6.10 group_public_t Struct Reference . . . 28

6.10.1 Detailed Description . . . 28
6.10.2 Member Data Documentation . . . 28
6.10.2.1 read_key . . . 28
6.10.2.2 transfer_key . . . 29
6.10.2.3 write_key . . . 29
6.11 group_t Struct Reference . . . 29
6.11.1 Detailed Description . . . 30
6.11.2 Member Data Documentation . . . 31
6.11.2.1 group_id . . . 31
6.11.2.2 private . . . 31
6.11.2.3 public . . . 31
6.12 message_header_t Struct Reference . . . 31
6.12.1 Detailed Description . . . 32
6.12.2 Member Data Documentation . . . 32
6.12.2.1 message_length . . . 32
6.12.2.2 operation . . . 32
6.13 message_interrogate_file_info_encrypted_t Struct Reference . . . 32
6.13.1 Detailed Description . . . 33
6.13.2 Member Data Documentation . . . 34
6.13.2.1 file_info . . . 34
6.13.2.2 iv . . . 34
6.13.2.3 tag . . . 34
6.14 message_interrogate_file_info_t Struct Reference . . . 34
6.14.1 Detailed Description . . . 35
6.14.2 Member Data Documentation . . . 35
6.14.2.1 files . . . 35
6.14.2.2 num_files . . . 35
6.15 message_interrogate_file_metadata_t Struct Reference . . . 35
6.15.1 Detailed Description . . . 36
6.15.2 Member Data Documentation . . . 36
6.15.2.1 group_id . . . 36
6.15.2.2 name . . . 36
6.15.2.3 slot . . . 36
6.16 message_interrogate_group_list_encrypted_t Struct Reference . . . 37
6.16.1 Detailed Description . . . 37
6.16.2 Member Data Documentation . . . 38
6.16.2.1 group_list . . . 38
6.16.2.2 iv . . . 38
6.16.2.3 tag . . . 38
6.17 message_interrogate_group_list_t Struct Reference . . . 38
6.17.1 Detailed Description . . . 39
6.17.2 Member Data Documentation . . . 39

6.17.2.1 group_ids . . . 39
6.17.2.2 num_groups . . . 39
6.18 message_list_file_entry_t Struct Reference . . . 39
6.18.1 Member Data Documentation . . . 40
6.18.1.1 group_id . . . 40
6.18.1.2 name . . . 40
6.18.1.3 slot . . . 40
6.19 message_list_response_t Struct Reference . . . 41
6.19.1 Detailed Description . . . 41
6.19.2 Member Data Documentation . . . 42
6.19.2.1 entries . . . 42
6.19.2.2 num_files . . . 42
6.20 message_write_header_t Struct Reference . . . 42
6.20.1 Detailed Description . . . 43
6.20.2 Member Data Documentation . . . 43
6.20.2.1 file_length . . . 43
6.20.2.2 file_uuid . . . 43
6.20.2.3 group_id . . . 43
6.20.2.4 name . . . 43
6.20.2.5 pin . . . 44
6.20.2.6 slot . . . 44

7 File Documentation **45**
7.1 /workspaces/2026-ectf-osu-design/docs/design.md File Reference . . . 45
7.2 /workspaces/2026-ectf-osu-design/docs/index.md File Reference . . . 45
7.3 /workspaces/2026-ectf-osu-design/docs/misc.md File Reference . . . 45
7.4 include/async_uart.h File Reference . . . 45
7.4.1 Function Documentation . . . 46
7.4.1.1 async_uart_receive() . . . 46
7.5 async_uart.h . . . 47
7.6 include/constants.h File Reference . . . 47
7.6.1 Macro Definition Documentation . . . 48
7.6.1.1 CHUNK_SIZE . . . 48
7.6.1.2 MAX_FILE_SIZE . . . 48
7.6.1.3 MAX_GROUPS . . . 48
7.6.1.4 NUM_SLOTS . . . 48
7.6.1.5 PIN_DELAY . . . 48
7.6.1.6 WRITE_HEADER_SIZE . . . 49
7.7 constants.h . . . 49
7.8 include/datastructures.h File Reference . . . 49
7.9 datastructures.h . . . 50
7.10 include/file.h File Reference . . . 50

7.10.1 Macro Definition Documentation . . . 52
7.10.1.1 FLASH_SECTOR_SIZE . . . 52
7.10.1.2 FLASH_WORD_SIZE . . . 52
7.10.1.3 MAX_PADDING_BACK . . . 52
7.10.2 Function Documentation . . . 52
7.10.2.1 file_read_fat() . . . 52
7.10.2.2 file_write_fat() . . . 53
7.10.3 Variable Documentation . . . 54
7.10.3.1 FILE_ADDRESS_TABLE . . . 54
7.10.3.2 SLOTS . . . 54
7.11 file.h . . . 55
7.12 include/group.h File Reference . . . 55
7.13 group.h . . . 56
7.14 include/message/header.h File Reference . . . 57
7.14.1 Enumeration Type Documentation . . . 58
7.14.1.1 message_operation_t . . . 58
7.14.2 Function Documentation . . . 59
7.14.2.1 message_header_receive_ack() . . . 59
7.14.2.2 message_header_request() . . . 59
7.14.2.3 message_header_response() . . . 60
7.14.2.4 message_header_send_ack() . . . 61
7.14.2.5 message_header_send_debug() . . . 62
7.14.2.6 message_header_send_error() . . . 63
7.15 header.h . . . 64
7.16 include/message/interrogate.h File Reference . . . 64
7.16.1 Function Documentation . . . 66
7.16.1.1 message_interrogate() . . . 66
7.17 interrogate.h . . . 67
7.18 include/message/list.h File Reference . . . 68
7.18.1 Function Documentation . . . 69
7.18.1.1 message_list_response() . . . 69
7.19 list.h . . . 70
7.20 include/message/listen.h File Reference . . . 71
7.20.1 Function Documentation . . . 71
7.20.1.1 message_listen() . . . 71
7.21 listen.h . . . 72
7.22 include/message/read.h File Reference . . . 73
7.22.1 Function Documentation . . . 73
7.22.1.1 message_read_response() . . . 73
7.23 read.h . . . 74
7.24 include/message/receive.h File Reference . . . 75
7.24.1 Function Documentation . . . 75

7.24.1.1 message_recieve() . . . 75
7.25 receive.h . . . 76
7.26 include/message/write.h File Reference . . . 77
7.26.1 Function Documentation . . . 78
7.26.1.1 message_write_parse_header() . . . 78
7.26.1.2 message_write_response() . . . 78
7.27 write.h . . . 79
7.28 include/pin_utils.h File Reference . . . 80
7.28.1 Function Documentation . . . 81
7.28.1.1 init_pin() . . . 81
7.28.1.2 validate_pin() . . . 81
7.29 pin_utils.h . . . 82
7.30 include/random.h File Reference . . . 83
7.30.1 Function Documentation . . . 83
7.30.1.1 random_fill_buffer() . . . 83
7.31 random.h . . . 84
7.32 include/secrets.h File Reference . . . 84
7.32.1 Macro Definition Documentation . . . 86
7.32.1.1 STAGE1_KEY . . . 86
7.32.1.2 STAGE1_PADDING . . . 86
7.32.1.3 STAGE1_PIN_HASH . . . 86
7.32.1.4 STAGE1_PIN_ITERATIONS . . . 86
7.32.1.5 STAGE2_RAND_INT . . . 86
7.32.2 Variable Documentation . . . 86
7.32.2.1 groups . . . 86
7.32.2.2 TRANSFER_KEY . . . 87
7.33 secrets.h . . . 87
7.34 include/ti_msp_dl_config.h File Reference . . . 87
7.34.1 Macro Definition Documentation . . . 89
7.34.1.1 CONFIG_LP_MSPM0L2228 . . . 89
7.34.1.2 CONFIG_MSPM0L2228 . . . 89
7.34.1.3 CPUCLK_FREQ . . . 89
7.34.1.4 GPIO_HOST_IOMUX_RX . . . 89
7.34.1.5 GPIO_HOST_IOMUX_RX_FUNC . . . 89
7.34.1.6 GPIO_HOST_IOMUX_TX . . . 89
7.34.1.7 GPIO_HOST_IOMUX_TX_FUNC . . . 89
7.34.1.8 GPIO_HOST_RX_PIN . . . 90
7.34.1.9 GPIO_HOST_RX_PORT . . . 90
7.34.1.10 GPIO_HOST_TX_PIN . . . 90
7.34.1.11 GPIO_HOST_TX_PORT . . . 90
7.34.1.12 GPIO_HSM_IOMUX_RX . . . 90
7.34.1.13 GPIO_HSM_IOMUX_RX_FUNC . . . 90

7.34.1.14 GPIO_HSM_IOMUX_TX . . . 90
7.34.1.15 GPIO_HSM_IOMUX_TX_FUNC . . . 90
7.34.1.16 GPIO_HSM_RX_PIN . . . 90
7.34.1.17 GPIO_HSM_RX_PORT . . . 90
7.34.1.18 GPIO_HSM_TX_PIN . . . 91
7.34.1.19 GPIO_HSM_TX_PORT . . . 91
7.34.1.20 HOST_BAUD_RATE . . . 91
7.34.1.21 HOST_FBRD_32_MHZ_115200_BAUD . . . 91
7.34.1.22 HOST_IBRD_32_MHZ_115200_BAUD . . . 91
7.34.1.23 HOST_INST . . . 91
7.34.1.24 HOST_INST_FREQUENCY . . . 91
7.34.1.25 HOST_INST_INT_IRQN . . . 91
7.34.1.26 HOST_INST_IRQHandler . . . 91
7.34.1.27 HSM_BAUD_RATE . . . 91
7.34.1.28 HSM_FBRD_32_MHZ_115200_BAUD . . . 92
7.34.1.29 HSM_IBRD_32_MHZ_115200_BAUD . . . 92
7.34.1.30 HSM_INST . . . 92
7.34.1.31 HSM_INST_FREQUENCY . . . 92
7.34.1.32 HSM_INST_INT_IRQN . . . 92
7.34.1.33 HSM_INST_IRQHandler . . . 92
7.34.1.34 POWER_STARTUP_DELAY . . . 92
7.34.1.35 TRNG_INT_IIDX . . . 92
7.34.1.36 TRNG_INT_IRQN . . . 92
7.34.1.37 WWDT0_INST . . . 92
7.34.1.38 WWDT0_INT_IRQN . . . 93
7.34.2 Function Documentation . . . 93
7.34.2.1 SYSCFG_DL_GPIO_init() . . . 93
7.34.2.2 SYSCFG_DL_HOST_init() . . . 93
7.34.2.3 SYSCFG_DL_HSM_init() . . . 93
7.34.2.4 SYSCFG_DL_init() . . . 93
7.34.2.5 SYSCFG_DL_initPower() . . . 93
7.34.2.6 SYSCFG_DL_SYSCTL_init() . . . 94
7.34.2.7 SYSCFG_DL_TRNG_init() . . . 94
7.34.2.8 SYSCFG_DL_WWDT0_init() . . . 94
7.35 ti_msp_dl_config.h . . . 94
7.36 include/utils.h File Reference . . . 96
7.36.1 Function Documentation . . . 97
7.36.1.1 utils_find_group() . . . 97
7.36.1.2 utils_random_delay() . . . 98
7.36.1.3 utils_receive_bytes() . . . 98
7.36.1.4 utils_send_buffer() . . . 99
7.36.1.5 utils_send_buffer_no_ack() . . . 100

7.36.1.6 utils_verify_pin() . . . 101
7.37 utils.h . . . 102
7.38 src/async_uart.c File Reference . . . 102
7.38.1 Function Documentation . . . 103
7.38.1.1 async_uart_receive() . . . 103
7.38.1.2 DMA_IRQHandler() . . . 104
7.38.2 Variable Documentation . . . 104
7.38.2.1 transfers . . . 104
7.39 src/file.c File Reference . . . 104
7.39.1 Function Documentation . . . 105
7.39.1.1 file_read_fat() . . . 105
7.39.1.2 file_write_fat() . . . 106
7.39.2 Variable Documentation . . . 107
7.39.2.1 FILE_ADDRESS_TABLE . . . 107
7.39.2.2 SLOTS . . . 107
7.40 src/main.c File Reference . . . 108
7.40.1 Function Documentation . . . 108
7.40.1.1 HardFault_Handler() . . . 108
7.40.1.2 main() . . . 109
7.40.1.3 NMI_Handler() . . . 110
7.40.1.4 UART0_IRQHandler() . . . 110
7.41 src/message/header.c File Reference . . . 110
7.41.1 Function Documentation . . . 111
7.41.1.1 message_header_receive_ack() . . . 111
7.41.1.2 message_header_request() . . . 112
7.41.1.3 message_header_response() . . . 113
7.41.1.4 message_header_send_ack() . . . 114
7.41.1.5 message_header_send_debug() . . . 115
7.41.1.6 message_header_send_error() . . . 116
7.42 src/message/interrogate.c File Reference . . . 117
7.42.1 Function Documentation . . . 117
7.42.1.1 message_interrogate() . . . 117
7.43 src/message/list.c File Reference . . . 119
7.43.1 Function Documentation . . . 120
7.43.1.1 message_list_response() . . . 120
7.44 src/message/listen.c File Reference . . . 120
7.44.1 Macro Definition Documentation . . . 121
7.44.1.1 DMA_CHANNEL . . . 121
7.44.2 Function Documentation . . . 121
7.44.2.1 message_listen() . . . 121
7.45 src/message/read.c File Reference . . . 122
7.45.1 Macro Definition Documentation . . . 123

7.45.1.1 DMA_CHANNEL . . . 123
7.45.1.2 READ_HEADER_LENGTH . . . 123
7.45.2 Function Documentation . . . 124
7.45.2.1 message_read_response() . . . 124
7.45.3 Variable Documentation . . . 124
7.45.3.1 buffers . . . 124
7.46 src/message/receive.c File Reference . . . 125
7.46.1 Function Documentation . . . 125
7.46.1.1 message_recieve() . . . 125
7.47 src/message/write.c File Reference . . . 126
7.47.1 Macro Definition Documentation . . . 127
7.47.1.1 DMA_CHANNEL . . . 127
7.47.2 Function Documentation . . . 127
7.47.2.1 message_write_parse_header() . . . 127
7.47.2.2 message_write_response() . . . 128
7.47.3 Variable Documentation . . . 129
7.47.3.1 buffers . . . 129
7.48 src/pin_utils.c File Reference . . . 129
7.48.1 Macro Definition Documentation . . . 130
7.48.1.1 COMPILER_FUNC_OPTS . . . 130
7.48.1.2 RANDOM_FUNC . . . 131
7.48.1.3 SEED . . . 131
7.48.2 Function Documentation . . . 131
7.48.2.1 init_pin() . . . 131
7.48.2.2 validate_pin() . . . 131
7.49 src/random.c File Reference . . . 132
7.49.1 Function Documentation . . . 133
7.49.1.1 random_fill_buffer() . . . 133
7.50 src/secrets.c File Reference . . . 133
7.50.1 Variable Documentation . . . 134
7.50.1.1 groups . . . 134
7.50.1.2 TRANSFER_KEY . . . 134
7.51 src/utils.c File Reference . . . 134
7.51.1 Function Documentation . . . 135
7.51.1.1 utils_find_group() . . . 135
7.51.1.2 utils_random_delay() . . . 136
7.51.1.3 utils_receive_bytes() . . . 137
7.51.1.4 utils_send_buffer() . . . 138
7.51.1.5 utils_send_buffer_no_ack() . . . 139
7.51.1.6 utils_verify_pin() . . . 140

Index **141**

Chapter 1

Overview

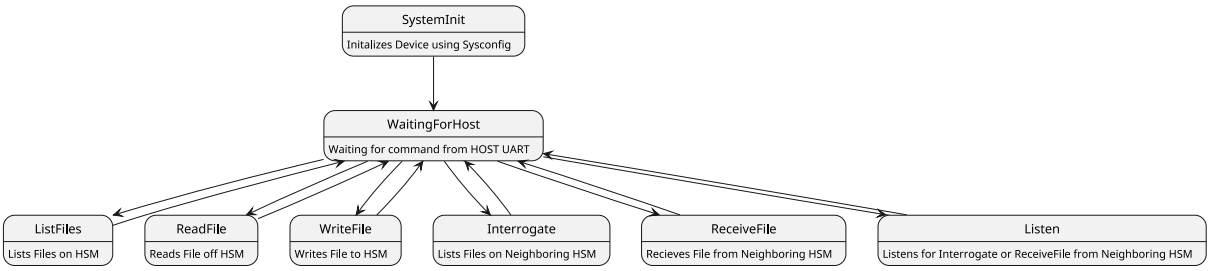
The goal for our design was to use asymetric encryption to ensure that secrets to read, write, and transfer files are only provisioned on HSMs with valid permissions to perform the operation. We used monocypher's implementation of ED25519 and EdDSA to perform our asymetric cryptographic operations and the MSPM0l2228's hardware AES for accelerated encryption and decryption.

1.1 Contents

- High Level Design
- Miscellaneous Implementation Details
- Read Command
- Write Command
- List Command
- Interrogate Command
- Recieve Command
- Listen Command

Chapter 2

High Level Design



2.0.1 Pin management

Problem statement:

- Assume that any user input at any time could cause an arbitrary memory read
  - Meaning the moment uart is open, assume any clear text secrets are vulnerable
- They keystore is out of scope, no truly secure places exist to hide data
- The timing requirements, combined with the slow CPU, mean that any hash we use is bound to be insecure
- The pin is defined by 6 hexadecimal digits, or 16,777,216 combinations
  - This is small enough simply divulging an easily computable hash is akin to the pin itself being leaked

Proposed mitigations:

- Hashing should be done with BLAKE2b loops, easily tuned for security versus effort
- Double hash system:
  - First phase is PIN + key only, bulk of the hash work
    - The key in the generate_secrets.py.
  - Second phase is done on device, after boot with much device specific data.
    - Data included:
      - Use the BOOTCRC from FACTORYREGION, this is a crc of all the device specific values, uuids and calibration ids.
      - A second IV generated at boot with the TRNG module
      - More random data generated in generate_secrets.py, also randomly put in the code segment via C macros
    - This will not be done with multiple iterations, and instead rely on high input amounts and low data continuity.
  - In general, the hashing input must be spread throughout ram, requirng attacks to use multiple arbitrary read attacks to obtain

2.1 Groups

Every group has 3 permissions and each permission has a corresponding asymetric key pair. This means every group has 3 different key pairs. An HSM has the permission for a group if it is provisioned with the private key for a permission in a group. **All HSMs are provisioned with all public keys.** This is important to remember for later.

Group	
Read	Private Key
	Public Key
Write	Private Key
	Public Key
Transfer	Private Key
	Public Key

2.2 File Storage

Files are stored using the file_slot_entry_t struct.

2.2.1 File Encryption

When a file is uploaded to the board a random X25519 Key Pair is generated from the TRNG. A X25519 Key Exchange done with this randomly generated key and the public key for the file's group. This generated secret is then used to encrypt the file using AES-GCM. After the file is encrypted the private portion of the key pair and the generated secret key are erased from memory. The public key of the randomly generated key pair is then stored in the metadata for the file. Since the randomly generated private key has been erased the only way the AES secret can be regenerated is using the file groups read private key and the public key stored in the files metadata. The file should only be able to be decrypted by HSMs that have the read permission for the file's group (Only HSMs with the read permission have the read private key).

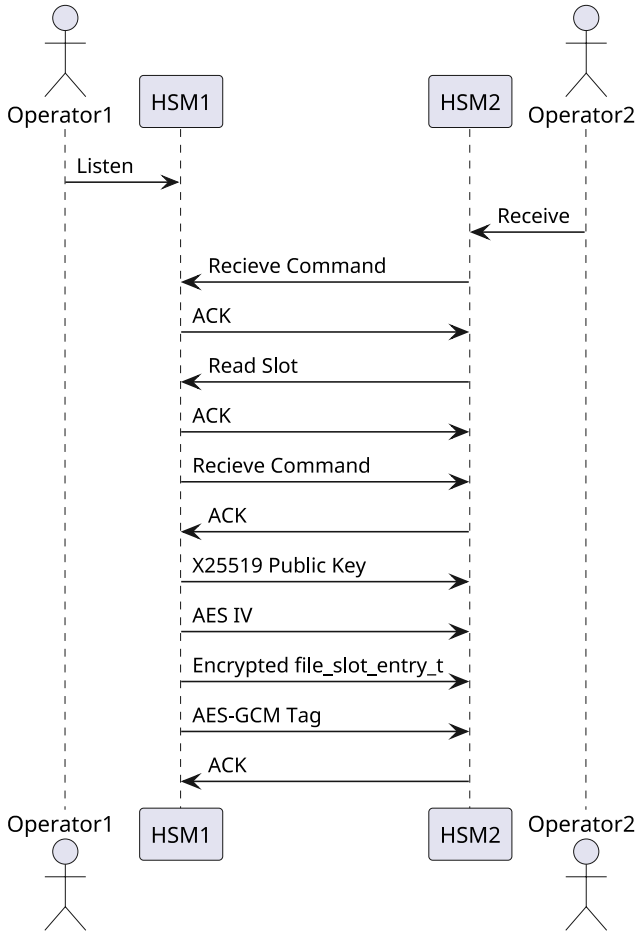
NOTE: Anyone can encrypt the file using the read public key so we must perform verification using another method.

2.2.2 File Verification

Files are only supposed to be written by HSMs that have the write permission for a group (and the coresponding write private key). To verify that the file was written by a valid writer we can sign the files metadata using ED25519. Any HSM can verify the metadata using the write public key for the coresponding group.

2.3 File Transfer

When a transfer is initated the sender starts by generating a random X25519 Key Pair. The public key of the randomly generated pair should then be sent to receiving HSM. After generating the random key pair a secret should be generated using the randomly generated private key and the group's transfer public key that the sender has stored internally. The entire file_slot_entry_t struct should be encrypted using the generated secret and sent over UART. If the receiving HSM has the transfer private key / transfer permission they will be able to regenerate the secret using the transfer private key and the public key that was provided via UART.



NOTE: This is a very high level overview, some metadata will have to be exchanged.

Chapter 3

Miscellaneous Details

This page details some of the implementation specifics that didn't really fit into any other section.

3.1 Stalls

In our design we have configured the WWDT for 4 seconds. The timer is only reset in the main event loop so if a function gets stuck at any point the watchdog will expire and trigger a reset. This makes any issues with incomplete messages or other errors recoverable.

3.2 UART Errors

The UART peripheral has the ability to detect frame errors, overrrun errors, and noise errors. We have enabled the interrupts for all of these exceptions as any of these errors would be breaking for our implementation. This allows errors to be caught early and helps diagnose issues that would simply arise later in the testing of our design.

3.3 Random Data

All of the random data generated in our design comes from the random_fill_buffer function which uses the true random number generator on the MSPM0l2228 chip.

3.4 File System Layout

The FAT and file system are represented using structs that are placed in a section that is in the flash memory. These structs cannot be written to in a traditional sense but can be written using the FlashCTL. I like this design because it allows us to reference files using a simple C array and the write address can be obtained using the & operation. For more information on the filesystem itself you can look at the FILE_ADDRESS_TABLE variable and the SLOTS variable. For information of the layout of each entry in the FAT and file system you can take a look at the file_address_table_entry and the file_slot_entry_t structs respectively.

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

async_uart_ctx
Async UART context structure . . . 13
BufferNode
Buffer node for linked list of data chunks . . . 15
file_address_table_t
Padded File Address Table . . . 17
file_fat_entry_t
Required FAT Structure . . . 18
file_metadata_signed_t
Signed File Metadata . . . 19
file_metadata_t
Full File Metadata . . . 20
file_slot_entry_t
Structure of data in each "slot" of memory . . . 23
group_permissions_t
Group Permissions Structure . . . 24
group_private_t
Group Private Keys Structure . . . 26
group_public_t
Group Public Keys Structure . . . 28
group_t
Group Structure as Stored on Flash . . . 29
message_header_t
Message header structure . . . 31
message_interrogate_file_info_encrypted_t
Encrypted interrogation file info structure . . . 32
message_interrogate_file_info_t
Interrogated file info structure (aligned to AES block size) . . . 34
message_interrogate_file_metadata_t
Interrogated file metadata structure (packed) . . . 35
message_interrogate_group_list_encrypted_t
Encrypted interrogated group list structure . . . 37
message_interrogate_group_list_t
Interrogated group list structure (aligned to AES block size) . . . 38
message_list_file_entry_t . . . 39

message_list_response_t
Packed list response structure 41
message_write_header_t
Write file command header structure 42

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/async_uart.h . . . 45
include/constants.h . . . 47
include/datastructures.h . . . 49
include/file.h . . . 50
include/group.h . . . 55
include/pin_utils.h . . . 80
include/random.h . . . 83
include/secrets.h . . . 84
include/ti_msp_dl_config.h . . . 87
include/utils.h . . . 96
include/message/header.h . . . 57
include/message/interrogate.h . . . 64
include/message/list.h . . . 68
include/message/listen.h . . . 71
include/message/read.h . . . 73
include/message/receive.h . . . 75
include/message/write.h . . . 77
src/async_uart.c . . . 102
src/file.c . . . 104
src/main.c . . . 108
src/pin_utils.c . . . 129
src/random.c . . . 132
src/secrets.c . . . 133
src/utils.c . . . 134
src/message/header.c . . . 110
src/message/interrogate.c . . . 117
src/message/list.c . . . 119
src/message/listen.c . . . 120
src/message/read.c . . . 122
src/message/receive.c . . . 125
src/message/write.c . . . 126

Chapter 6

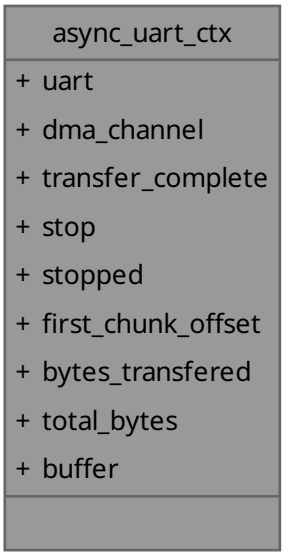
Class Documentation

6.1 async_uart_ctx Struct Reference

Async UART context structure.

```
#include <async_uart.h>
```

Collaboration diagram for async_uart_ctx:



Public Attributes

- UART_Regs * uart

  UART peripheral registers.

- uint8_t dma_channel

  DMA channel used for transfer.
- bool transfer_complete

  Flag indicating transfer completion.
- bool stop

  Stop transfer request flag.
- bool stopped

  Flag indicating transfer has stopped.
- size_t first_chunk_offset

  Offset of first chunk in buffer (for ACKing)
- size_t bytes_transfered

  Number of bytes transferred so far.
- size_t total_bytes

  Total number of bytes to transfer.
- uint8_t * buffer

  Buffer for data transfer.

6.1.1 Detailed Description

Async UART context structure.

6.1.2 Member Data Documentation

6.1.2.1 buffer

```
uint8_t* async_uart_ctx::buffer
```

Buffer for data transfer.

6.1.2.2 bytes_transfered

```
size_t async_uart_ctx::bytes_transfered
```

Number of bytes transferred so far.

6.1.2.3 dma_channel

```
uint8_t async_uart_ctx::dma_channel
```

DMA channel used for transfer.

6.1.2.4 first_chunk_offset

```
size_t async_uart_ctx::first_chunk_offset
```

Offset of first chunk in buffer (for ACKing)

6.1.2.5 stop

```
bool async_uart_ctx::stop
```

Stop transfer request flag.

6.1.2.6 stopped

```
bool async_uart_ctx::stopped
```

Flag indicating transfer has stopped.

6.1.2.7 total_bytes

```
size_t async_uart_ctx::total_bytes
```

Total number of bytes to transfer.

6.1.2.8 transfer_complete

```
bool async_uart_ctx::transfer_complete
```

Flag indicating transfer completion.

6.1.2.9 uart

```
UART_Regs* async_uart_ctx::uart
```

UART peripheral registers.

The documentation for this struct was generated from the following file:

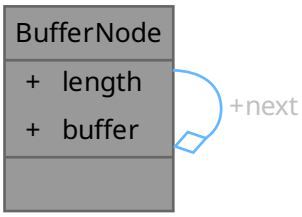
- include/async_uart.h

6.2 BufferNode Struct Reference

Buffer node for linked list of data chunks.

```
#include <datastructures.h>
```

Collaboration diagram for BufferNode:



Public Attributes

- struct BufferNode * next

  Pointer to next node in list.
- uint16_t length

  Length of data in buffer.
- uint32_t buffer [256/4]

  Data buffer (CHUNK_SIZE bytes)

6.2.1 Detailed Description

Buffer node for linked list of data chunks.

6.2.2 Member Data Documentation

6.2.2.1 buffer

```
uint32_t BufferNode::buffer[256/4]
```

Data buffer (CHUNK_SIZE bytes)

6.2.2.2 length

```
uint16_t BufferNode::length
```

Length of data in buffer.

6.2.2.3 next

```
struct BufferNode* BufferNode::next
```

Pointer to next node in list.

The documentation for this struct was generated from the following file:

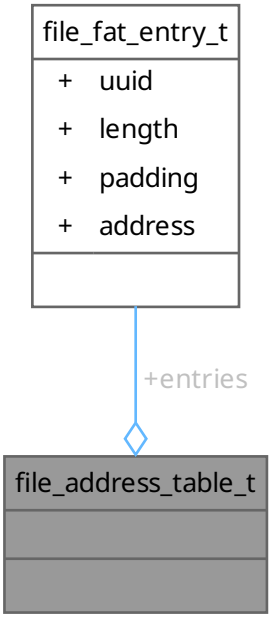
- include/datastructures.h

6.3 file_address_table_t Struct Reference

Padded File Address Table.

`#include <file.h>`

Collaboration diagram for file_address_table_t:



Public Attributes

- file_fat_entry_t entries [8]

6.3.1 Detailed Description

Padded File Address Table.

This struct is used to pad the array to a whole number of sectors. My understanding is that it isn't possible to pad C arrays so this acts as a little work around.

6.3.2 Member Data Documentation

6.3.2.1 entries

`file_fat_entry_t file_address_table_t::entries[8]`

The documentation for this struct was generated from the following file:

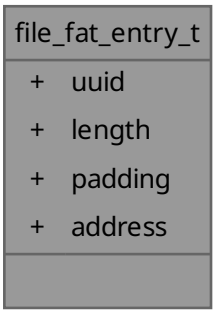
- include/file.h

6.4 file_fat_entry_t Struct Reference

Required FAT Structure.

```
#include <file.h>
```

Collaboration diagram for file_fat_entry_t:



Public Attributes

- uint8_t uuid [16]

  UUID of the file in that slot.
- uint16_t length

  Length of the file.
- uint16_t padding

  Unused to pad to 32-bit alignment.
- uint32_t address

  Starting flash address of the file.

6.4.1 Detailed Description

Required FAT Structure.

6.4.2 Member Data Documentation

6.4.2.1 address

```
uint32_t file_fat_entry_t::address
```

Starting flash address of the file.

6.4.2.2 length

`uint16_t file_fat_entry_t::length`

Length of the file.

6.4.2.3 padding

`uint16_t file_fat_entry_t::padding`

Unused to pad to 32-bit alignment.

6.4.2.4 uuid

`uint8_t file_fat_entry_t::uuid[16]`

UUID of the file in that slot.

The documentation for this struct was generated from the following file:

- include/file.h

6.5 file_metadata_signed_t Struct Reference

Signed File Metadata.

`#include <file.h>`

Collaboration diagram for file_metadata_signed_t:

Public Attributes

- file_metadata_t metadata

  File Metadata.
- uint8_t writer_signature [64]

  Ed25519 Signature from File Writer.

6.5.1 Detailed Description

Signed File Metadata.

Signing the file metadata with the group writer key allows us to validate that the file was written by an HSM with the write permission for that group. Only HSMs with the write permission for a group have the private key.

6.5.2 Member Data Documentation

6.5.2.1 metadata

```
file_metadata_t file_metadata_signed_t::metadata
```

File Metadata.

6.5.2.2 writer_signature

```
uint8_t file_metadata_signed_t::writer_signature[64]
```

Ed25519 Signature from File Writer.

The documentation for this struct was generated from the following file:

- include/file.h

6.6 file_metadata_t Struct Reference

Full File Metadata.

```
#include <file.h>
```

Collaboration diagram for file_metadata_t:

Public Attributes

- uint8_t uuid [16]

  File UUID.
- uint16_t group_id

  File Group ID.
- char name [32]

  File Name.
- uint16_t file_size

  File Size.
- uint32_t file_signature [4]

  File AES-GCM Tag.
- uint8_t encryption_public_key [32]

6.6.1 Detailed Description

Full File Metadata.

To generate the file's encryption key you must permform an X25519 Key Exchange using the public key from the metadata and the private read key from the file's group.

6.6.2 Member Data Documentation

6.6.2.1 encryption_public_key

```
uint8_t file_metadata_t::encryption_public_key[32]
```

This is the public key of a keypair generated for this file specifically.

6.6.2.2 file_signature

```
uint32_t file_metadata_t::file_signature[4]
```

File AES-GCM Tag.

6.6.2.3 file_size

```
uint16_t file_metadata_t::file_size
```

File Size.

6.6.2.4 group_id

```
uint16_t file_metadata_t::group_id
```

File Group ID.

6.6.2.5 name

```
char file_metadata_t::name[32]
```

File Name.

6.6.2.6 uuid

```
uint8_t file_metadata_t::uuid[16]
```

File UUID.

The documentation for this struct was generated from the following file:

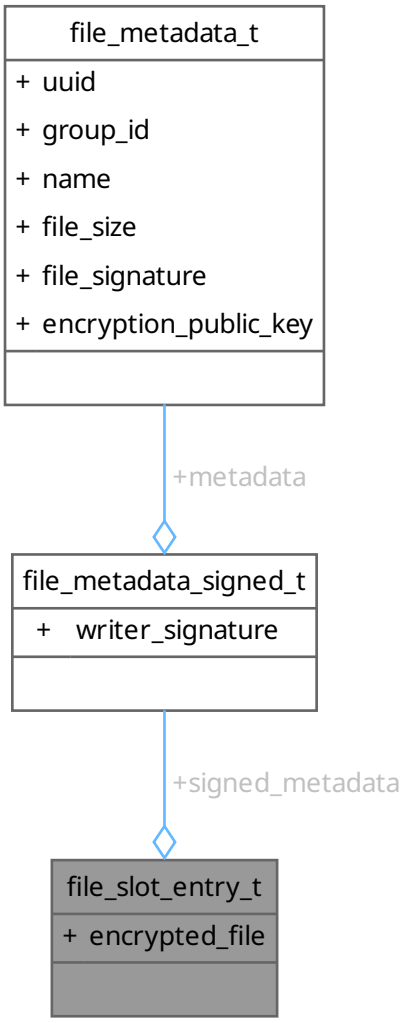
- include/file.h

6.7 file_slot_entry_t Struct Reference

Structure of data in each "slot" of memory.

```
#include <file.h>
```

Collaboration diagram for file_slot_entry_t:



Public Attributes

- file_metadata_signed_t signed_metadata

  Signed File Metadata.
- uint8_t encrypted_file [8192+(16 - 8192 % 16)]

  Encrypted File with Padding using AES-GCM.

6.7.1 Detailed Description

Structure of data in each "slot" of memory.

This struct is designed to be written to flash so we have to change how the memory alignment works. Each member of this struct is aligned to the flash word size (8 bytes / 64 bits). This allows us to write each structure seperatly without having to worry about ECC which is automatically generated for flash word writes. The entire structure is aligned to the flash sector size (1024 bytes) so each entry can be cleanly erased without having to write back any of the other members in an array.

6.7.2 Member Data Documentation

6.7.2.1 encrypted_file

```
uint8_t file_slot_entry_t::encrypted_file[8192+(16 - 8192 % 16)]
```

Encrypted File with Padding using AES-GCM.

6.7.2.2 signed_metadata

```
file_metadata_signed_t file_slot_entry_t::signed_metadata
```

Signed File Metadata.

The documentation for this struct was generated from the following file:

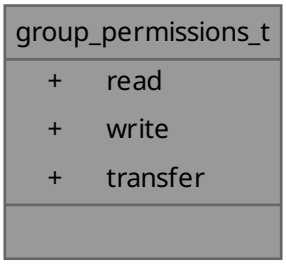
- include/file.h

6.8 group_permissions_t Struct Reference

Group Permissions Structure.

```
#include <group.h>
```

Collaboration diagram for group_permissions_t:



Public Attributes

- bool read

  Read permission flag.
- bool write

  Write permission flag.
- bool transfer

  Transfer permission flag.

6.8.1 Detailed Description

Group Permissions Structure.

Group permissions structure

6.8.2 Member Data Documentation

6.8.2.1 read

```
bool group_permissions_t::read
```

Read permission flag.

6.8.2.2 transfer

```
bool group_permissions_t::transfer
```

Transfer permission flag.

6.8.2.3 write

```
bool group_permissions_t::write
```

Write permission flag.

The documentation for this struct was generated from the following file:

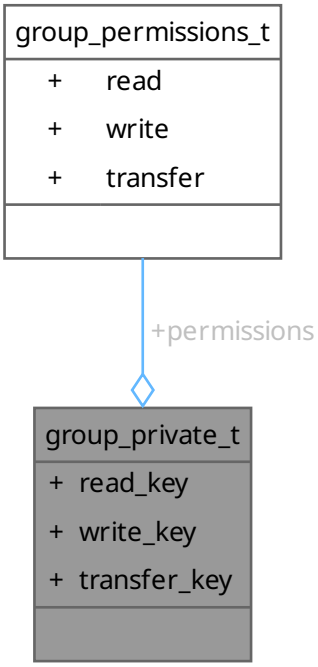
- include/group.h

6.9 group_private_t Struct Reference

Group Private Keys Structure.

`#include <group.h>`

Collaboration diagram for group_private_t:



Public Attributes

- group_permissions_t permissions

  Group permissions.
- uint8_t read_key [32]

  Group read private key (32 bytes)
- uint8_t write_key [64]

  Group write private key (64 bytes for Ed25519)
- uint8_t transfer_key [32]

  Group transfer private key (32 bytes)

6.9.1 Detailed Description

Group Private Keys Structure.

In our current implementation all the keys should be the same size and format even though we are using different algorithms in mbedtls. All the keys should be 32 bytes (storing 255 bits). A provisioned HSM will only have the keys for permissions that the HSM has. Eg. if an HSM is provisioned for Reading and Transfering for a group it will only the read_key and transfer_key fields will be valid in the corresponding group structure.

For more information refer to the `PSA Crypto API`

Precondition

read_key is valid if and only if permissions.read is true

write_key is valid if and only if permissions.write is true

transfer_key is valid if and only if permissions.transfer is true

Group private keys structure

Each group has separate private keys for different permissions. Only keys for granted permissions are stored.

6.9.2 Member Data Documentation

6.9.2.1 permissions

```
group_permissions_t group_private_t::permissions
```

Group permissions.

6.9.2.2 read_key

```
uint8_t group_private_t::read_key[32]
```

Group read private key (32 bytes)

6.9.2.3 transfer_key

```
uint8_t group_private_t::transfer_key[32]
```

Group transfer private key (32 bytes)

6.9.2.4 write_key

```
uint8_t group_private_t::write_key[64]
```

Group write private key (64 bytes for Ed25519)

The documentation for this struct was generated from the following file:

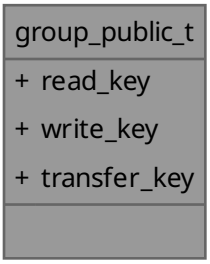
- include/group.h

6.10 group_public_t Struct Reference

Group Public Keys Structure.

```
#include <group.h>
```

Collaboration diagram for group_public_t:



Public Attributes

- uint8_t read_key [32]

  Group read public key (32 bytes for X25519)
- uint8_t write_key [32]

  Group write public key (32 bytes for Ed25519)
- uint8_t transfer_key [32]

  Group transfer public key (32 bytes)

6.10.1 Detailed Description

Group Public Keys Structure.

Every provisioned HSM must contain the public keys for all the permissions of every group.

Group public keys structure

Every provisioned HSM must contain the public keys for all permissions of every group.

6.10.2 Member Data Documentation

6.10.2.1 read_key

```
uint8_t group_public_t::read_key[32]
```

Group read public key (32 bytes for X25519)

6.10.2.2 transfer_key

```
uint8_t group_public_t::transfer_key[32]
```

Group transfer public key (32 bytes)

6.10.2.3 write_key

```
uint8_t group_public_t::write_key[32]
```

Group write public key (32 bytes for Ed25519)

The documentation for this struct was generated from the following file:

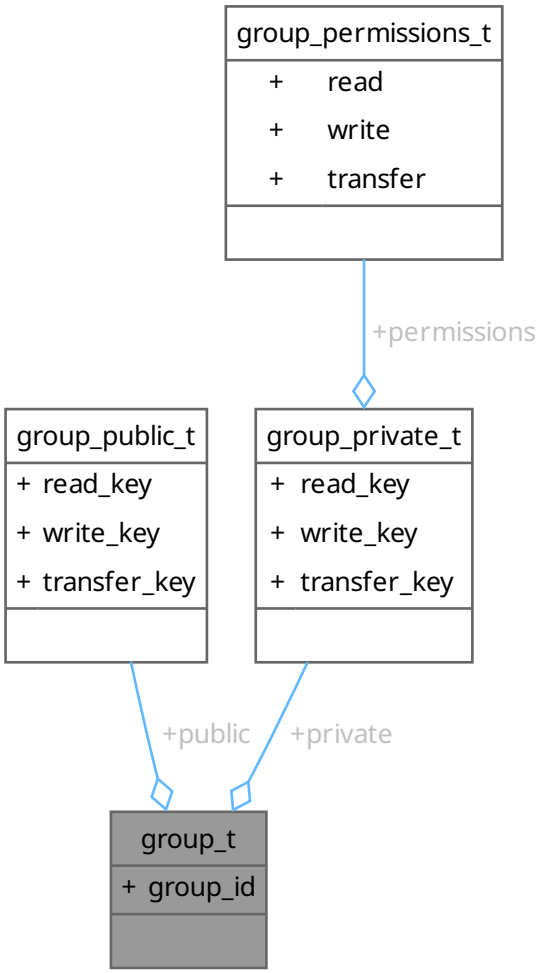
- include/group.h

6.11 group_t Struct Reference

Group Structure as Stored on Flash.

```
#include <group.h>
```

Collaboration diagram for group_t:



Public Attributes

- uint16_t group_id

  Group identifier.
- group_public_t public

  Group public keys.
- group_private_t private

  Group private keys.

6.11.1 Detailed Description

Group Structure as Stored on Flash.

A const array of this structure should be generated by the host tools for each HSM. The const qualifier tells the compiler to store the data in the .rodata section on the flash as stated in the `TI Arm Clang Compiler Tools User's Guide`.

6.11.2 Member Data Documentation

6.11.2.1 group_id

```
uint16_t group_t::group_id
```

Group identifier.

6.11.2.2 private

```
group_private_t group_t::private
```

Group private keys.

6.11.2.3 public

```
group_public_t group_t::public
```

Group public keys.

The documentation for this struct was generated from the following file:

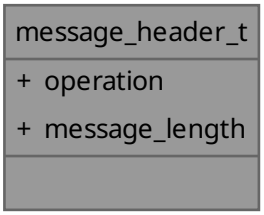
- include/group.h

6.12 message_header_t Struct Reference

Message header structure.

```
#include <header.h>
```

Collaboration diagram for message_header_t:



Public Attributes

- message_operation_t operation

  Message operation code.
- uint16_t message_length

  Length of message payload in bytes.

6.12.1 Detailed Description

Message header structure.

All messages begin with '' followed by operation code and message length.

6.12.2 Member Data Documentation

6.12.2.1 message_length

```
uint16_t message_header_t::message_length
```

Length of message payload in bytes.

6.12.2.2 operation

```
message_operation_t message_header_t::operation
```

Message operation code.

The documentation for this struct was generated from the following file:

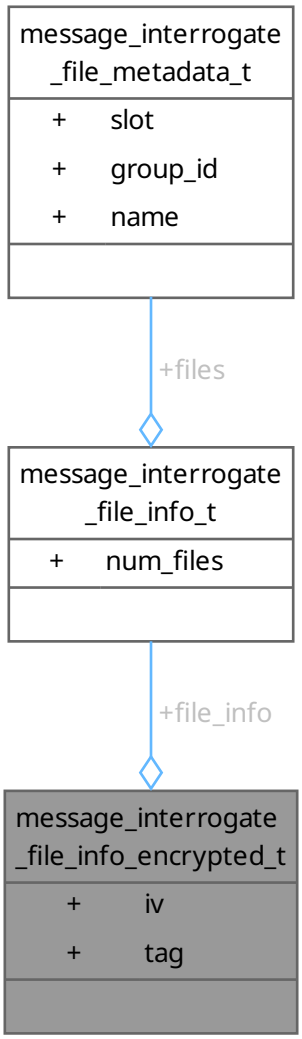
- include/message/header.h

6.13 message_interrogate_file_info_encrypted_t Struct Reference

Encrypted interrogation file info structure.

```
#include <interrogate.h>
```

Collaboration diagram for message_interrogate_file_info_encrypted_t:



Public Attributes

- uint32_t iv [4]

  IV for AES-GCM encryption (128 bits)
- message_interrogate_file_info_t file_info

  File info data.
- uint32_t tag [4]

  Authentication tag (128 bits)

6.13.1 Detailed Description

Encrypted interrogation file info structure.

6.13.2 Member Data Documentation

6.13.2.1 file_info

`message_interrogate_file_info_t message_interrogate_file_info_encrypted_t::file_info`

File info data.

6.13.2.2 iv

`uint32_t message_interrogate_file_info_encrypted_t::iv[4]`

IV for AES-GCM encryption (128 bits)

6.13.2.3 tag

`uint32_t message_interrogate_file_info_encrypted_t::tag[4]`

Authentication tag (128 bits)

The documentation for this struct was generated from the following file:

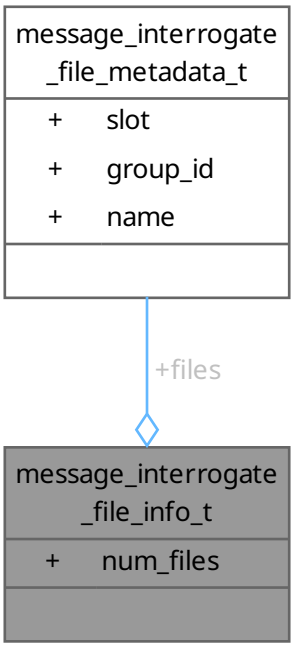
- include/message/interrogate.h

6.14 message_interrogate_file_info_t Struct Reference

Interrogated file info structure (aligned to AES block size)

`#include <interrogate.h>`

Collaboration diagram for message_interrogate_file_info_t:



Public Attributes

- uint32_t num_files

  Number of files available.
- message_interrogate_file_metadata_t files [8]

  Array of file metadata entries.

6.14.1 Detailed Description

Interrogated file info structure (aligned to AES block size)

6.14.2 Member Data Documentation

6.14.2.1 files

```
message_interrogate_file_metadata_t message_interrogate_file_info_t::files[8]
```

Array of file metadata entries.

6.14.2.2 num_files

```
uint32_t message_interrogate_file_info_t::num_files
```

Number of files available.

The documentation for this struct was generated from the following file:

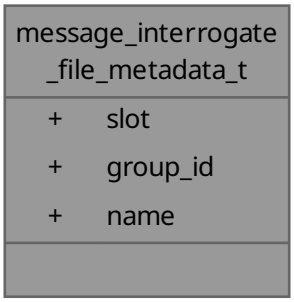
- include/message/interrogate.h

6.15 message_interrogate_file_metadata_t Struct Reference

Interrogated file metadata structure (packed)

```
#include <interrogate.h>
```

Collaboration diagram for message_interrogate_file_metadata_t:



Public Attributes

- uint8_t slot

  File slot number.
- uint16_t group_id

  Group ID owning the file.
- char name [32]

  File name (32 bytes)

6.15.1 Detailed Description

Interrogated file metadata structure (packed)

6.15.2 Member Data Documentation

6.15.2.1 group_id

```
uint16_t message_interrogate_file_metadata_t::group_id
```

Group ID owning the file.

6.15.2.2 name

```
char message_interrogate_file_metadata_t::name[32]
```

File name (32 bytes)

6.15.2.3 slot

```
uint8_t message_interrogate_file_metadata_t::slot
```

File slot number.

The documentation for this struct was generated from the following file:

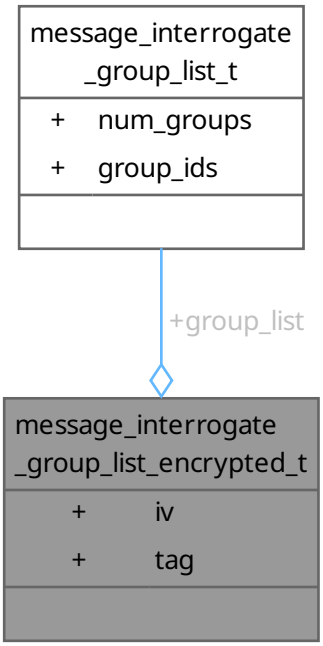
- include/message/interrogate.h

6.16 message_interrogate_group_list_encrypted_t Struct Reference

Encrypted interrogated group list structure.

```
#include <interrogate.h>
```

Collaboration diagram for message_interrogate_group_list_encrypted_t:



Public Attributes

- uint32_t iv [4]

  IV for AES-GCM encryption (128 bits)
- message_interrogate_group_list_t group_list

  Group list data.
- uint32_t tag [4]

  Authentication tag (128 bits)

6.16.1 Detailed Description

Encrypted interrogated group list structure.

6.16.2 Member Data Documentation

6.16.2.1 group_list

```
message_interrogate_group_list_t message_interrogate_group_list_encrypted_t::group_list
```

Group list data.

6.16.2.2 iv

```
uint32_t message_interrogate_group_list_encrypted_t::iv[4]
```

IV for AES-GCM encryption (128 bits)

6.16.2.3 tag

```
uint32_t message_interrogate_group_list_encrypted_t::tag[4]
```

Authentication tag (128 bits)

The documentation for this struct was generated from the following file:

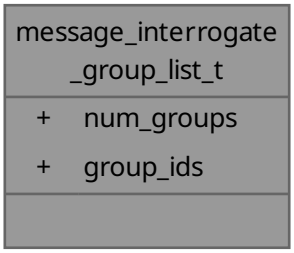
- include/message/interrogate.h

6.17 message_interrogate_group_list_t Struct Reference

Interrogated group list structure (aligned to AES block size)

```
#include <interrogate.h>
```

Collaboration diagram for message_interrogate_group_list_t:



Public Attributes

- uint16_t num_groups

  Number of groups available.
- uint16_t group_ids [8]

  Array of group IDs.

6.17.1 Detailed Description

Interrogated group list structure (aligned to AES block size)

6.17.2 Member Data Documentation

6.17.2.1 group_ids

```
uint16_t message_interrogate_group_list_t::group_ids[8]
```

Array of group IDs.

6.17.2.2 num_groups

```
uint16_t message_interrogate_group_list_t::num_groups
```

Number of groups available.

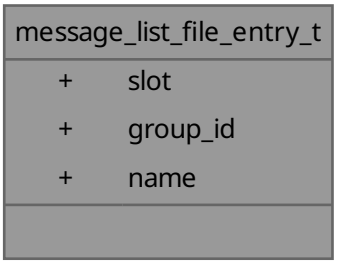
The documentation for this struct was generated from the following file:

- include/message/interrogate.h

6.18 message_list_file_entry_t Struct Reference

```
#include <list.h>
```

Collaboration diagram for message_list_file_entry_t:



Public Attributes

- uint8_t slot

  File slot number.
- uint16_t group_id

  Group ID owning the file.
- char name [32]

  File name (32 bytes)

6.18.1 Member Data Documentation

6.18.1.1 group_id

```
uint16_t message_list_file_entry_t::group_id
```

Group ID owning the file.

6.18.1.2 name

```
char message_list_file_entry_t::name[32]
```

File name (32 bytes)

6.18.1.3 slot

```
uint8_t message_list_file_entry_t::slot
```

File slot number.

The documentation for this struct was generated from the following file:

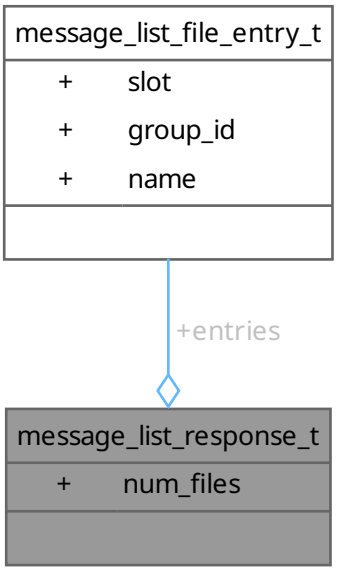
- include/message/list.h

6.19 message_list_response_t Struct Reference

Packed list response structure.

```
#include <list.h>
```

Collaboration diagram for message_list_response_t:



Public Attributes

- uint32_t num_files

  Number of files available.
- message_list_file_entry_t entries [8]

  Array of file entries.

6.19.1 Detailed Description

Packed list response structure.

Warning

All items in this struct should only be assigned using byte operations.

6.19.2 Member Data Documentation

6.19.2.1 entries

```
message_list_file_entry_t message_list_response_t::entries[8]
```

Array of file entries.

6.19.2.2 num_files

```
uint32_t message_list_response_t::num_files
```

Number of files available.

The documentation for this struct was generated from the following file:

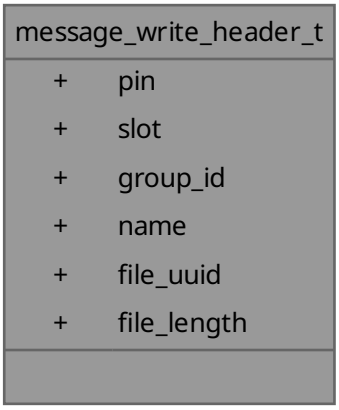
- include/message/list.h

6.20 message_write_header_t Struct Reference

Write file command header structure.

```
#include <write.h>
```

Collaboration diagram for message_write_header_t:



Public Attributes

- uint8_t pin [6]

  Host PIN for authentication (6 bytes)
- uint8_t slot

  Target file slot number.
- uint16_t group_id

  Group ID for file access permissions.
- uint8_t name [32]

  File name (32 bytes)
- uint8_t file_uuid [16]

  File UUID for identification (16 bytes)
- uint16_t file_length

  File length in bytes.

6.20.1 Detailed Description

Write file command header structure.

6.20.2 Member Data Documentation

6.20.2.1 file_length

```
uint16_t message_write_header_t::file_length
```

File length in bytes.

6.20.2.2 file_uuid

```
uint8_t message_write_header_t::file_uuid[16]
```

File UUID for identification (16 bytes)

6.20.2.3 group_id

```
uint16_t message_write_header_t::group_id
```

Group ID for file access permissions.

6.20.2.4 name

```
uint8_t message_write_header_t::name[32]
```

File name (32 bytes)

6.20.2.5 pin

```
uint8_t message_write_header_t::pin[6]
```

Host PIN for authentication (6 bytes)

6.20.2.6 slot

```
uint8_t message_write_header_t::slot
```

Target file slot number.

The documentation for this struct was generated from the following file:

- include/message/write.h

Chapter 7

File Documentation

7.1 /workspaces/2026-ectf-osu-design/docs/design.md File Reference

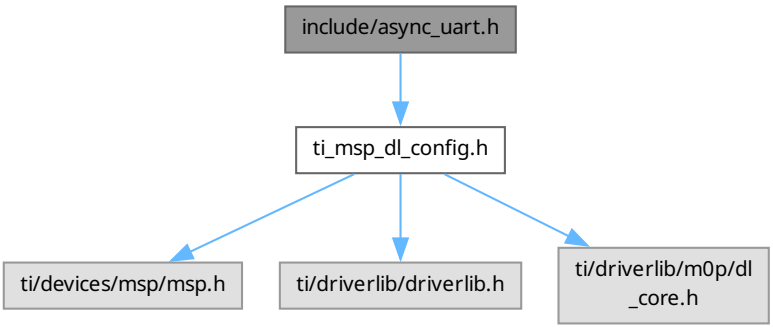
7.2 /workspaces/2026-ectf-osu-design/docs/index.md File Reference

7.3 /workspaces/2026-ectf-osu-design/docs/misc.md File Reference

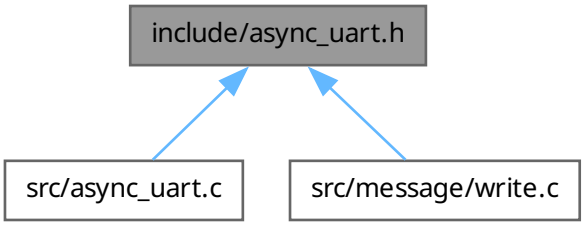
7.4 include/async_uart.h File Reference

```
#include "ti_msp_dl_config.h"
```

Include dependency graph for async_uart.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct async_uart_ctx

  Async UART context structure.

Functions

- int async_uart_receive (async_uart_ctx *ctx)

  Starts asynchronous UART receive operation.

7.4.1 Function Documentation

7.4.1.1 async_uart_receive()

```
int async_uart_receive (
            async_uart_ctx * ctx)
```

Starts asynchronous UART receive operation.

Parameters

in,out	*ctx*	Pointer to async UART context

Returns

0 on success, negative value on error

This function fills the buffer passed in the ctx struct with bytes from the UART. We utilize the DMA interrupt to ACK every 256 bytes to allow other operations to be performed concurrently with IO. Here is the caller graph for this function:

7.5 async_uart.h

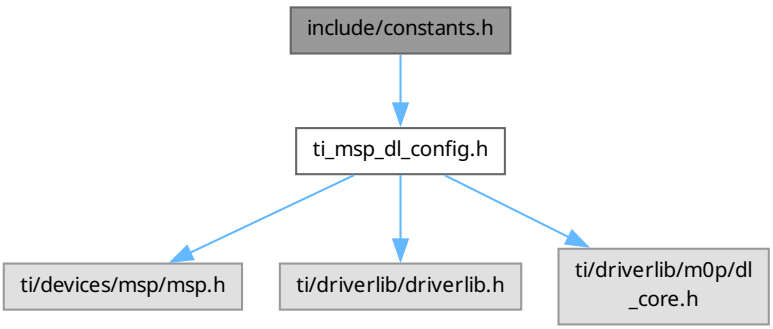
Go to the documentation of this file.

```
#pragma once

#include "ti_msp_dl_config.h"

typedef struct {
    UART_Regs* uart;

    uint8_t dma_channel;

    bool transfer_complete;

    bool stop;

    bool stopped;

    size_t first_chunk_offset;

    size_t bytes_transfered;

    size_t total_bytes;

    uint8_t* buffer;
} async_uart_ctx;

int async_uart_receive(async_uart_ctx* ctx);
```

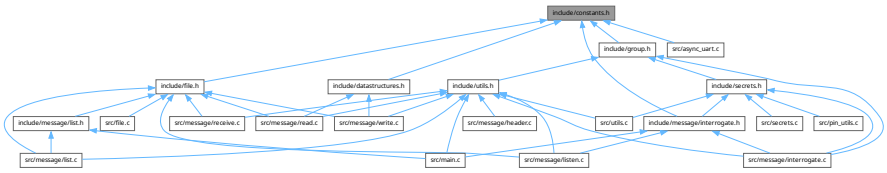
7.6 include/constants.h File Reference

`#include "ti_msp_dl_config.h"`

Include dependency graph for constants.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define MAX_FILE_SIZE 8192

  Maximum file size in bytes.
- #define MAX_GROUPS 32

  Maximum number of groups supported.
- #define NUM_SLOTS 8

  Number of file slots available.
- #define CHUNK_SIZE 256

  Size of each data chunk in bytes.
- #define WRITE_HEADER_SIZE 59

  Size of write header in bytes.
- #define PIN_DELAY 3.5 ∗ 32000000

  Delay cycles for PIN validation (3.5 seconds at 32MHz)

7.6.1 Macro Definition Documentation

7.6.1.1 CHUNK_SIZE

```
#define CHUNK_SIZE 256
```

Size of each data chunk in bytes.

7.6.1.2 MAX_FILE_SIZE

```
#define MAX_FILE_SIZE 8192
```

Maximum file size in bytes.

7.6.1.3 MAX_GROUPS

```
#define MAX_GROUPS 32
```

Maximum number of groups supported.

7.6.1.4 NUM_SLOTS

```
#define NUM_SLOTS 8
```

Number of file slots available.

7.6.1.5 PIN_DELAY

```
#define PIN_DELAY 3.5 * 32000000
```

Delay cycles for PIN validation (3.5 seconds at 32MHz)

7.6.1.6 WRITE_HEADER_SIZE

```
#define WRITE_HEADER_SIZE 59
```

Size of write header in bytes.

7.7 constants.h

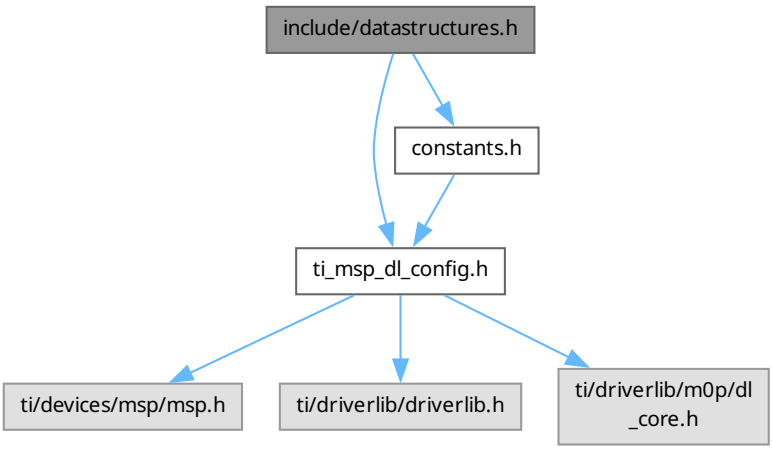
Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "ti_msp_dl_config.h"
00004
00006 #define MAX_FILE_SIZE 8192
00007
00009 #define MAX_GROUPS 32
00010
00012 #define NUM_SLOTS 8
00013
00015 #define CHUNK_SIZE 256
00016
00018 #define WRITE_HEADER_SIZE 59
00019
00021 #define PIN_DELAY 3.5 * 32000000
```

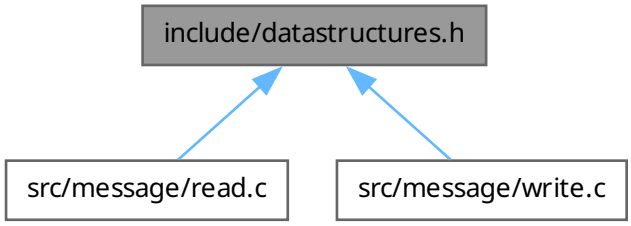
7.8 include/datastructures.h File Reference

```
#include "ti_msp_dl_config.h"
#include "constants.h"
```

Include dependency graph for datastructures.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct BufferNode

  Buffer node for linked list of data chunks.

7.9 datastructures.h

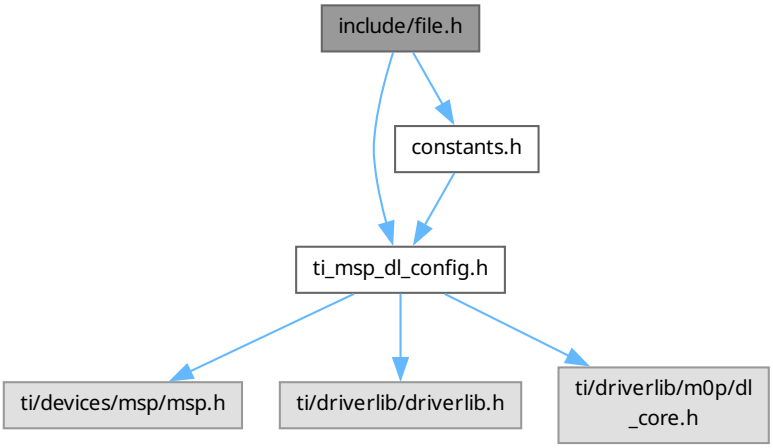
Go to the documentation of this file.

```
#pragma once

#include "ti_msp_dl_config.h"
#include "constants.h"

typedef struct BufferNode {
    struct BufferNode* next;

    uint16_t length;

    uint32_t buffer[CHUNK_SIZE / 4];
} BufferNode;
```

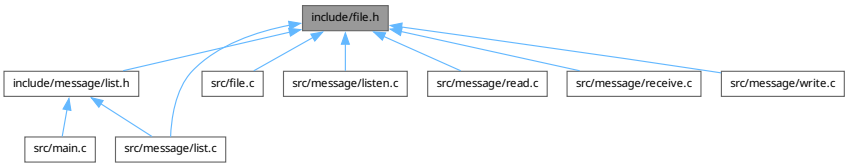
7.10 include/file.h File Reference

```
#include "ti_msp_dl_config.h"
#include "constants.h"
```

Include dependency graph for file.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct file_fat_entry_t

  Required FAT Structure.
- struct file_address_table_t

  Padded File Address Table.
- struct file_metadata_t

  Full File Metadata.
- struct file_metadata_signed_t

  Signed File Metadata.
- struct file_slot_entry_t

  Structure of data in each "slot" of memory.

Macros

- #define FLASH_SECTOR_SIZE 1024
- #define FLASH_WORD_SIZE 8
- #define MAX_PADDING_BACK (16 - MAX_FILE_SIZE % 16)

  Maximum padding required at back of file to align with AES block boundary.

Functions

- int file_read_fat (file_address_table_t *fat)

  Reads the FAT table from flash.
- int file_write_fat (file_address_table_t *fat)

  Writes the FAT table to flash.

Variables

- file_address_table_t FILE_ADDRESS_TABLE
- file_slot_entry_t SLOTS [8]

7.10.1 Macro Definition Documentation

7.10.1.1 FLASH_SECTOR_SIZE

```
#define FLASH_SECTOR_SIZE 1024
```

7.10.1.2 FLASH_WORD_SIZE

```
#define FLASH_WORD_SIZE 8
```

7.10.1.3 MAX_PADDING_BACK

```
#define MAX_PADDING_BACK (16 - MAX_FILE_SIZE % 16)
```

Maximum padding required at back of file to align with AES block boundary.

7.10.2 Function Documentation

7.10.2.1 file_read_fat()

```
int file_read_fat (
            file_address_table_t * fat)
```

Reads the FAT table from flash.

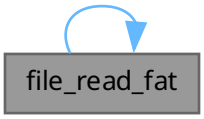
Parameters

out	*fat*	Pointer to FAT table structure to populate

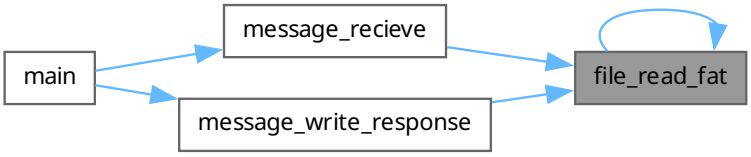
Returns

0 on success, negative value on error

Here is the call graph for this function:



Here is the caller graph for this function:



7.10.2.2 file_write_fat()

```
int file_write_fat (
            file_address_table_t * fat)
```

Writes the FAT table to flash.

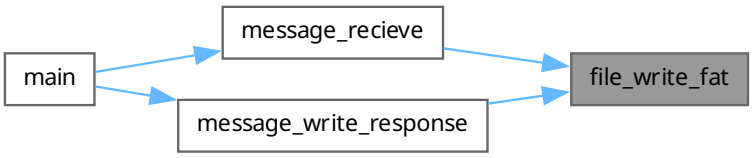
Parameters

in	*fat*	Pointer to FAT table structure to write

Returns

0 on success, negative value on error

Here is the caller graph for this function:



7.10.3 Variable Documentation

7.10.3.1 FILE_ADDRESS_TABLE

```
file_address_table_t FILE_ADDRESS_TABLE [extern]
```

Struct representing the File Address Table on Flash.

Note

This struct is only writable using the FlashCTL.

7.10.3.2 SLOTS

```
file_slot_entry_t SLOTS[8] [extern]
```

Struct representing the File System on Flash.

Note

This struct is only writable using the FlashCTL.

7.11 file.h

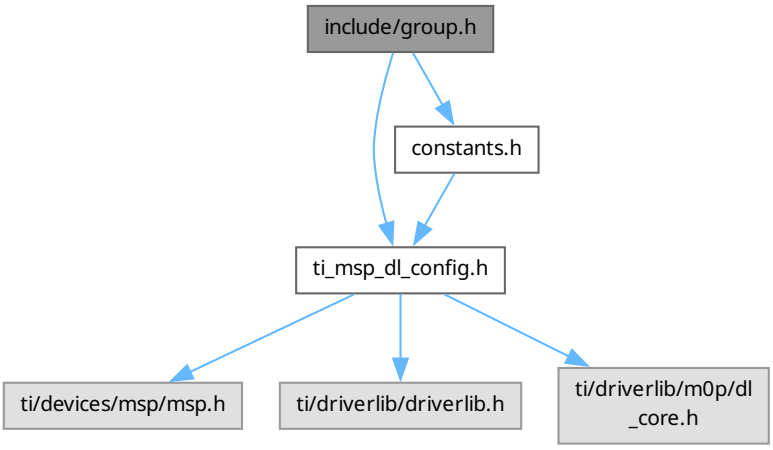
Go to the documentation of this file.

```
#pragma once

#include "ti_msp_dl_config.h"
#include "constants.h"

#define FLASH_SECTOR_SIZE 1024
#define FLASH_WORD_SIZE 8

typedef struct {
    uint8_t uuid[16];

    uint16_t length;

    uint16_t padding;

    uint32_t address;
} file_fat_entry_t;

typedef struct __attribute__((aligned(FLASH_SECTOR_SIZE))){
    file_fat_entry_t entries[NUM_SLOTS];
} file_address_table_t;


typedef struct {
    uint8_t uuid[16];

    uint16_t group_id;

    char name[32];

    uint16_t file_size;

    uint32_t file_signature[4];

    uint8_t encryption_public_key[32];
} file_metadata_t;


typedef struct __attribute__((aligned(FLASH_WORD_SIZE))) {
    file_metadata_t metadata;

    uint8_t writer_signature[64];
} file_metadata_signed_t;

#define MAX_PADDING_BACK (16 - MAX_FILE_SIZE % 16)

typedef struct __attribute__((aligned(FLASH_SECTOR_SIZE))) {
    file_metadata_signed_t signed_metadata __attribute__((aligned(FLASH_WORD_SIZE)));

    uint8_t encrypted_file[MAX_FILE_SIZE + MAX_PADDING_BACK]
__attribute__((aligned(FLASH_WORD_SIZE)));
} file_slot_entry_t;

extern file_address_table_t FILE_ADDRESS_TABLE;
extern file_slot_entry_t SLOTS[NUM_SLOTS];

int file_read_fat(file_address_table_t* fat);

int file_write_fat(file_address_table_t* fat);
```

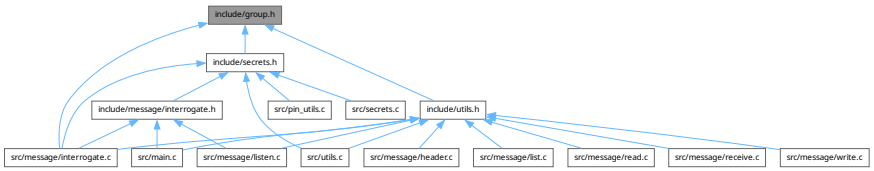
7.12 include/group.h File Reference

```
#include "ti_msp_dl_config.h"
#include "constants.h"
```

Include dependency graph for group.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct group_permissions_t

  Group Permissions Structure.
- struct group_private_t

  Group Private Keys Structure.
- struct group_public_t

  Group Public Keys Structure.
- struct group_t

  Group Structure as Stored on Flash.

7.13 group.h

Go to the documentation of this file.

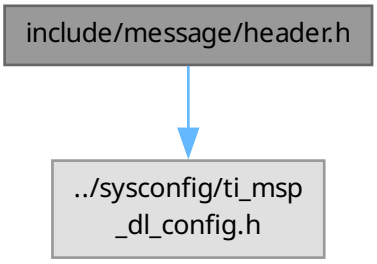
```
00001 #pragma once
00002
00003 #include "ti_msp_dl_config.h"
00004 #include "constants.h"
00005
00012 typedef struct {
```

```
    bool read;

    bool write;

    bool transfer;
} group_permissions_t;

typedef struct {
    group_permissions_t permissions;

    uint8_t read_key[32];

    uint8_t write_key[64];

    uint8_t transfer_key[32];
} group_private_t;

typedef struct {
    uint8_t read_key[32];

    uint8_t write_key[32];

    uint8_t transfer_key[32];
} group_public_t;

typedef struct {
    uint16_t group_id;

    group_public_t public;

    group_private_t private;
} group_t;
```

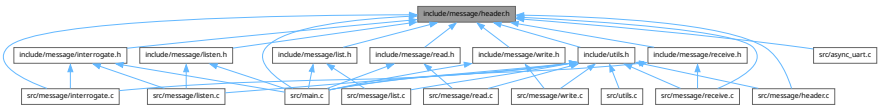
7.14 include/message/header.h File Reference

`#include "../sysconfig/ti_msp_dl_config.h"`

Include dependency graph for header.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct message_header_t

  Message header structure.

Enumerations

- enum message_operation_t {
  MESSAGE_LIST = 'L' , MESSAGE_READ = 'R' , MESSAGE_WRITE = 'W' , MESSAGE_RECEIVE = 'C' ,
  MESSAGE_INTERROGATE = 'I' , MESSAGE_LISTEN = 'N' , MESSAGE_ACK = 'A' , MESSAGE_ERROR = 'E' ,
  MESSAGE_DEBUG = 'D' }

  Message operation codes.

Functions

- int message_header_request (UART_Regs ∗uart, message_header_t ∗header)

  Parses message header from UART.
- void message_header_response (UART_Regs ∗uart, message_header_t ctx)

  Starts a response to the host tools.
- void message_header_send_error (UART_Regs ∗uart, const char ∗msg, size_t msg_len)

  Sends an error message to the host tools.
- void message_header_send_debug (UART_Regs ∗uart, const char ∗msg, size_t msg_len)

  Sends a debug message to the host tools.
- void message_header_send_ack (UART_Regs ∗uart)

  Sends an ACK message to the host tools.
- void message_header_receive_ack (UART_Regs ∗uart)

  Blocks until ACK message is received.

7.14.1 Enumeration Type Documentation

7.14.1.1 message_operation_t

```
enum message_operation_t
```

Message operation codes.

Enumerator

MESSAGE_LIST	List files command/response.
MESSAGE_READ	Read file command/response.
MESSAGE_WRITE	Write file command/response.
MESSAGE_RECEIVE	Receive file command/response.
MESSAGE_INTERROGATE	Interrogate files command/response.
MESSAGE_LISTEN	Listen command/response.
MESSAGE_ACK	Acknowledge the receipt of data.
MESSAGE_ERROR	Notify of an error/failure.
MESSAGE_DEBUG	Provide debug information.

7.14.2 Function Documentation

7.14.2.1 message_header_receive_ack()

```
void message_header_receive_ack (
            UART_Regs * uart)
```

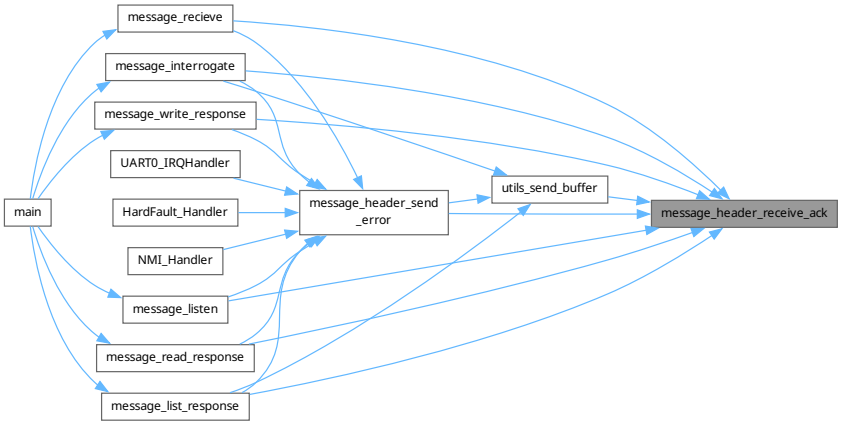
Blocks until ACK message is received.

Parameters

in	*uart*	UART registers for communication

Here is the call graph for this function:

Here is the caller graph for this function:



7.14.2.2 message_header_request()

```
int message_header_request (
            UART_Regs * uart,
            message_header_t * header)
```

Parses message header from UART.

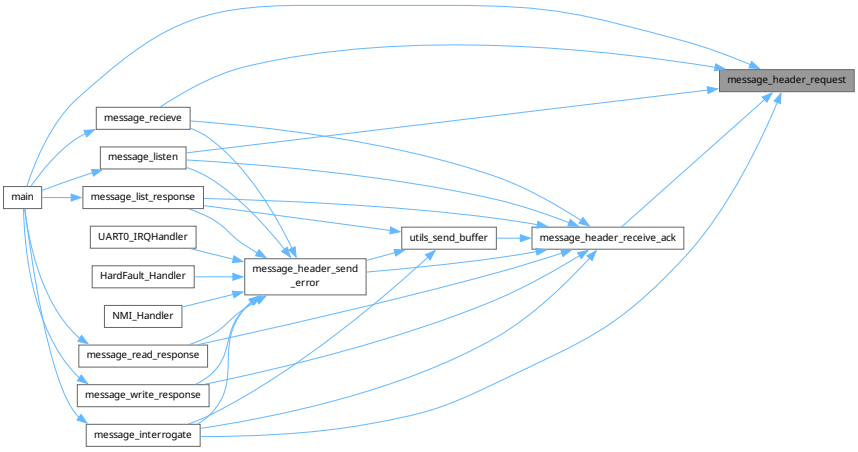
Parameters

in	*uart*	UART registers for communication
out	*header*	Output header structure

Returns

0 on success, negative value on error

Here is the caller graph for this function:



7.14.2.3 message_header_response()

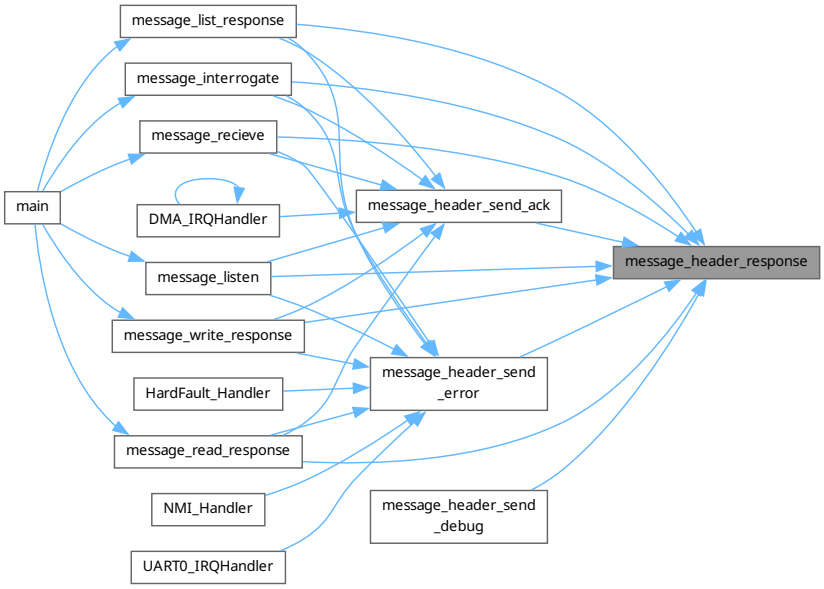
```
void message_header_response (
            UART_Regs * uart,
            message_header_t ctx)
```

Starts a response to the host tools.

Parameters

in	*uart*	UART registers for communication
in	*ctx*	Message header containing response context

Here is the caller graph for this function:



7.14.2.4 message_header_send_ack()

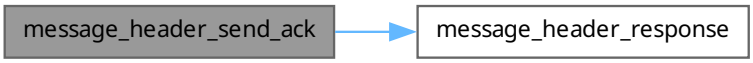
```
void message_header_send_ack (
            UART_Regs * uart)
```

Sends an ACK message to the host tools.

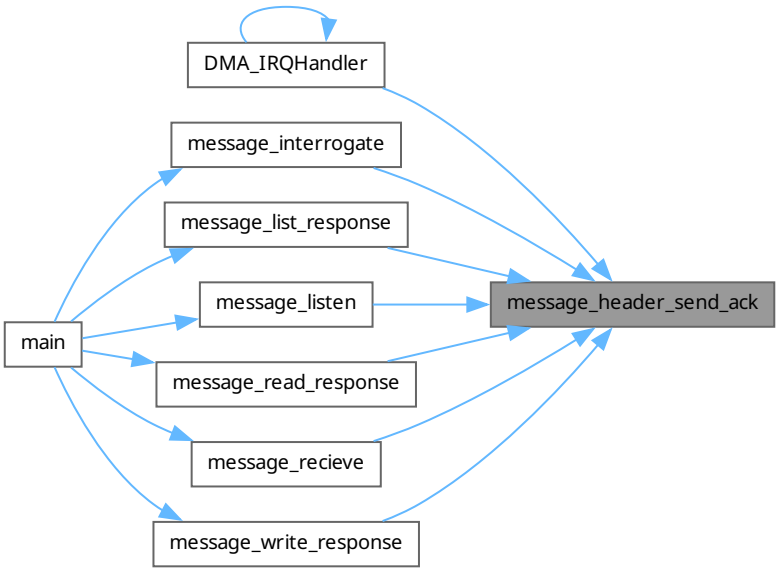
Parameters

in	*uart*	UART registers for communication

Here is the call graph for this function:



Here is the caller graph for this function:



7.14.2.5 message_header_send_debug()

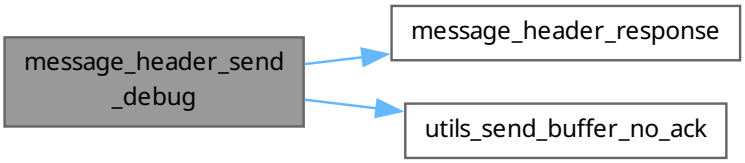
```
void message_header_send_debug (
            UART_Regs * uart,
            const char * msg,
            size_t msg_len)
```

Sends a debug message to the host tools.

Parameters

in	*uart*	UART registers for communication
in	*msg*	Debug message string
in	*msg_len*	Length of debug message

Here is the call graph for this function:



7.14.2.6 message_header_send_error()

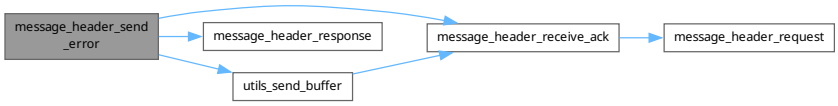
```
void message_header_send_error (
            UART_Regs * uart,
            const char * msg,
            size_t msg_len)
```

Sends an error message to the host tools.

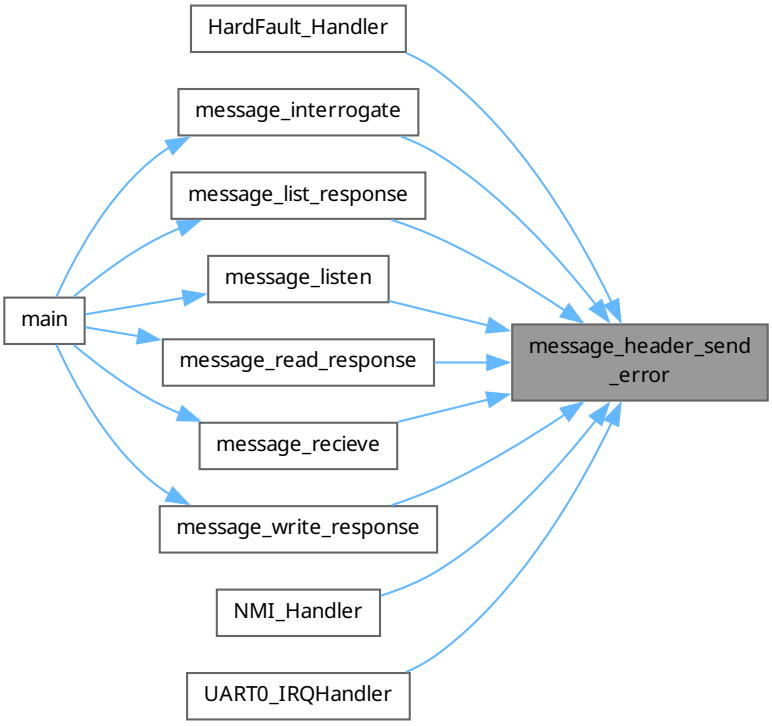
Parameters

in	*uart*	UART registers for communication
in	*msg*	Error message string
in	*msg_len*	Length of error message

Here is the call graph for this function:



Here is the caller graph for this function:



7.15 header.h

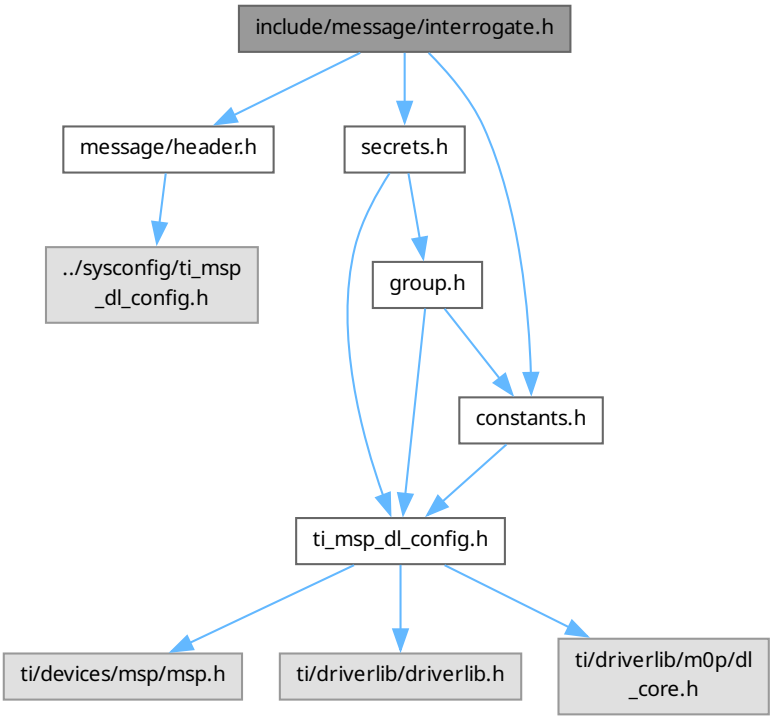
Go to the documentation of this file.

```
#pragma once

#include "../sysconfig/ti_msp_dl_config.h"

typedef enum {
    MESSAGE_LIST = 'L',

    MESSAGE_READ = 'R',

    MESSAGE_WRITE = 'W',

    MESSAGE_RECEIVE = 'C',

    MESSAGE_INTERROGATE = 'I',

    MESSAGE_LISTEN = 'N',

    MESSAGE_ACK = 'A',

    MESSAGE_ERROR = 'E',

    MESSAGE_DEBUG = 'D',
} message_operation_t;


typedef struct {
    message_operation_t operation;

    uint16_t message_length;
} message_header_t;

int message_header_request(UART_Regs *uart, message_header_t* header);

void message_header_response(UART_Regs* uart, message_header_t ctx);

void message_header_send_error(UART_Regs* uart, const char* msg, size_t msg_len);

void message_header_send_debug(UART_Regs* uart, const char* msg, size_t msg_len);

void message_header_send_ack(UART_Regs* uart);

void message_header_receive_ack(UART_Regs* uart);
```

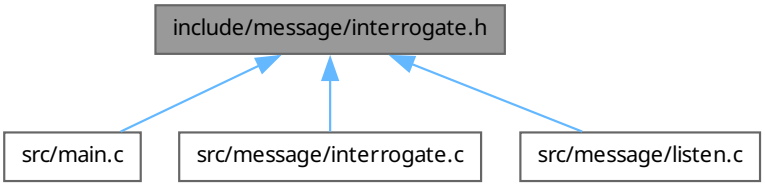
7.16 include/message/interrogate.h File Reference

```
#include "message/header.h"
#include "constants.h"
#include "secrets.h"
```

Include dependency graph for interrogate.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct message_interrogate_group_list_t

  Interrogated group list structure (aligned to AES block size)
- struct message_interrogate_group_list_encrypted_t

  Encrypted interrogated group list structure.
- struct message_interrogate_file_metadata_t

  Interrogated file metadata structure (packed)
- struct message_interrogate_file_info_t

Interrogated file info structure (aligned to AES block size)

- struct message_interrogate_file_info_encrypted_t

  Encrypted interrogation file info structure.

Functions

- void message_interrogate (message_header_t header)

  Responds to Interrogate Request.

7.16.1 Function Documentation

7.16.1.1 message_interrogate()

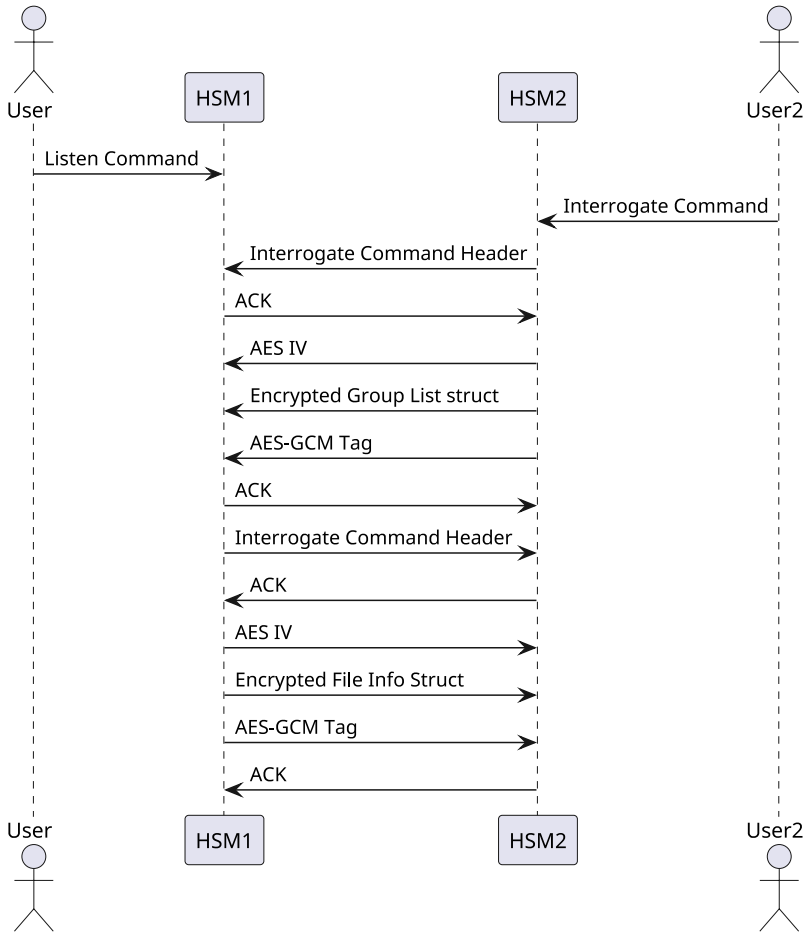
```
void message_interrogate (
            message_header_t header)
```

Responds to Interrogate Request.

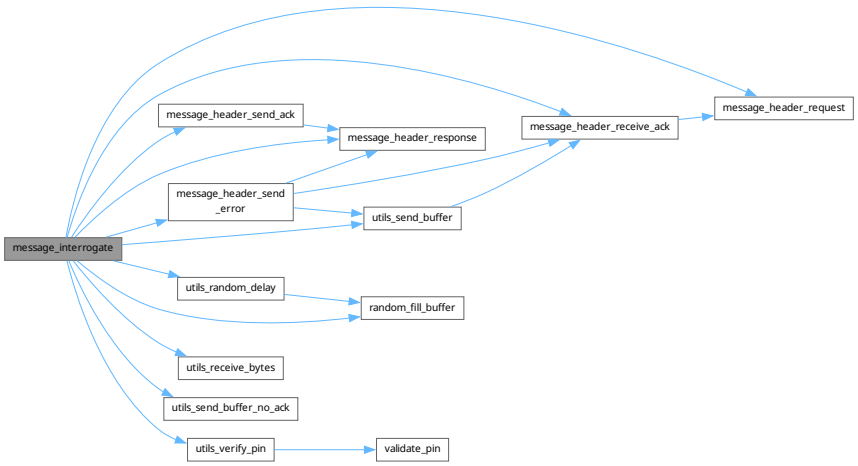
Parameters

in	*header*	Message header containing request context

Initally our plan for interrogate was to sign an ED25519 public key using a secret transfer key but that was not feasible given the time constraints. Since we couldn't use asymetric encryption for this task we fell back to using a simple static AES key. In theory you could still use signatures for verification but at the end of the day it would have little impact on security.



Here is the call graph for this function:



Here is the caller graph for this function:

7.17 interrogate.h

Go to the documentation of this file.

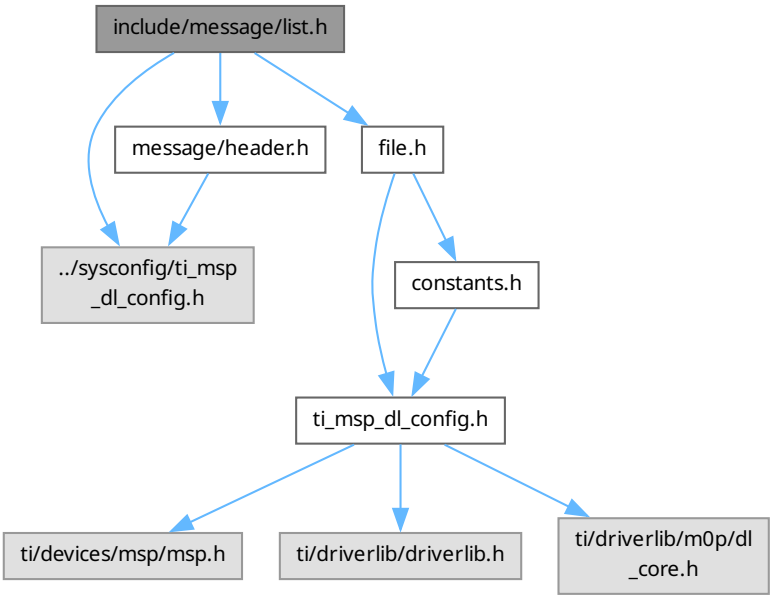
```
#pragma once

#include "message/header.h"
#include "constants.h"
#include "secrets.h"

typedef struct __attribute__((aligned(16))) {
    uint16_t num_groups;

    uint16_t group_ids[8];
} message_interrogate_group_list_t;

typedef struct {
    uint32_t iv[4];

    message_interrogate_group_list_t group_list;

    uint32_t tag[4];
} message_interrogate_group_list_encrypted_t;

typedef struct __attribute__((packed)) {
    uint8_t slot;

    uint16_t group_id;

    char name[32];
} message_interrogate_file_metadata_t;
```

```
typedef struct __attribute__((aligned(16))) {
    uint32_t num_files;

    message_interrogate_file_metadata_t files[8];
} message_interrogate_file_info_t;

typedef struct {
    uint32_t iv[4];

    message_interrogate_file_info_t file_info;

    uint32_t tag[4];
} message_interrogate_file_info_encrypted_t;

void message_interrogate(message_header_t header);
```

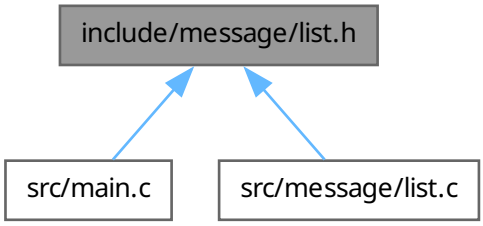
7.18 include/message/list.h File Reference

```
#include "../sysconfig/ti_msp_dl_config.h"
#include "message/header.h"
#include "file.h"
```

Include dependency graph for list.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct message_list_file_entry_t
- struct message_list_response_t

  Packed list response structure.

Functions

- void message_list_response (message_header_t header)

  Responds to a List Request.

7.18.1 Function Documentation

7.18.1.1 message_list_response()

```
void message_list_response (
            message_header_t header)
```

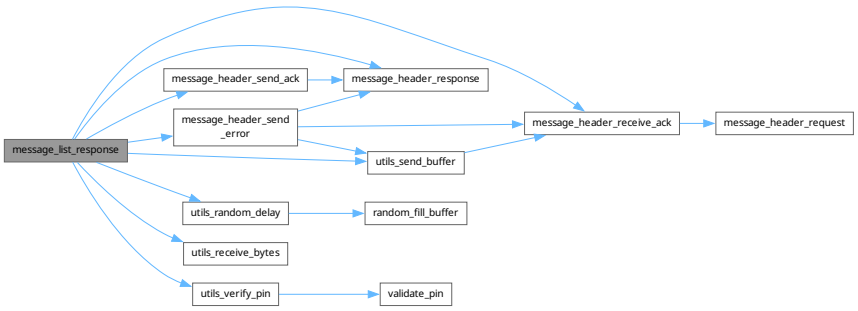
Responds to a List Request.

Parameters

in	*header*	Message header containing request context

List is fairly simple in function. We simply iterrate over the file system checking if the length of each structure is in less than the max file size and if it is less than the max file size we know a file exists in that slot. Here is the call

graph for this function:



Here is the caller graph for this function:

7.19 list.h

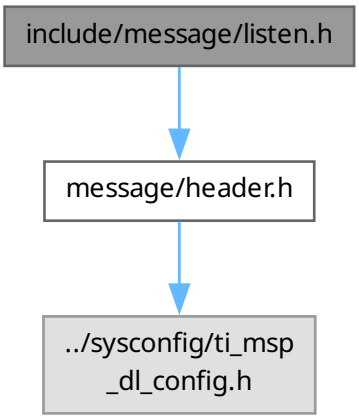
Go to the documentation of this file.

```
#pragma once

#include "../sysconfig/ti_msp_dl_config.h"
#include "message/header.h"
#include "file.h"

typedef struct __attribute__((packed)) {
    uint8_t slot;

    uint16_t group_id;

    char name[32];
} message_list_file_entry_t;

typedef struct __attribute__((packed)) {
    uint32_t num_files;

    message_list_file_entry_t entries[NUM_SLOTS];
} message_list_response_t;

void message_list_response(message_header_t header);
```

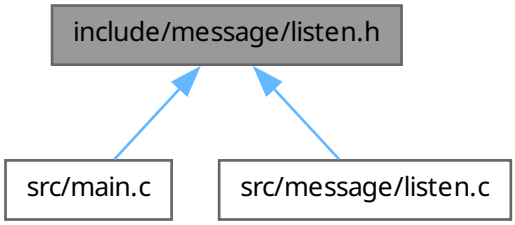
7.20 include/message/listen.h File Reference

```
#include "message/header.h"
```

Include dependency graph for listen.h:



This graph shows which files directly or indirectly include this file:



Functions

- void message_listen (message_header_t header)

  Responds to Listen Request.

7.20.1 Function Documentation

7.20.1.1 message_listen()

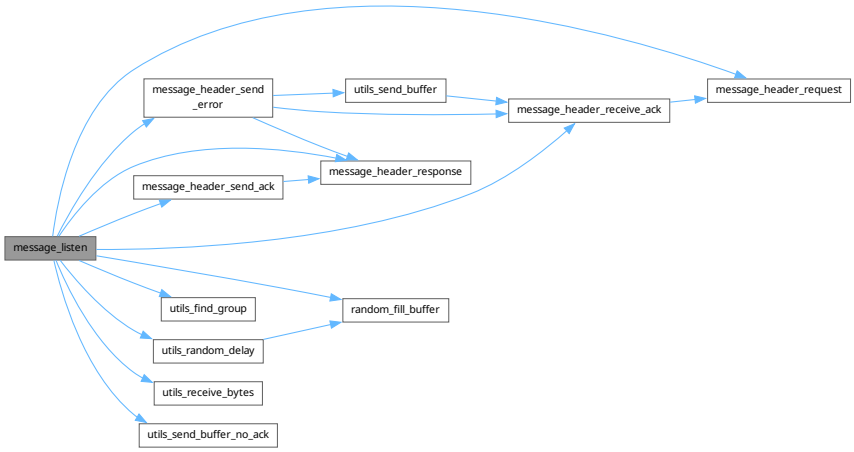
```
void message_listen (
            message_header_t header)
```

Responds to Listen Request.

Parameters

in	*header*	Message header containing request context

The listen is other end of the recieve command and the interrogate command. The listen command starts by recieving a header from the remote HSM and then based on the header opcode the listen command provides a response in accordance to the recieve command definition or the interrogate command definition. Here is the call graph for this function:



Here is the caller graph for this function:

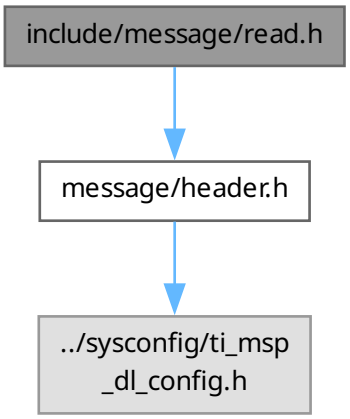
7.21 listen.h

Go to the documentation of this file.

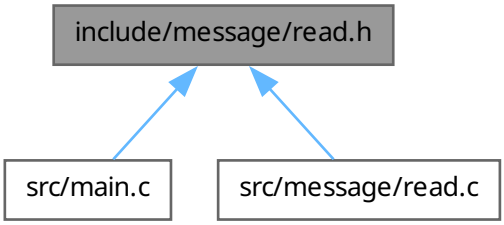
```
00001 #pragma once
00002
00003 #include "message/header.h"
00004
00010 void message_listen(message_header_t header);
```

7.22 include/message/read.h File Reference

`#include "message/header.h"`
Include dependency graph for read.h:



This graph shows which files directly or indirectly include this file:



Functions

- void message_read_response (message_header_t header)
  Responds to Read Request.

7.22.1 Function Documentation

7.22.1.1 message_read_response()

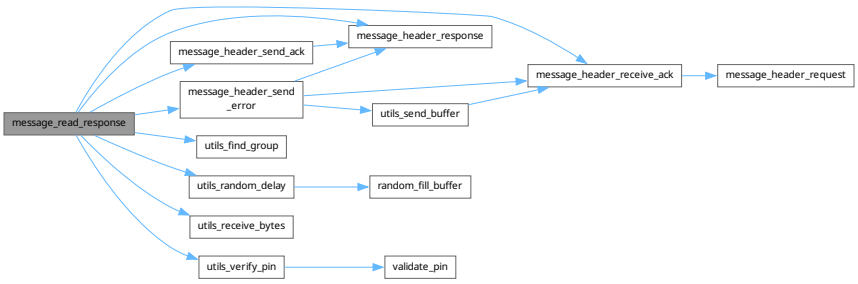
```
void message_read_response (
            message_header_t header)
```

Responds to Read Request.

Parameters

in	*header*	Message header containing request context

The read command is a fairly simple command to process. We start by verifing that the metadata of the file is valid by verifing the signature that is stored alongside the metadata. After verifing the signature we generate the decryption AES secret using the read private key that the board should be provissioned with if it has valid permissions and the public key stored inside the signed metadata. The file is then decrypted in chunks and transfered over UART with the last chunk only being sent if the GCM tag is valid. Here is the call graph for this function:



Here is the caller graph for this function:

7.23 read.h

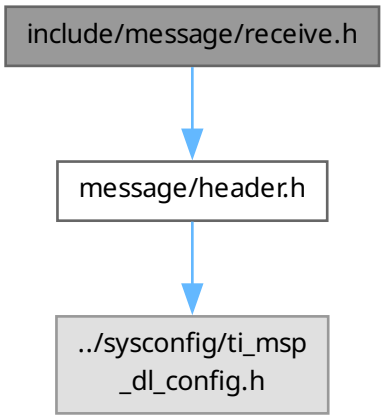
Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "message/header.h"
00009 void message_read_response(message_header_t header);
```

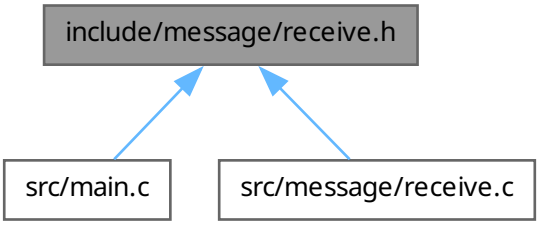
7.24 include/message/receive.h File Reference

```
#include "message/header.h"
```

Include dependency graph for receive.h:



This graph shows which files directly or indirectly include this file:



Functions

- void message_recieve (message_header_t header)

  Responds to Receive Request.

7.24.1 Function Documentation

7.24.1.1 message_recieve()

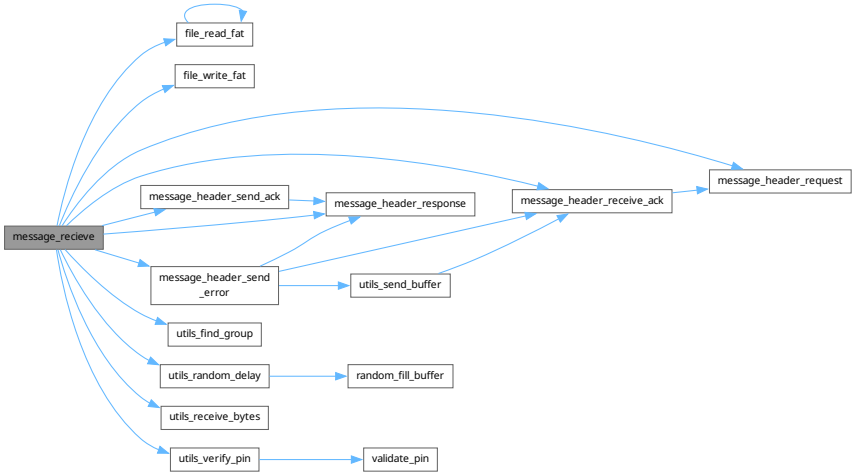
```
void message_recieve (
            message_header_t header)
```

Responds to Receive Request.

Parameters

in	*header*	Message header containing request context

Our recieve design uses asymetric encryption to ensure secrecy. Instead of trying to verify that an HSM can is allowed to decrypt a file we designed our system where we encrypt the file so it can only be decrypted by an HSM that has the private transfer key for the file group. We accomplish this by generating the AES secret key from a randomly generated ED25519 keypair and the public transfer key that every HSM is provisioned with. We then send the public key from the generated keypair to the recieving HSM so that the secret can be regenerated with the private transfer key. We transfer the entire file_slot_entry_t as it is one less point of failure in our design.Here is the call graph for this function:



Here is the caller graph for this function:

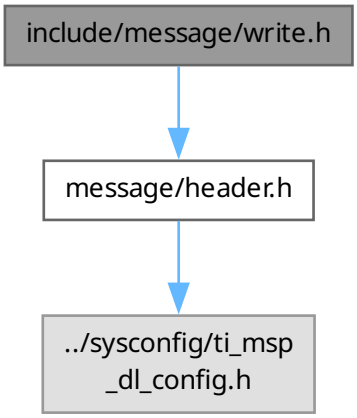
7.25 receive.h

Go to the documentation of this file.

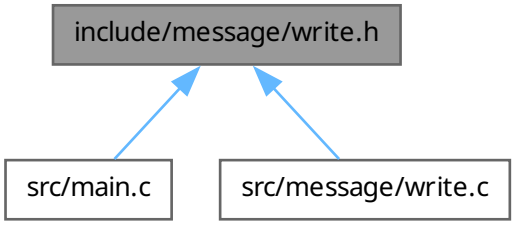
```
00001 #pragma once
00002
00003 #include "message/header.h"
00004
00010 void message_recieve(message_header_t header);
```

7.26 include/message/write.h File Reference

`#include "message/header.h"`
Include dependency graph for write.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct message_write_header_t

  Write file command header structure.

Functions

- message_write_header_t message_write_parse_header (UART_Regs ∗uart)

  Parses the header of the Write File command.
- void message_write_response (message_header_t header)

  Responds to Write Request.

7.26.1 Function Documentation

7.26.1.1 message_write_parse_header()

```
message_write_header_t message_write_parse_header (
            UART_Regs * uart)
```

Parses the header of the Write File command.

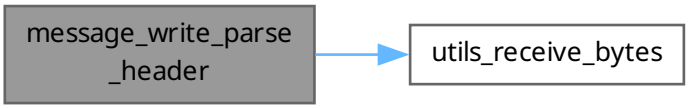
Parameters

in	*uart*	UART registers for communication

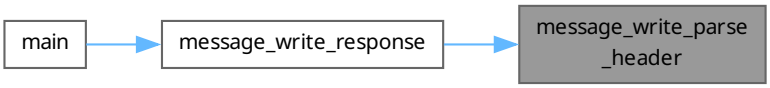
Returns

Write header structure containing parsed information

Here is the call graph for this function:



Here is the caller graph for this function:



7.26.1.2 message_write_response()

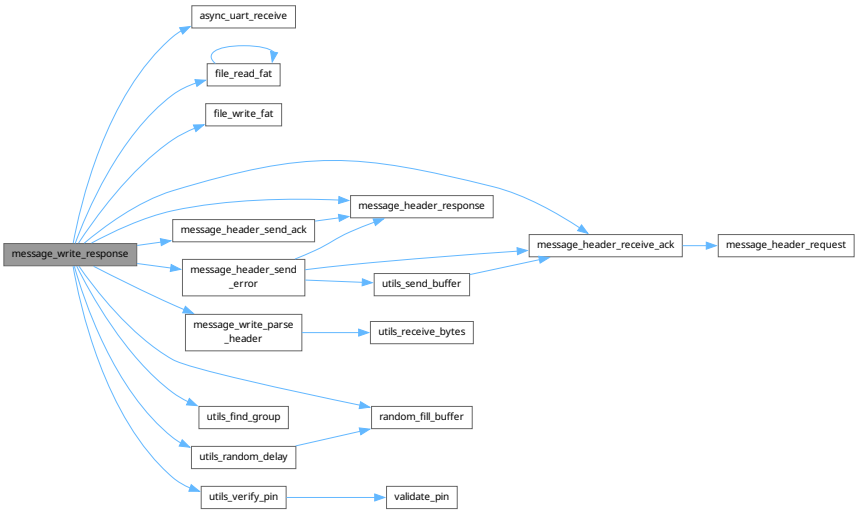
```
void message_write_response (
            message_header_t header)
```

Responds to Write Request.

Parameters

in	*header*	Message header containing request context

This was fun to write and sucked to debug. Our implementation works by concurrently reading bytes from the UART with DMA while the encryption keys are derived and calculated. The encryption calculations are interrupted by the DMA to send ACKs and to re-enable the DMA for each chunk. After the entire file is read into memory then the AES is used to encrypt the file using the generated secret. Each file gets a randomly generated key-pair that is used in combination with the read public key to generate the secret key for the AES encryption. We use AES-GCM so we can verify the file when being read back to the user. The public key from the random key-pair is stored in the metadata that is then signed using the writer private key. This allows us to check that a file was created by a valid writer by verifing that the metadata signature is valid. Here is the call graph for this function:



Here is the caller graph for this function:

7.27 write.h

Go to the documentation of this file.

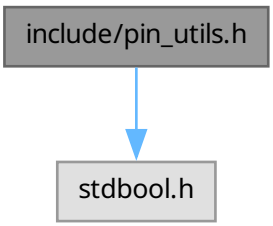
```
00001 #pragma once
00002
00003 #include "message/header.h"
00004
00005
00009 typedef struct {
00011     uint8_t pin[6];
```

```
00012
00014     uint8_t slot;
00015
00017     uint16_t group_id;
00018
00020     uint8_t name[32];
00021
00023     uint8_t file_uuid[16];
00024
00026     uint16_t file_length;
00027 } message_write_header_t;
00028
00036 message_write_header_t message_write_parse_header(UART_Regs* uart);
00037
00043 void message_write_response(message_header_t header);
```

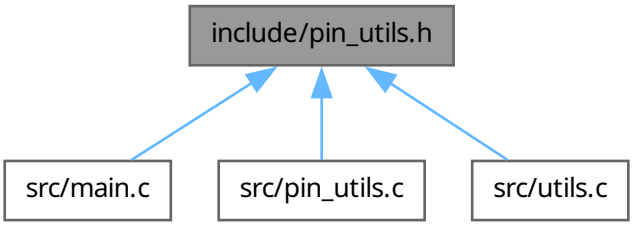
7.28 include/pin_utils.h File Reference

`#include <stdbool.h>`

Include dependency graph for pin_utils.h:



This graph shows which files directly or indirectly include this file:



Functions

- bool validate_pin (const char pin[6])

  Validates PIN with 2-stage BLAKE2b hashing.
- void init_pin ()

  Initializes PIN validation system.

7.28.1 Function Documentation

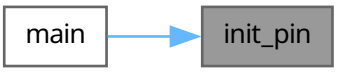
7.28.1.1 init_pin()

```
void init_pin ()
```

Initializes PIN validation system.

Here is the call graph for this function:

Here is the caller graph for this function:



7.28.1.2 validate_pin()

```
bool validate_pin (
            const char pin[6])
```

Validates PIN with 2-stage BLAKE2b hashing.

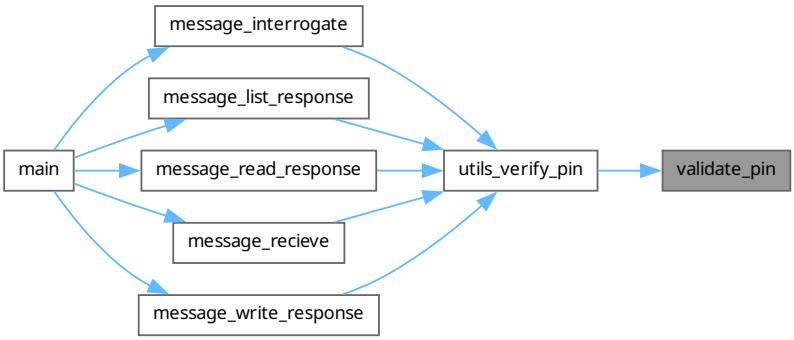
Parameters

in	*pin*	PIN string to validate (6 characters)

Returns

true if PIN is valid, false otherwise

Here is the caller graph for this function:



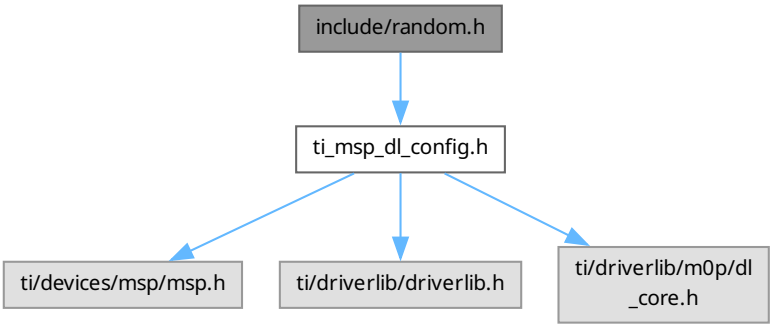
7.29 pin_utils.h

Go to the documentation of this file.

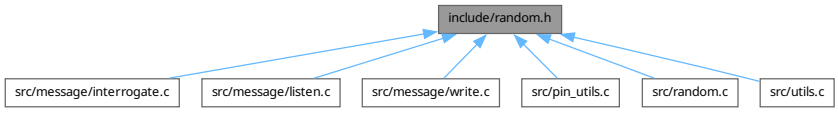
```
#pragma once
#include <stdbool.h>



bool validate_pin(const char pin[6]);

void init_pin();
```

7.30 include/random.h File Reference

`#include "ti_msp_dl_config.h"`
Include dependency graph for random.h:



This graph shows which files directly or indirectly include this file:



Functions

- void random_fill_buffer (unsigned char ∗buffer, size_t output_size)
  Fills buffer with random data using hardware TRNG.

7.30.1 Function Documentation

7.30.1.1 random_fill_buffer()

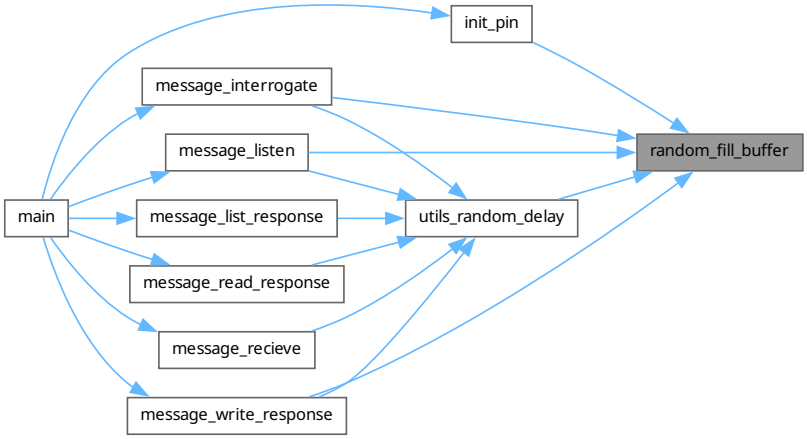
```
void random_fill_buffer (
            unsigned char * buffer,
            size_t output_size)
```

Fills buffer with random data using hardware TRNG.

Parameters

out	*buffer*	Buffer to fill with random data
in	*output_size*	Number of random bytes to generate

Here is the caller graph for this function:



7.31 random.h

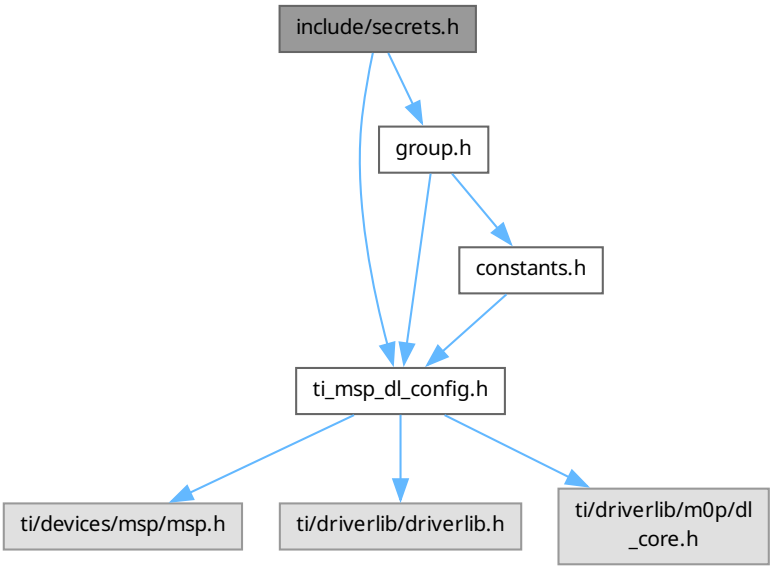
Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "ti_msp_dl_config.h"
00004
00011 void random_fill_buffer(unsigned char *buffer, size_t output_size);
```

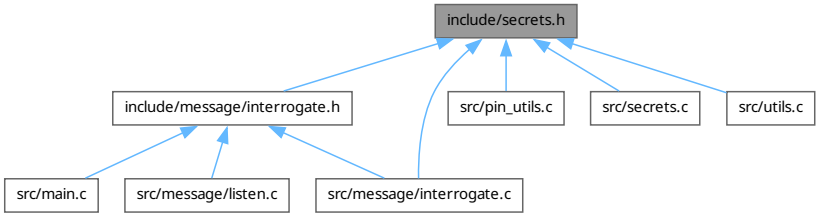
7.32 include/secrets.h File Reference

```
#include "ti_msp_dl_config.h"
#include "group.h"
```

Include dependency graph for secrets.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define STAGE1_PIN_ITERATIONS 51
- #define STAGE1_KEY { 0x20, 0x0b, 0x35, 0x10, 0x56, 0x10, 0x79, 0x07, 0x2d, 0x5e, 0x28, 0xd0, 0x63, 0x96, 0x76, 0xcf, 0x63, 0x3d, 0xd1, 0x34, 0xb8, 0xc8, 0xae, 0x69, 0xac, 0xab, 0x95, 0x1b, 0x73, 0xdd, 0x31, 0xd9, }
- #define STAGE1_PADDING { 0xad, 0xc1, 0xff, 0x6b, 0x2b, 0xb7, 0x93, 0xb5, 0xc0, 0x6d, 0x27, 0x23, 0xae, 0xee, 0x28, 0x31, 0x45, 0xc0, 0xf3, 0x91, 0x71, 0xd7, 0xde, 0x25, 0x1d, 0xb5, 0xe2, 0xc4, 0x23, 0x52, 0xe9, 0x1b, 0xa0, 0xcf, 0x0f, 0x63, 0xaf, 0x05, 0xe5, 0x06, 0xf2, 0x2c, 0x23, 0x7d, 0x1e, 0xb0, 0x37, 0x5e, 0x46, 0xc9, 0xcb, 0xd9, 0x8b, 0x48, 0x72, 0xe3, 0x45, 0x3b, }
- #define STAGE2_RAND_INT 14661271
- #define STAGE1_PIN_HASH { 0x72, 0x38, 0x8d, 0xff, 0x73, 0xb1, 0x53, 0xb5, 0x8b, 0xb4, 0x43, 0x07, 0x51, 0x4f, 0x7c, 0x8b, 0x3e, 0xf5, 0x9e, 0x95, 0xdb, 0x9c, 0x54, 0x7a, 0x26, 0xce, 0xc9, 0xf6, 0x77, 0x61, 0x06, 0x07, 0xe4, 0x43, 0x8c, 0x7d, 0xdb, 0xd6, 0xa3, 0xa7, 0x25, 0xe9, 0x42, 0x3e, 0xd6, 0x0d, 0xc1, 0x0c, 0x59, 0x41, 0x52, 0x49, 0x62, 0xa5, 0x84, 0x80, 0xbf, 0x18, 0x94, 0x67, 0x8c, 0x5d, 0x41, 0xd4, }

Variables

- const uint8_t TRANSFER_KEY [32]
- const group_t groups [1]

7.32.1 Macro Definition Documentation

7.32.1.1 STAGE1_KEY

```
#define STAGE1_KEY { 0x20, 0x0b, 0x35, 0x10, 0x56, 0x10, 0x79, 0x07, 0x2d, 0x5e, 0x28, 0xd0,
0x63, 0x96, 0x76, 0xcf, 0x63, 0x3d, 0xd1, 0x34, 0xb8, 0xc8, 0xae, 0x69, 0xac, 0xab, 0x95,
0x1b, 0x73, 0xdd, 0x31, 0xd9, }
```

7.32.1.2 STAGE1_PADDING

```
#define STAGE1_PADDING { 0xad, 0xc1, 0xff, 0x6b, 0x2b, 0xb7, 0x93, 0xb5, 0xc0, 0x6d, 0x27,
0x23, 0xae, 0xee, 0x28, 0x31, 0x45, 0xc0, 0xf3, 0x91, 0x71, 0xd7, 0xde, 0x25, 0x1d, 0xb5,
0xe2, 0xc4, 0x23, 0x52, 0xe9, 0x1b, 0xa0, 0xcf, 0x0f, 0x63, 0xaf, 0x05, 0xe5, 0x06, 0xf2,
0x2c, 0x23, 0x7d, 0x1e, 0xb0, 0x37, 0x5e, 0x46, 0xc9, 0xcb, 0xd9, 0x8b, 0x48, 0x72, 0xe3,
0x45, 0x3b, }
```

7.32.1.3 STAGE1_PIN_HASH

```
#define STAGE1_PIN_HASH { 0x72, 0x38, 0x8d, 0xff, 0x73, 0xb1, 0x53, 0xb5, 0x8b, 0xb4, 0x43,
0x07, 0x51, 0x4f, 0x7c, 0x8b, 0x3e, 0xf5, 0x9e, 0x95, 0xdb, 0x9c, 0x54, 0x7a, 0x26, 0xce,
0xc9, 0xf6, 0x77, 0x61, 0x06, 0x07, 0xe4, 0x43, 0x8c, 0x7d, 0xdb, 0xd6, 0xa3, 0xa7, 0x25,
0xe9, 0x42, 0x3e, 0xd6, 0x0d, 0xc1, 0x0c, 0x59, 0x41, 0x52, 0x49, 0x62, 0xa5, 0x84, 0x80,
0xbf, 0x18, 0x94, 0x67, 0x8c, 0x5d, 0x41, 0xd4, }
```

7.32.1.4 STAGE1_PIN_ITERATIONS

```
#define STAGE1_PIN_ITERATIONS 51
```

7.32.1.5 STAGE2_RAND_INT

```
#define STAGE2_RAND_INT 14661271
```

7.32.2 Variable Documentation

7.32.2.1 groups

```
const group_t groups[1]  [extern]
```

7.32.2.2 TRANSFER_KEY

```
const uint8_t TRANSFER_KEY[32]  [extern]
```

7.33 secrets.h

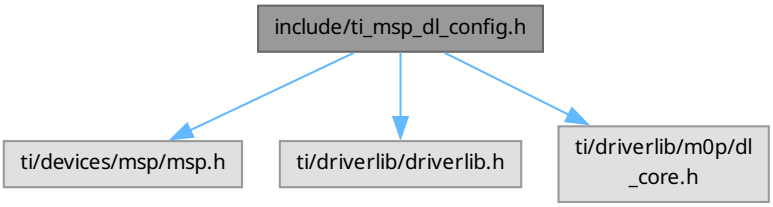
Go to the documentation of this file.

```
#pragma once

#include "ti_msp_dl_config.h"
#include "group.h"



#define STAGE1_PIN_ITERATIONS 51
#define STAGE1_KEY { 0x20, 0x0b, 0x35, 0x10, 0x56, 0x10, 0x79, 0x07, 0x2d, 0x5e, 0x28, 0xd0, 0x63, 0x96, 0x76, 0xcf, 0x63, 0x3d, 0xd1, 0x34, 0xb8, 0xc8, 0xae, 0x69, 0xac, 0xab, 0x95, 0x1b, 0x73, 0xdd, 0x31, 0xd9,  }
#define STAGE1_PADDING { 0xad, 0xc1, 0xff, 0x6b, 0x2b, 0xb7, 0x93, 0xb5, 0xc0, 0x6d, 0x27, 0x23, 0xae, 0xee, 0x28, 0x31, 0x45, 0xc0, 0xf3, 0x91, 0x71, 0xd7, 0xde, 0x25, 0x1d, 0xb5, 0xe2, 0xc4, 0x23, 0x52, 0xe9, 0x1b, 0xa0, 0xcf, 0x0f, 0x63, 0xaf, 0x05, 0xe5, 0x06, 0xf2, 0x2c, 0x23, 0x7d, 0x1e, 0xb0, 0x37, 0x5e, 0x46, 0xc9, 0xcb, 0xd9, 0x8b, 0x48, 0x72, 0xe3, 0x45, 0x3b,  }
#define STAGE2_RAND_INT 14661271
#define STAGE1_PIN_HASH { 0x72, 0x38, 0x8d, 0xff, 0x73, 0xb1, 0x53, 0xb5, 0x8b, 0xb4, 0x43, 0x07, 0x51, 0x4f, 0x7c, 0x8b, 0x3e, 0xf5, 0x9e, 0x95, 0xdb, 0x9c, 0x54, 0x7a, 0x26, 0xce, 0xc9, 0xf6, 0x77, 0x61, 0x06, 0x07, 0xe4, 0x43, 0x8c, 0x7d, 0xdb, 0xd6, 0xa3, 0xa7, 0x25, 0xe9, 0x42, 0x3e, 0xd6, 0x0d, 0xc1, 0x0c, 0x59, 0x41, 0x52, 0x49, 0x62, 0xa5, 0x84, 0x80, 0xbf, 0x18, 0x94, 0x67, 0x8c, 0x5d, 0x41, 0xd4,  }

extern const uint8_t TRANSFER_KEY[32];

extern const group_t groups[1];

```

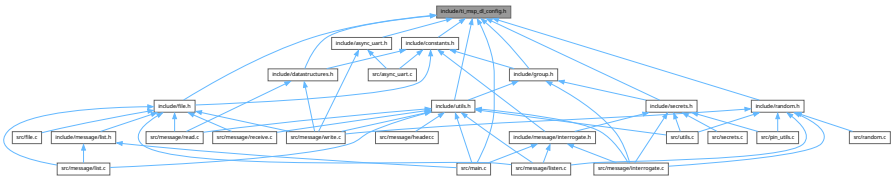
7.34 include/ti_msp_dl_config.h File Reference

```
#include <ti/devices/msp/msp.h>
#include <ti/driverlib/driverlib.h>
#include <ti/driverlib/m0p/dl_core.h>
```

Include dependency graph for ti_msp_dl_config.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define CONFIG_LP_MSPM0L2228
- #define CONFIG_MSPM0L2228
- #define POWER_STARTUP_DELAY (16)
- #define CPUCLK_FREQ 32000000
- #define HOST_INST UART0
- #define HOST_INST_FREQUENCY 32000000
- #define HOST_INST_IRQHandler UART0_IRQHandler
- #define HOST_INST_INT_IRQN UART0_INT_IRQn
- #define GPIO_HOST_RX_PORT GPIOA
- #define GPIO_HOST_TX_PORT GPIOA
- #define GPIO_HOST_RX_PIN DL_GPIO_PIN_11
- #define GPIO_HOST_TX_PIN DL_GPIO_PIN_10
- #define GPIO_HOST_IOMUX_RX (IOMUX_PINCM26)
- #define GPIO_HOST_IOMUX_TX (IOMUX_PINCM25)
- #define GPIO_HOST_IOMUX_RX_FUNC IOMUX_PINCM26_PF_UART0_RX
- #define GPIO_HOST_IOMUX_TX_FUNC IOMUX_PINCM25_PF_UART0_TX
- #define HOST_BAUD_RATE (115200)
- #define HOST_IBRD_32_MHZ_115200_BAUD (17)
- #define HOST_FBRD_32_MHZ_115200_BAUD (23)
- #define HSM_INST UART1
- #define HSM_INST_FREQUENCY 32000000
- #define HSM_INST_IRQHandler UART1_IRQHandler
- #define HSM_INST_INT_IRQN UART1_INT_IRQn
- #define GPIO_HSM_RX_PORT GPIOA
- #define GPIO_HSM_TX_PORT GPIOA
- #define GPIO_HSM_RX_PIN DL_GPIO_PIN_9
- #define GPIO_HSM_TX_PIN DL_GPIO_PIN_8
- #define GPIO_HSM_IOMUX_RX (IOMUX_PINCM20)
- #define GPIO_HSM_IOMUX_TX (IOMUX_PINCM19)
- #define GPIO_HSM_IOMUX_RX_FUNC IOMUX_PINCM20_PF_UART1_RX
- #define GPIO_HSM_IOMUX_TX_FUNC IOMUX_PINCM19_PF_UART1_TX
- #define HSM_BAUD_RATE (115200)
- #define HSM_IBRD_32_MHZ_115200_BAUD (17)
- #define HSM_FBRD_32_MHZ_115200_BAUD (23)
- #define TRNG_INT_IRQN (TRNG_INT_IRQn)
- #define TRNG_INT_IIDX (DL_INTERRUPT_GROUP1_IIDX_TRNG)
- #define WWDT0_INST (WWDT0)
- #define WWDT0_INT_IRQN (WWDT0_INT_IRQn)

Functions

- void SYSCFG_DL_init (void)
- void SYSCFG_DL_initPower (void)
- void SYSCFG_DL_GPIO_init (void)
- void SYSCFG_DL_SYSCTL_init (void)
- void SYSCFG_DL_HOST_init (void)
- void SYSCFG_DL_HSM_init (void)
- void SYSCFG_DL_TRNG_init (void)
- void SYSCFG_DL_WWDT0_init (void)

7.34.1 Macro Definition Documentation

7.34.1.1 CONFIG_LP_MSPM0L2228

```
#define CONFIG_LP_MSPM0L2228
```

7.34.1.2 CONFIG_MSPM0L2228

```
#define CONFIG_MSPM0L2228
```

7.34.1.3 CPUCLK_FREQ

```
#define CPUCLK_FREQ 32000000
```

7.34.1.4 GPIO_HOST_IOMUX_RX

```
#define GPIO_HOST_IOMUX_RX (IOMUX_PINCM26)
```

7.34.1.5 GPIO_HOST_IOMUX_RX_FUNC

```
#define GPIO_HOST_IOMUX_RX_FUNC IOMUX_PINCM26_PF_UART0_RX
```

7.34.1.6 GPIO_HOST_IOMUX_TX

```
#define GPIO_HOST_IOMUX_TX (IOMUX_PINCM25)
```

7.34.1.7 GPIO_HOST_IOMUX_TX_FUNC

```
#define GPIO_HOST_IOMUX_TX_FUNC IOMUX_PINCM25_PF_UART0_TX
```

7.34.1.8 GPIO_HOST_RX_PIN

```
#define GPIO_HOST_RX_PIN DL_GPIO_PIN_11
```

7.34.1.9 GPIO_HOST_RX_PORT

```
#define GPIO_HOST_RX_PORT GPIOA
```

7.34.1.10 GPIO_HOST_TX_PIN

```
#define GPIO_HOST_TX_PIN DL_GPIO_PIN_10
```

7.34.1.11 GPIO_HOST_TX_PORT

```
#define GPIO_HOST_TX_PORT GPIOA
```

7.34.1.12 GPIO_HSM_IOMUX_RX

```
#define GPIO_HSM_IOMUX_RX (IOMUX_PINCM20)
```

7.34.1.13 GPIO_HSM_IOMUX_RX_FUNC

```
#define GPIO_HSM_IOMUX_RX_FUNC IOMUX_PINCM20_PF_UART1_RX
```

7.34.1.14 GPIO_HSM_IOMUX_TX

```
#define GPIO_HSM_IOMUX_TX (IOMUX_PINCM19)
```

7.34.1.15 GPIO_HSM_IOMUX_TX_FUNC

```
#define GPIO_HSM_IOMUX_TX_FUNC IOMUX_PINCM19_PF_UART1_TX
```

7.34.1.16 GPIO_HSM_RX_PIN

```
#define GPIO_HSM_RX_PIN DL_GPIO_PIN_9
```

7.34.1.17 GPIO_HSM_RX_PORT

```
#define GPIO_HSM_RX_PORT GPIOA
```

7.34.1.18 GPIO_HSM_TX_PIN

```
#define GPIO_HSM_TX_PIN DL_GPIO_PIN_8
```

7.34.1.19 GPIO_HSM_TX_PORT

```
#define GPIO_HSM_TX_PORT GPIOA
```

7.34.1.20 HOST_BAUD_RATE

```
#define HOST_BAUD_RATE (115200)
```

7.34.1.21 HOST_FBRD_32_MHZ_115200_BAUD

```
#define HOST_FBRD_32_MHZ_115200_BAUD (23)
```

7.34.1.22 HOST_IBRD_32_MHZ_115200_BAUD

```
#define HOST_IBRD_32_MHZ_115200_BAUD (17)
```

7.34.1.23 HOST_INST

```
#define HOST_INST UART0
```

7.34.1.24 HOST_INST_FREQUENCY

```
#define HOST_INST_FREQUENCY 32000000
```

7.34.1.25 HOST_INST_INT_IRQN

```
#define HOST_INST_INT_IRQN UART0_INT_IRQn
```

7.34.1.26 HOST_INST_IRQHandler

```
#define HOST_INST_IRQHandler UART0_IRQHandler
```

7.34.1.27 HSM_BAUD_RATE

```
#define HSM_BAUD_RATE (115200)
```

7.34.1.28 HSM_FBRD_32_MHZ_115200_BAUD

```
#define HSM_FBRD_32_MHZ_115200_BAUD (23)
```

7.34.1.29 HSM_IBRD_32_MHZ_115200_BAUD

```
#define HSM_IBRD_32_MHZ_115200_BAUD (17)
```

7.34.1.30 HSM_INST

```
#define HSM_INST UART1
```

7.34.1.31 HSM_INST_FREQUENCY

```
#define HSM_INST_FREQUENCY 32000000
```

7.34.1.32 HSM_INST_INT_IRQN

```
#define HSM_INST_INT_IRQN UART1_INT_IRQn
```

7.34.1.33 HSM_INST_IRQHandler

```
#define HSM_INST_IRQHandler UART1_IRQHandler
```

7.34.1.34 POWER_STARTUP_DELAY

```
#define POWER_STARTUP_DELAY (16)
```

7.34.1.35 TRNG_INT_IIDX

```
#define TRNG_INT_IIDX (DL_INTERRUPT_GROUP1_IIDX_TRNG)
```

7.34.1.36 TRNG_INT_IRQN

```
#define TRNG_INT_IRQN (TRNG_INT_IRQn)
```

7.34.1.37 WWDT0_INST

```
#define WWDT0_INST (WWDT0)
```

7.34.1.38 WWDT0_INT_IRQN

```
#define WWDT0_INT_IRQN (WWDT0_INT_IRQn)
```

7.34.2 Function Documentation

7.34.2.1 SYSCFG_DL_GPIO_init()

```
void SYSCFG_DL_GPIO_init (
            void )
```

7.34.2.2 SYSCFG_DL_HOST_init()

```
void SYSCFG_DL_HOST_init (
            void )
```

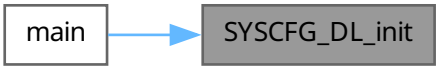
7.34.2.3 SYSCFG_DL_HSM_init()

```
void SYSCFG_DL_HSM_init (
            void )
```

7.34.2.4 SYSCFG_DL_init()

```
void SYSCFG_DL_init (
            void )
```

Here is the caller graph for this function:



7.34.2.5 SYSCFG_DL_initPower()

```
void SYSCFG_DL_initPower (
            void )
```

7.34.2.6 SYSCFG_DL_SYSCTL_init()

```
void SYSCFG_DL_SYSCTL_init (
            void )
```

7.34.2.7 SYSCFG_DL_TRNG_init()

```
void SYSCFG_DL_TRNG_init (
            void )
```

7.34.2.8 SYSCFG_DL_WWDT0_init()

```
void SYSCFG_DL_WWDT0_init (
            void )
```

7.35 ti_msp_dl_config.h

Go to the documentation of this file.

```
/*
 * Copyright (c) 2023, Texas Instruments Incorporated - http://www.ti.com
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 *
 * *  Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 *
 * *  Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 *
 * *  Neither the name of Texas Instruments Incorporated nor the names of
 *    its contributors may be used to endorse or promote products derived
 *    from this software without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR
 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS;
 * OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE,
 * EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
 */

/*
 *  ============ ti_msp_dl_config.h =============
 *  Configured MSPM0 DriverLib module declarations
 *
 *  DO NOT EDIT - This file is generated for the LP_MSPM0L2228
 *  by the SysConfig tool.
 */
#ifndef ti_msp_dl_config_h
#define ti_msp_dl_config_h

#define CONFIG_LP_MSPM0L2228
#define CONFIG_MSPM0L2228

#if defined(__ti_version__) || defined(__TI_COMPILER_VERSION__)
#define SYSCONFIG_WEAK __attribute__((weak))
#elif defined(__IAR_SYSTEMS_ICC__)
#define SYSCONFIG_WEAK __weak
#elif defined(__GNUC__)
```

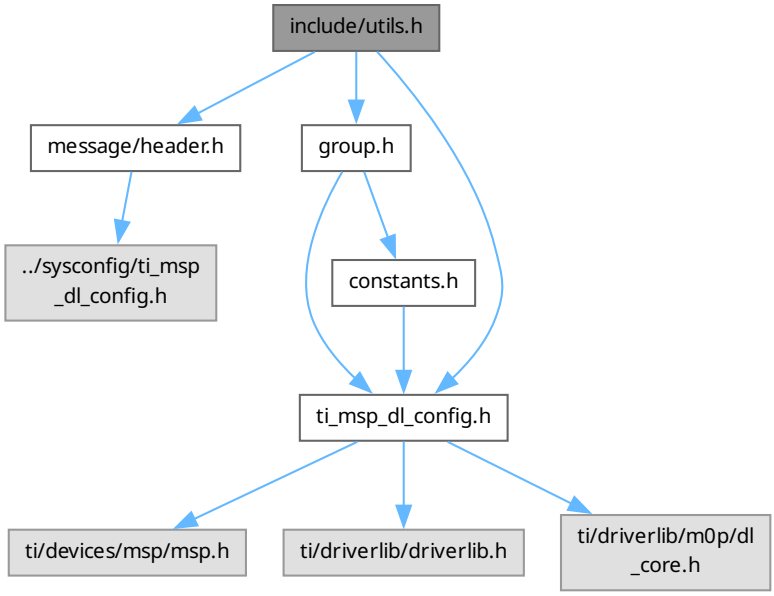
```
#define SYSCONFIG_WEAK __attribute__((weak))
#endif

#include <ti/devices/msp/msp.h>
#include <ti/driverlib/driverlib.h>
#include <ti/driverlib/m0p/dl_core.h>

#ifdef __cplusplus
extern "C" {
#endif

/*
 *  ======== SYSCFG_DL_init ========
 *  Perform all required MSP DL initialization
 *
 *  This function should be called once at a point before any use of
 *  MSP DL.
 */


/* clang-format off */

#define POWER_STARTUP_DELAY                                                (16)


#define CPUCLK_FREQ                                                     32000000



/* Defines for HOST */
#define HOST_INST                                                          UART0
#define HOST_INST_FREQUENCY                                             32000000
#define HOST_INST_IRQHandler                                       UART0_IRQHandler
#define HOST_INST_INT_IRQN                                           UART0_INT_IRQn
#define GPIO_HOST_RX_PORT                                                  GPIOA
#define GPIO_HOST_TX_PORT                                                  GPIOA
#define GPIO_HOST_RX_PIN                                             DL_GPIO_PIN_11
#define GPIO_HOST_TX_PIN                                             DL_GPIO_PIN_10
#define GPIO_HOST_IOMUX_RX                                          (IOMUX_PINCM26)
#define GPIO_HOST_IOMUX_TX                                          (IOMUX_PINCM25)
#define GPIO_HOST_IOMUX_RX_FUNC                          IOMUX_PINCM26_PF_UART0_RX
#define GPIO_HOST_IOMUX_TX_FUNC                          IOMUX_PINCM25_PF_UART0_TX
#define HOST_BAUD_RATE                                                  (115200)
#define HOST_IBRD_32_MHZ_115200_BAUD                                        (17)
#define HOST_FBRD_32_MHZ_115200_BAUD                                        (23)
/* Defines for HSM */
#define HSM_INST                                                           UART1
#define HSM_INST_FREQUENCY                                              32000000
#define HSM_INST_IRQHandler                                        UART1_IRQHandler
#define HSM_INST_INT_IRQN                                            UART1_INT_IRQn
#define GPIO_HSM_RX_PORT                                                   GPIOA
#define GPIO_HSM_TX_PORT                                                   GPIOA
#define GPIO_HSM_RX_PIN                                               DL_GPIO_PIN_9
#define GPIO_HSM_TX_PIN                                               DL_GPIO_PIN_8
#define GPIO_HSM_IOMUX_RX                                           (IOMUX_PINCM20)
#define GPIO_HSM_IOMUX_TX                                           (IOMUX_PINCM19)
#define GPIO_HSM_IOMUX_RX_FUNC                           IOMUX_PINCM20_PF_UART1_RX
#define GPIO_HSM_IOMUX_TX_FUNC                           IOMUX_PINCM19_PF_UART1_TX
#define HSM_BAUD_RATE                                                   (115200)
#define HSM_IBRD_32_MHZ_115200_BAUD                                         (17)
#define HSM_FBRD_32_MHZ_115200_BAUD                                         (23)



/* Defines for TRNG */
/*
 * The TRNG interrupt is part of INT_GROUP1. Refer to the TRM for more details
 * on interrupt grouping
 */
#define TRNG_INT_IRQN                                              (TRNG_INT_IRQn)
#define TRNG_INT_IIDX                                (DL_INTERRUPT_GROUP1_IIDX_TRNG)



/* Defines for WWDT */
#define WWDT0_INST                                                        (WWDT0)
#define WWDT0_INT_IRQN                                          (WWDT0_INT_IRQn)



/* clang-format on */

void SYSCFG_DL_init(void);
void SYSCFG_DL_initPower(void);
void SYSCFG_DL_GPIO_init(void);
void SYSCFG_DL_SYSCTL_init(void);
```

```
void SYSCFG_DL_HOST_init(void);
void SYSCFG_DL_HSM_init(void);
void SYSCFG_DL_TRNG_init(void);
void SYSCFG_DL_WWDT0_init(void);


#ifdef __cplusplus
}
#endif

#endif /* ti_msp_dl_config_h */
```

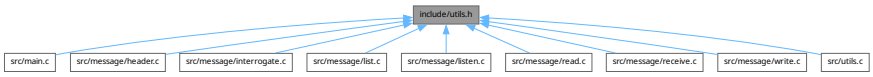
7.36 include/utils.h File Reference

```
#include "message/header.h"
#include "group.h"
#include "ti_msp_dl_config.h"
```

Include dependency graph for utils.h:



This graph shows which files directly or indirectly include this file:



Functions

- void utils_random_delay ()

  Delay between 0 - 4096 cycles to mitigate timing attacks.

- bool utils_verify_pin (uint8_t ∗pin, size_t pin_length)

  Verifies host PIN.
- void utils_send_buffer (UART_Regs ∗uart, const void ∗buffer, size_t length, size_t bytes_sent)

  Sends buffer over UART with ACKing.
- void utils_send_buffer_no_ack (UART_Regs ∗uart, const void ∗buffer, size_t length)

  Sends buffer over UART without ACKing.
- void utils_receive_bytes (UART_Regs ∗uart, void ∗buffer, size_t num_bytes)

  Receives specified number of bytes from UART.
- const group_t ∗ utils_find_group (uint16_t group_id)

  Finds group by ID.

7.36.1 Function Documentation

7.36.1.1 utils_find_group()

```
const group_t * utils_find_group (
            uint16_t group_id)
```

Finds group by ID.

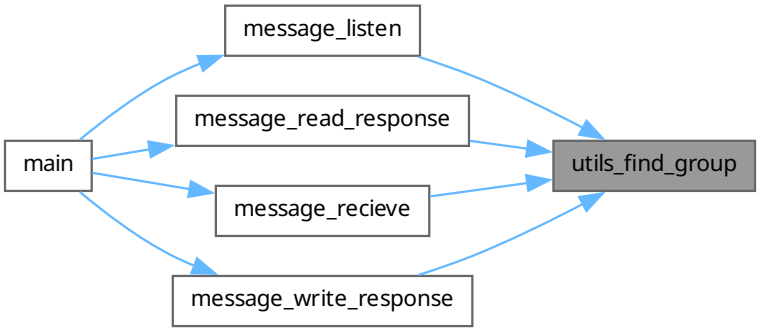
Parameters

in	*group↩ _id*	Group identifier to search for

Returns

Pointer to group if found, NULL otherwise

Here is the caller graph for this function:

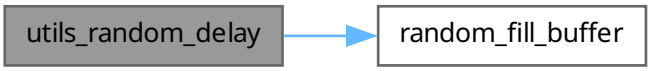


7.36.1.2 utils_random_delay()

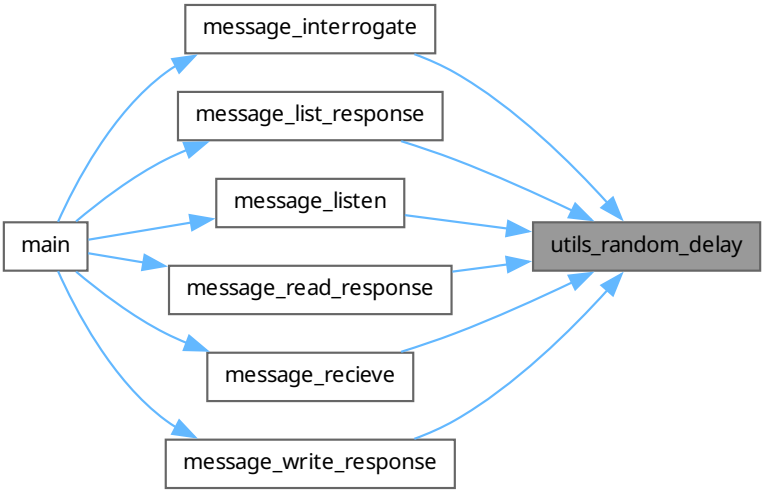
```
void utils_random_delay ()
```

Delay between 0 - 4096 cycles to mitigate timing attacks.

Here is the call graph for this function:



Here is the caller graph for this function:



7.36.1.3 utils_receive_bytes()

```
void utils_receive_bytes (
            UART_Regs * uart,
            void * buffer,
            size_t num_bytes)
```

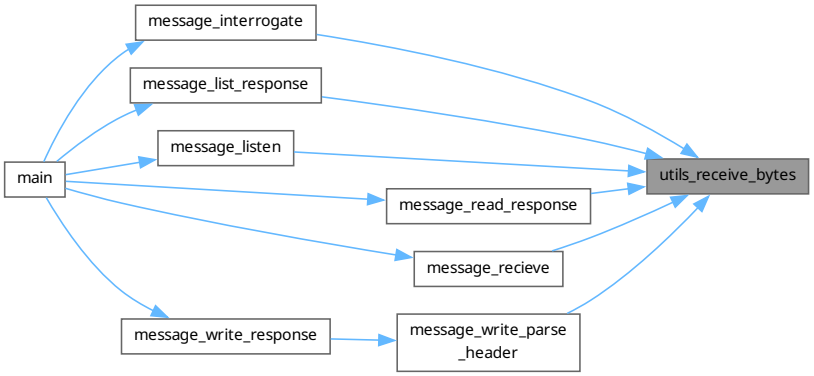
Receives specified number of bytes from UART.

Blocks until num_bytes have been read from the provided uart and stored into buffer.

Parameters

in	*uart*	UART peripheral
out	*buffer*	Destination buffer
in	*num_bytes*	Number of bytes to receive

Here is the caller graph for this function:



7.36.1.4 utils_send_buffer()

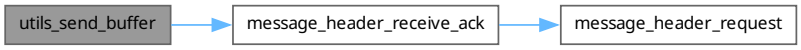
```
void utils_send_buffer (
            UART_Regs * uart,
            const void * buffer,
            size_t length,
            size_t bytes_sent)
```

Sends buffer over UART with ACKing.

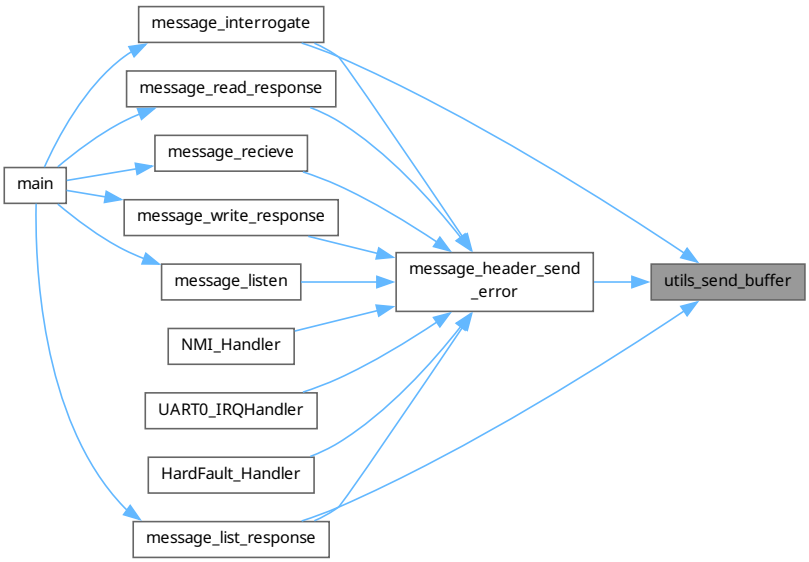
Parameters

in	*uart*	UART peripheral
in	*buffer*	Buffer to send
in	*length*	Buffer length
in	*bytes_sent*	Number of bytes already sent (for resumption)

Here is the call graph for this function:



Here is the caller graph for this function:



7.36.1.5 utils_send_buffer_no_ack()

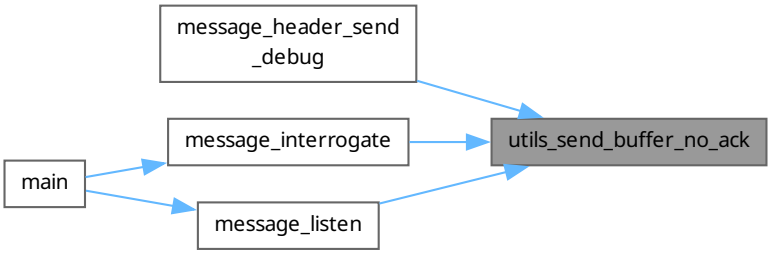
```
void utils_send_buffer_no_ack (
            UART_Regs * uart,
            const void * buffer,
            size_t length)
```

Sends buffer over UART without ACKing.

Parameters

in	*uart*	UART peripheral
in	*buffer*	Buffer to send
in	*length*	Buffer length

Here is the caller graph for this function:



7.36.1.6 utils_verify_pin()

```
bool utils_verify_pin (
            uint8_t * pin,
            size_t pin_length)
```

Verifies host PIN.

Parameters

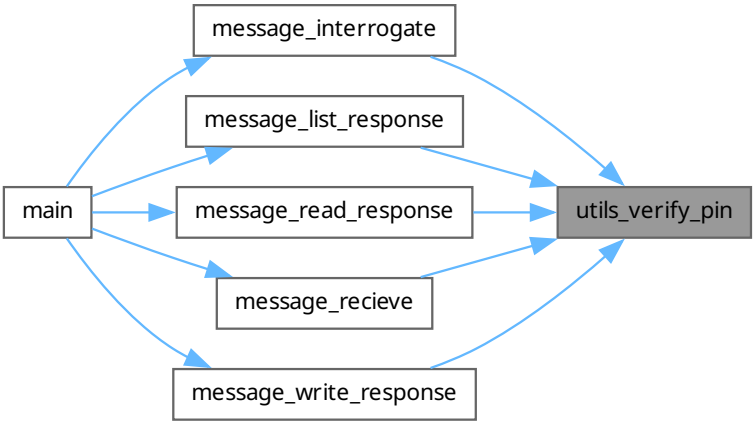
in	*pin*	PIN buffer
in	*pin_length*	Length of PIN

Returns

true if PIN is valid, false otherwise

Here is the call graph for this function:

Here is the caller graph for this function:



7.37 utils.h

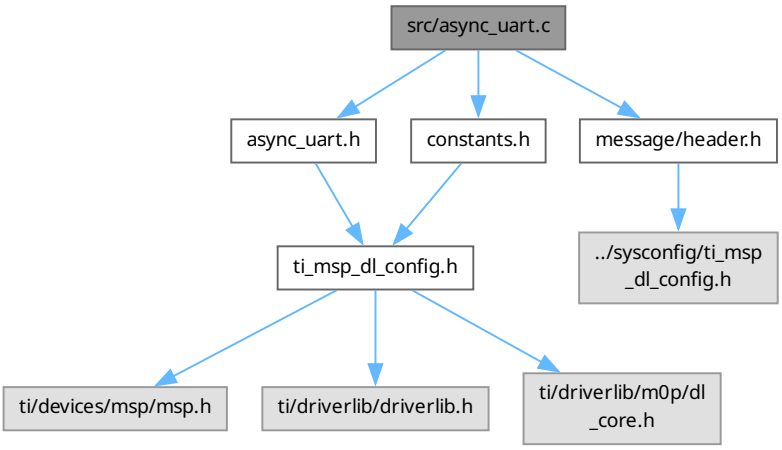
Go to the documentation of this file.

```
#pragma once
// #include <string.h>
// #include <stdio.h>
// #include <stdlib.h>
#include "message/header.h"
#include "group.h"

#include "ti_msp_dl_config.h"

void utils_random_delay();

bool utils_verify_pin(uint8_t* pin, size_t pin_length);

void utils_send_buffer(UART_Regs * uart, const void* buffer, size_t length, size_t bytes_sent);

void utils_send_buffer_no_ack(UART_Regs* uart, const void* buffer, size_t length);

void utils_receive_bytes(UART_Regs* uart, void* buffer, size_t num_bytes);

const group_t* utils_find_group(uint16_t group_id);


// #ifdef DEBUG
// #define _WRITE_TO(FUNC, ...) do{                                  \
//   char* _buffer;                                                  \
//   if (-1 != asprintf(&_buffer, __VA_ARGS__)){ \
//     FUNC(HOST_INST, _buffer, strlen(_buffer)); \
//     free(_buffer); \
//   } \
// } while(0);

// #define print_debug(...) _WRITE_TO(message_header_send_debug, __VA_ARGS__)
// #define print_error(...) _WRITE_TO(message_header_send_error, __VA_ARGS__)


// #else

// #define print_debug(...) {}
// #define print_error(...)  message_header_send_error(HOST_INST, 0, 0)

// #endif
```

7.38 src/async_uart.c File Reference

```
#include "async_uart.h"
#include "message/header.h"
#include "constants.h"
```

Include dependency graph for async_uart.c:



Functions

- int async_uart_receive (async_uart_ctx *ctx)

  Starts asynchronous UART receive operation.
- void DMA_IRQHandler (void)

Variables

- async_uart_ctx * transfers [16]

7.38.1 Function Documentation

7.38.1.1 async_uart_receive()

```
int async_uart_receive (
            async_uart_ctx * ctx)
```

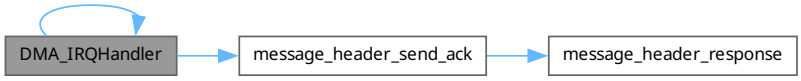
Starts asynchronous UART receive operation.

This function fills the buffer passed in the ctx struct with bytes from the UART. We utilize the DMA interrupt to ACK every 256 bytes to allow other operations to be performed concurrently with IO. Here is the caller graph for this function:

7.38.1.2 DMA_IRQHandler()

```
void DMA_IRQHandler (
            void )
```

Here is the call graph for this function:



Here is the caller graph for this function:

7.38.2 Variable Documentation

7.38.2.1 transfers

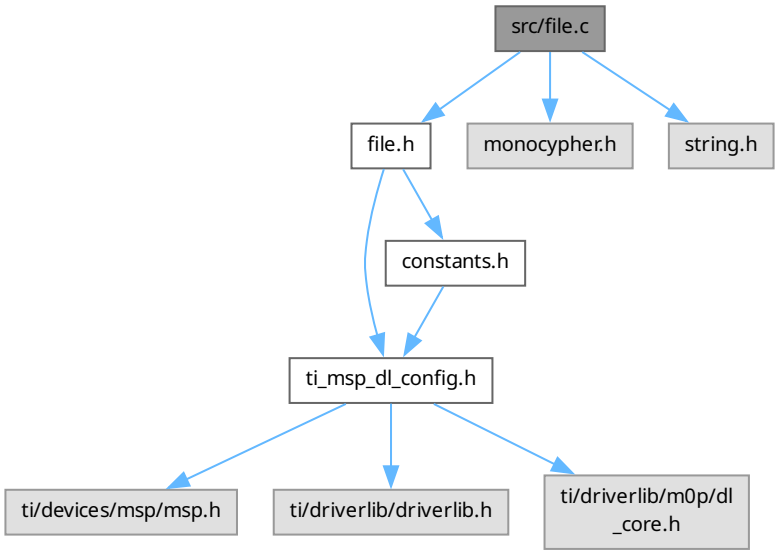
```
async_uart_ctx* transfers[16]
```

7.39 src/file.c File Reference

```
#include "file.h"
#include <monocypher.h>
```

```
#include <string.h>
```

Include dependency graph for file.c:



Functions

- int file_read_fat (file_address_table_t *fat)

  Reads the FAT table from flash.
- int file_write_fat (file_address_table_t *fat)

  Writes the FAT table to flash.

Variables

- file_address_table_t FILE_ADDRESS_TABLE
- file_slot_entry_t SLOTS [NUM_SLOTS]

7.39.1 Function Documentation

7.39.1.1 file_read_fat()

```
int file_read_fat (
            file_address_table_t * fat)
```

Reads the FAT table from flash.

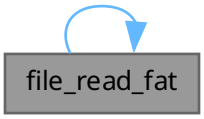
Parameters

out	*fat*	Pointer to FAT table structure to populate

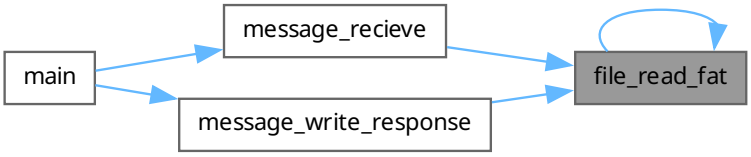
Returns

0 on success, negative value on error

Here is the call graph for this function:



Here is the caller graph for this function:



7.39.1.2 file_write_fat()

```
int file_write_fat (
            file_address_table_t * fat)
```

Writes the FAT table to flash.

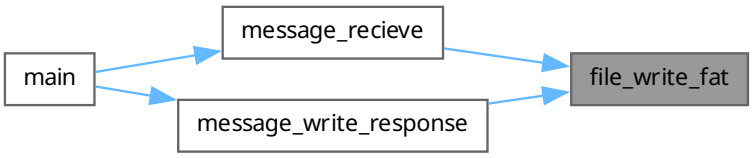
Parameters

in	*fat*	Pointer to FAT table structure to write

Returns

0 on success, negative value on error

Here is the caller graph for this function:



7.39.2 Variable Documentation

7.39.2.1 FILE_ADDRESS_TABLE

```
file_address_table_t FILE_ADDRESS_TABLE
```

Struct representing the File Address Table on Flash.

Note

This struct is only writable using the FlashCTL.

7.39.2.2 SLOTS

```
file_slot_entry_t SLOTS[NUM_SLOTS]
```

Struct representing the File System on Flash.

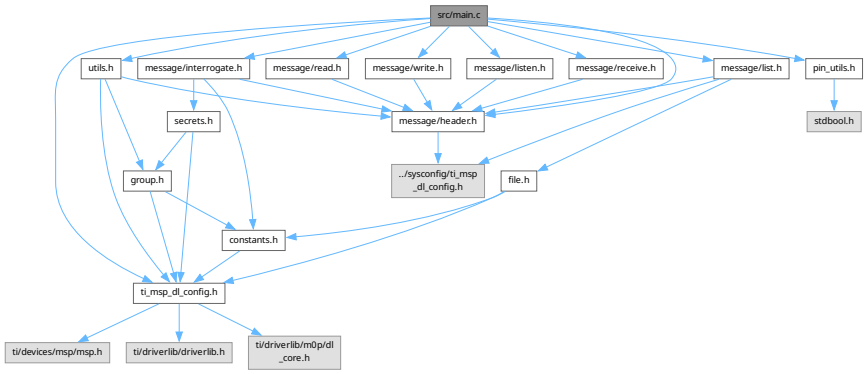
Note

This struct is only writable using the FlashCTL.

7.40 src/main.c File Reference

```
#include <ti_msp_dl_config.h>
#include "utils.h"
#include "message/header.h"
#include "message/list.h"
#include "message/read.h"
#include "message/write.h"
#include "message/listen.h"
#include "message/interrogate.h"
#include "message/receive.h"
#include "pin_utils.h"
```

Include dependency graph for main.c:



Functions

- int main (void)

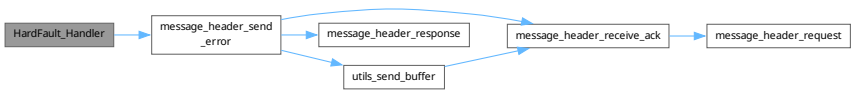
  Program Entrypoint.
- void NMI_Handler ()
- void HardFault_Handler ()
- void UART0_IRQHandler (void)

7.40.1 Function Documentation

7.40.1.1 HardFault_Handler()

```
void HardFault_Handler ()
```

Here is the call graph for this function:

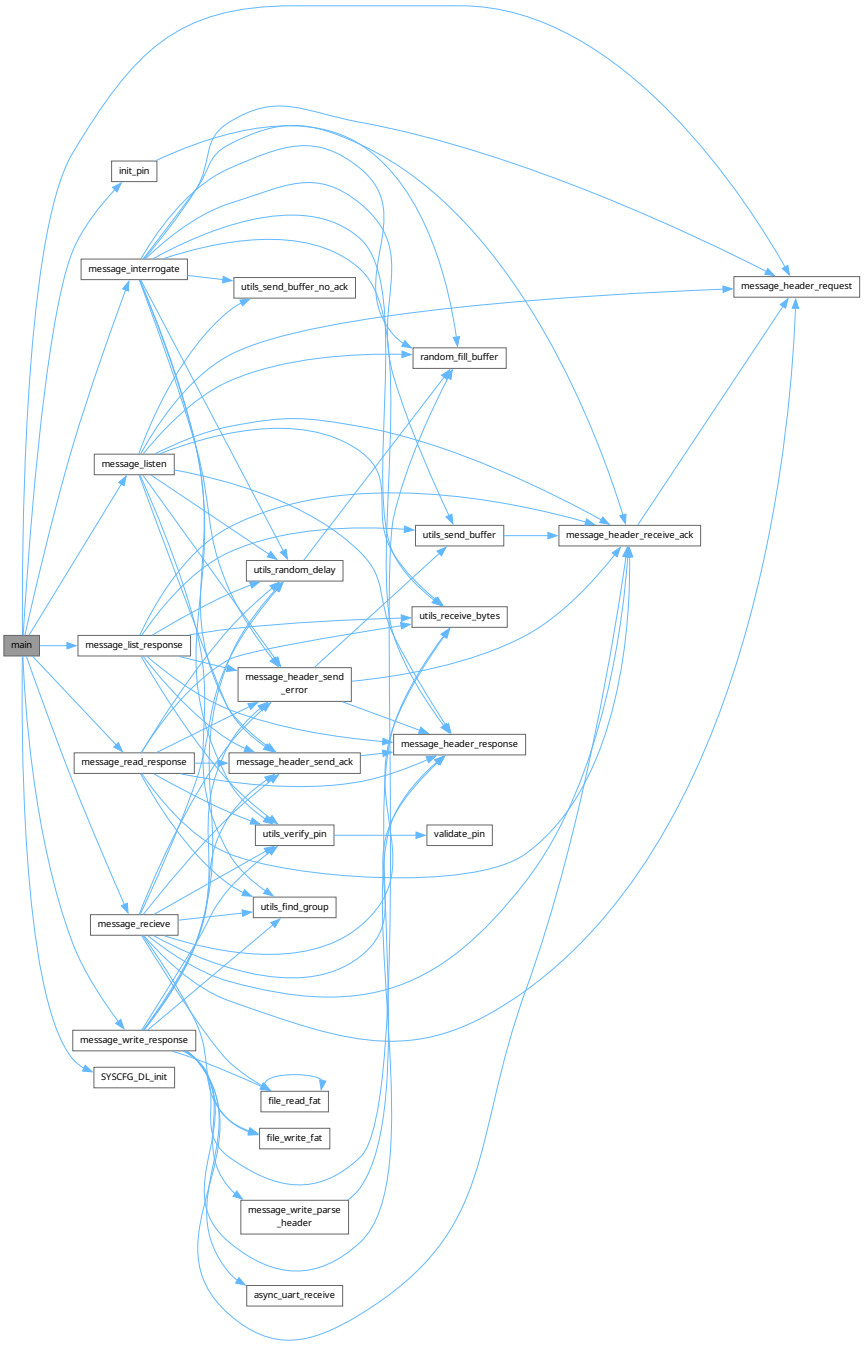


7.40.1.2 main()

```
int main (
            void )
```

Program Entrypoint.

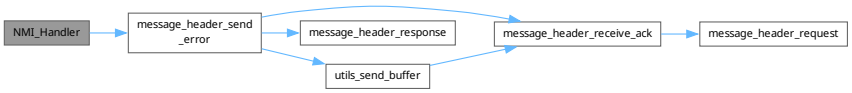
Here is the call graph for this function:



7.40.1.3 NMI_Handler()

```
void NMI_Handler ()
```

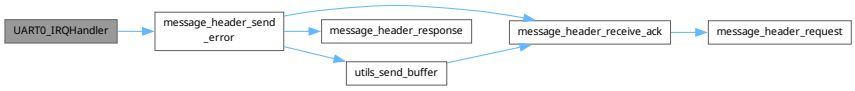
Here is the call graph for this function:



7.40.1.4 UART0_IRQHandler()

```
void UART0_IRQHandler (
            void )
```

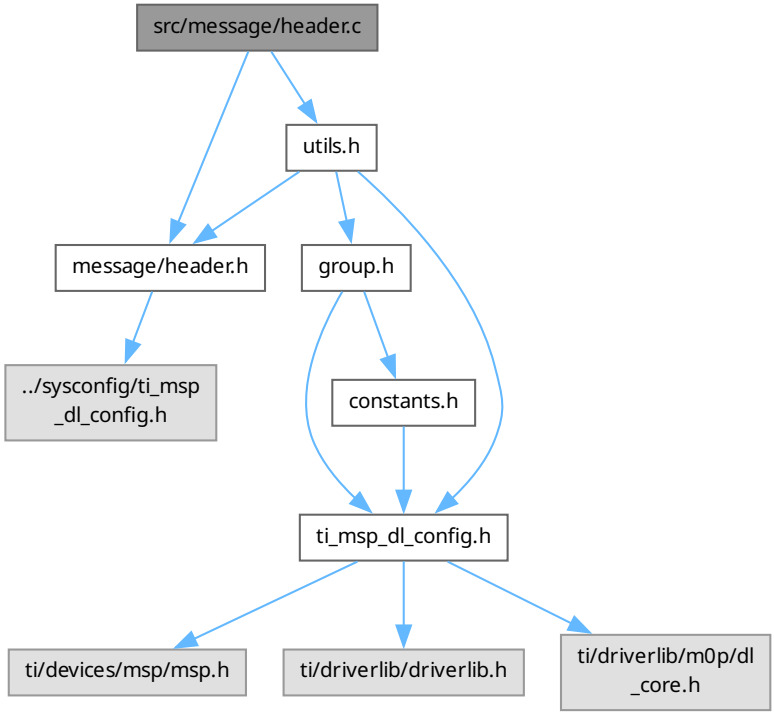
Here is the call graph for this function:



7.41 src/message/header.c File Reference

```
#include "message/header.h"
#include "utils.h"
```

Include dependency graph for header.c:



Functions

- int message_header_request (UART_Regs ∗uart, message_header_t ∗header)

  Parses message header from UART.
- void message_header_response (UART_Regs ∗uart, message_header_t header)

  Starts a response to the host tools.
- void message_header_send_error (UART_Regs ∗uart, const char ∗msg, size_t msg_len)

  Sends an error message to the host tools.
- void message_header_send_debug (UART_Regs ∗uart, const char ∗msg, size_t msg_len)

  Sends a debug message to the host tools.
- void message_header_send_ack (UART_Regs ∗uart)

  Sends an ACK message to the host tools.
- void message_header_receive_ack (UART_Regs ∗uart)

  Blocks until ACK message is received.

7.41.1 Function Documentation

7.41.1.1 message_header_receive_ack()

```
void message_header_receive_ack (
            UART_Regs * uart)
```

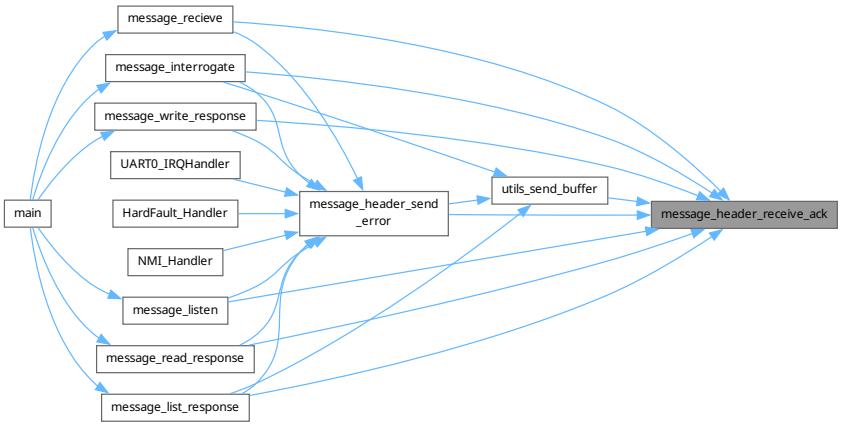
Blocks until ACK message is received.

Parameters

in	*uart*	UART registers for communication

Here is the call graph for this function:

Here is the caller graph for this function:



7.41.1.2 message_header_request()

```
int message_header_request (
            UART_Regs * uart,
            message_header_t * header)
```

Parses message header from UART.

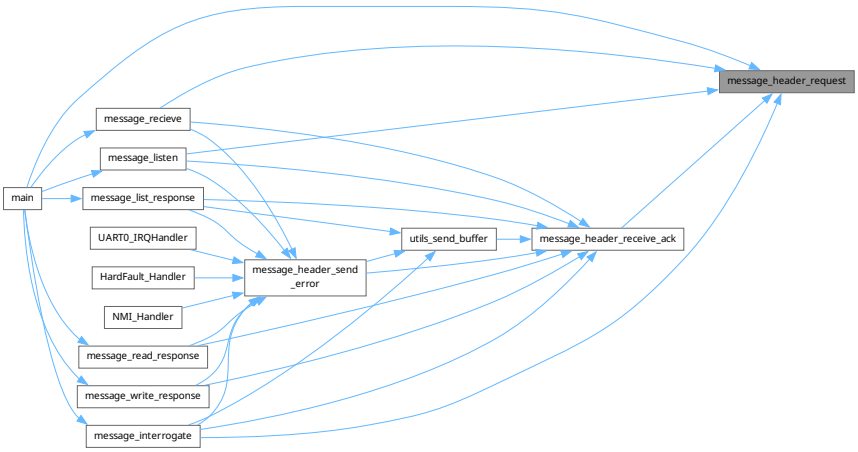
Parameters

in	*uart*	UART registers for communication
out	*header*	Output header structure

Returns

0 on success, negative value on error

Here is the caller graph for this function:



7.41.1.3 message_header_response()

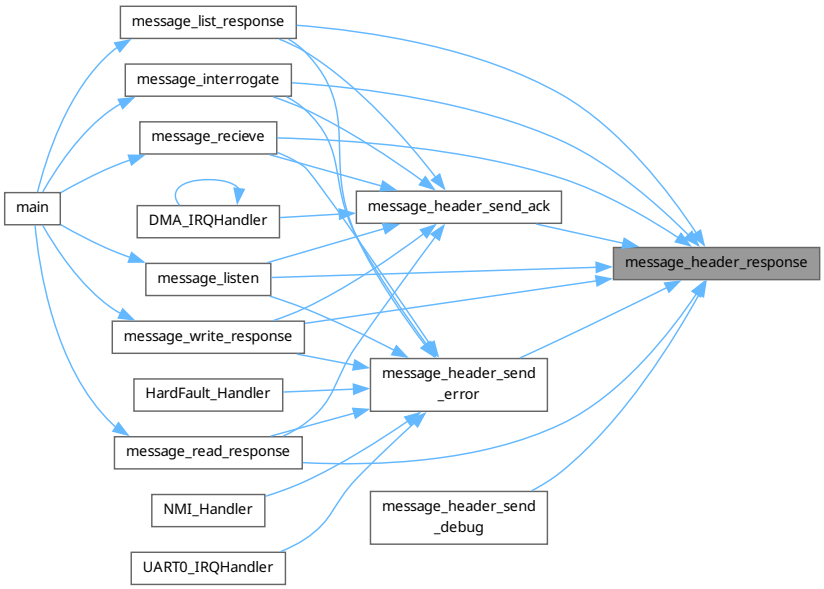
```
void message_header_response (
            UART_Regs * uart,
            message_header_t ctx)
```

Starts a response to the host tools.

Parameters

in	*uart*	UART registers for communication
in	*ctx*	Message header containing response context

Here is the caller graph for this function:



7.41.1.4 message_header_send_ack()

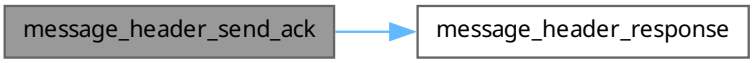
```
void message_header_send_ack (
            UART_Regs * uart)
```

Sends an ACK message to the host tools.

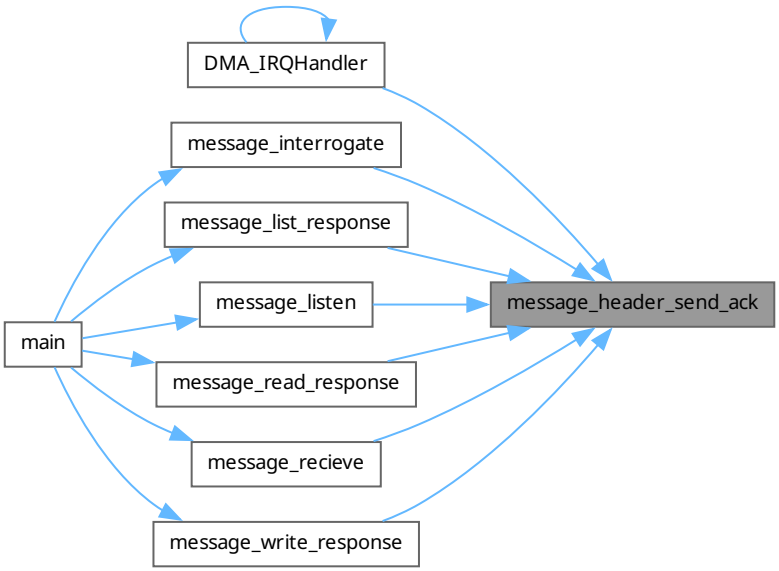
Parameters

in	*uart*	UART registers for communication

Here is the call graph for this function:



Here is the caller graph for this function:



7.41.1.5 message_header_send_debug()

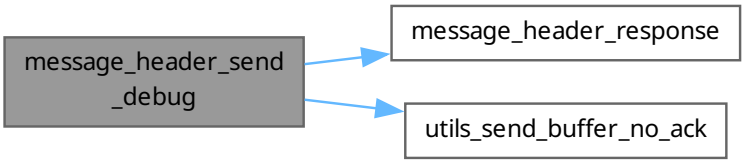
```
void message_header_send_debug (
            UART_Regs * uart,
            const char * msg,
            size_t msg_len)
```

Sends a debug message to the host tools.

Parameters

in	*uart*	UART registers for communication
in	*msg*	Debug message string
in	*msg_len*	Length of debug message

Here is the call graph for this function:



7.41.1.6 message_header_send_error()

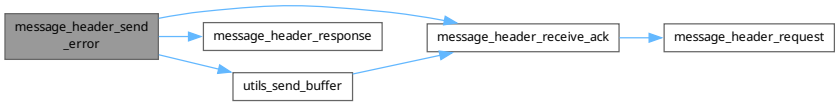
```
void message_header_send_error (
            UART_Regs * uart,
            const char * msg,
            size_t msg_len)
```

Sends an error message to the host tools.

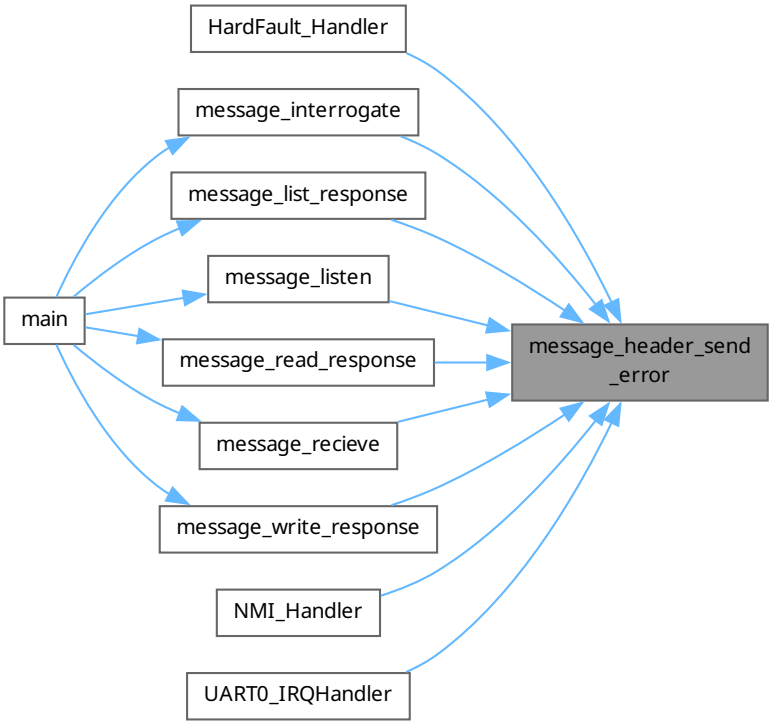
Parameters

in	*uart*	UART registers for communication
in	*msg*	Error message string
in	*msg_len*	Length of error message

Here is the call graph for this function:



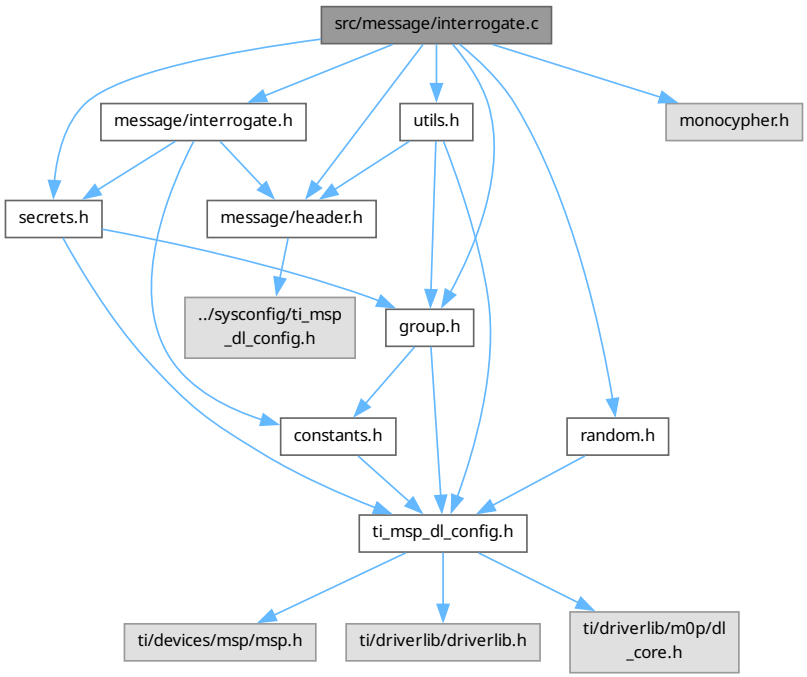
Here is the caller graph for this function:



7.42 src/message/interrogate.c File Reference

```
#include "message/interrogate.h"
#include "message/header.h"
#include "secrets.h"
#include "group.h"
#include "random.h"
#include "utils.h"
#include <monocypher.h>
```

Include dependency graph for interrogate.c:



Functions

- void message_interrogate (message_header_t header)

  Responds to Interrogate Request.

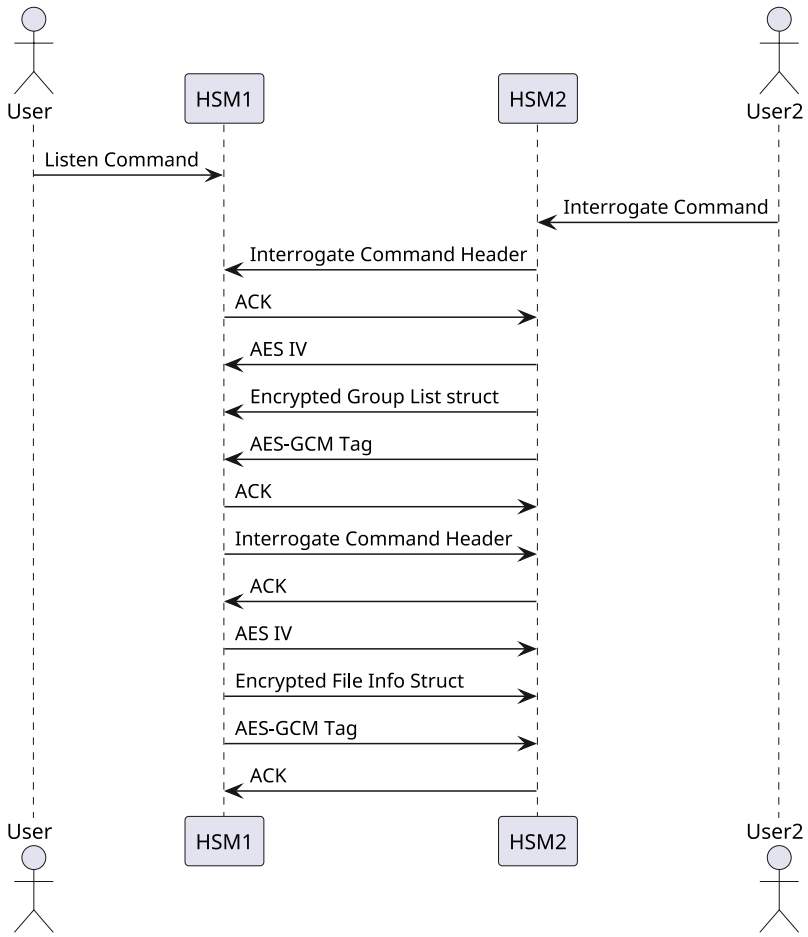
7.42.1 Function Documentation

7.42.1.1 message_interrogate()

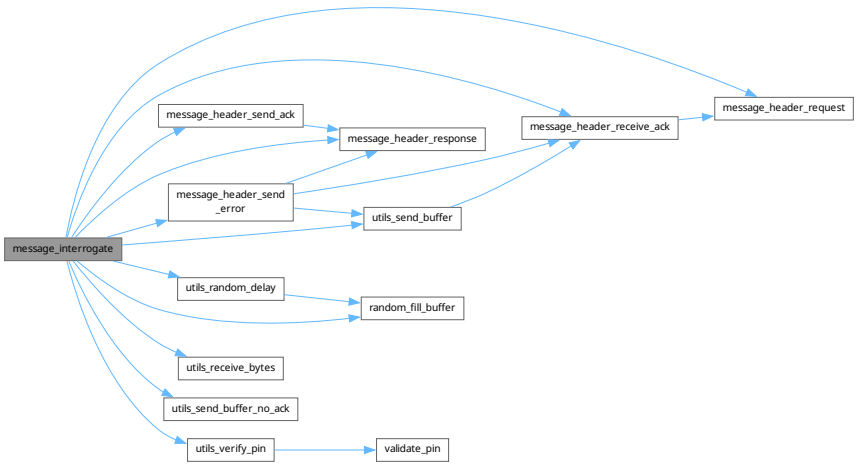
```
void message_interrogate (
            message_header_t header)
```

Responds to Interrogate Request.

Initally our plan for interrogate was to sign an ED25519 public key using a secret transfer key but that was not feasible given the time constraints. Since we couldn't use asymetric encryption for this task we fell back to using a simple static AES key. In theory you could still use signatures for verification but at the end of the day it would have little impact on security.



Here is the call graph for this function:

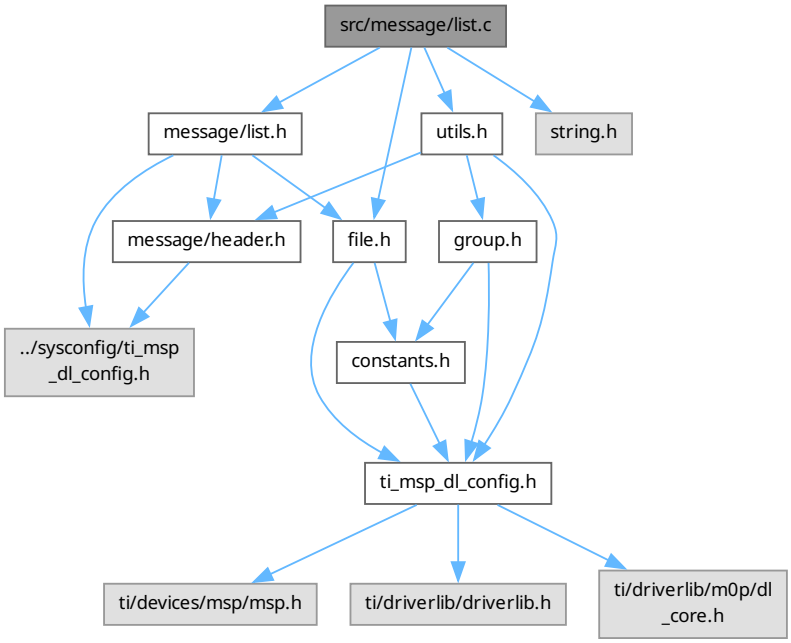


Here is the caller graph for this function:

7.43 src/message/list.c File Reference

```
#include "message/list.h"
#include "file.h"
#include "utils.h"
#include <string.h>
```

Include dependency graph for list.c:



Functions

- void message_list_response (message_header_t header)

  Responds to a List Request.

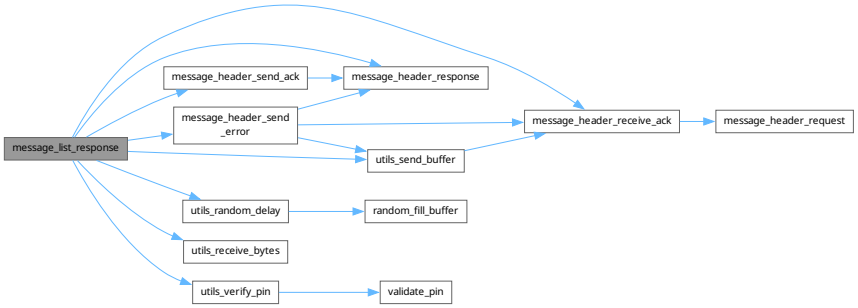
7.43.1 Function Documentation

7.43.1.1 message_list_response()

```
void message_list_response (
            message_header_t header)
```

Responds to a List Request.

List is fairly simple in function. We simply iterrate over the file system checking if the length of each structure is in less than the max file size and if it is less than the max file size we know a file exists in that slot. Here is the call graph for this function:



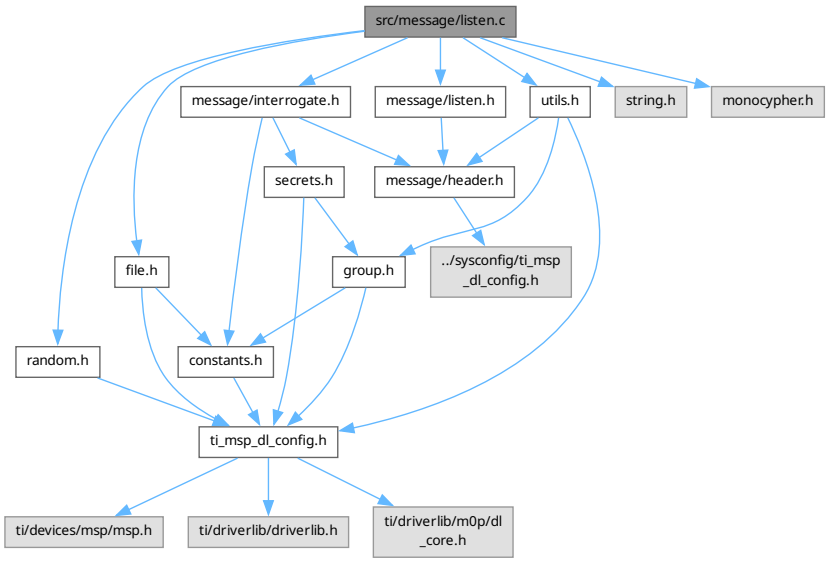
Here is the caller graph for this function:

7.44 src/message/listen.c File Reference

```
#include "message/listen.h"
#include "random.h"
#include "file.h"
#include "message/interrogate.h"
#include "utils.h"
#include <string.h>
```

```
#include <monocypher.h>
```

Include dependency graph for listen.c:



Macros

- #define DMA_CHANNEL 0

Functions

- void message_listen (message_header_t header)

  Responds to Listen Request.

7.44.1 Macro Definition Documentation

7.44.1.1 DMA_CHANNEL

```
#define DMA_CHANNEL 0
```

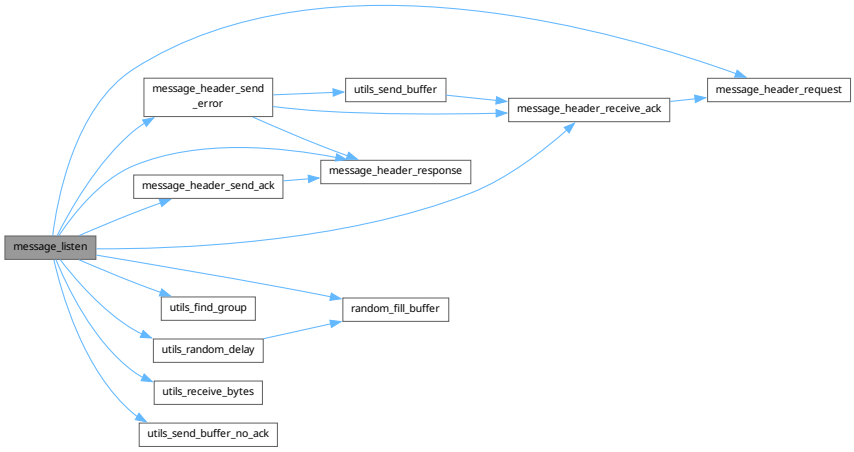
7.44.2 Function Documentation

7.44.2.1 message_listen()

```
void message_listen (
            message_header_t header)
```

Responds to Listen Request.

The listen is other end of the recieve command and the interrogate command. The listen command starts by recieving a header from the remote HSM and then based on the header opcode the listen command provides a response in accordance to the recieve command definition or the interrogate command definition. Here is the call graph for this function:

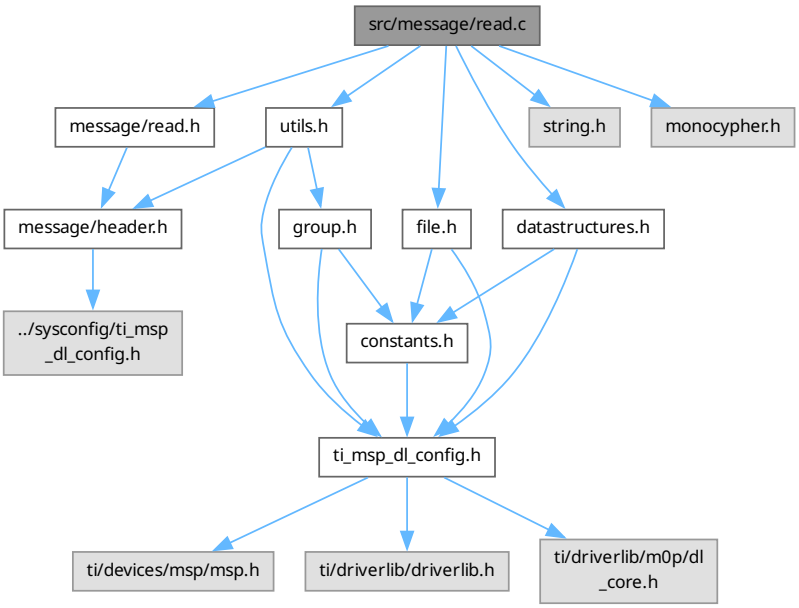


Here is the caller graph for this function:

7.45 src/message/read.c File Reference

```
#include "message/read.h"
#include "file.h"
#include "datastructures.h"
#include "utils.h"
#include <string.h>
#include <monocypher.h>
```

Include dependency graph for read.c:



Macros

- #define DMA_CHANNEL 2
- #define READ_HEADER_LENGTH 32

Functions

- void message_read_response (message_header_t header)

  Responds to Read Request.

Variables

- BufferNode buffers [2]

7.45.1 Macro Definition Documentation

7.45.1.1 DMA_CHANNEL

```
#define DMA_CHANNEL 2
```

7.45.1.2 READ_HEADER_LENGTH

```
#define READ_HEADER_LENGTH 32
```

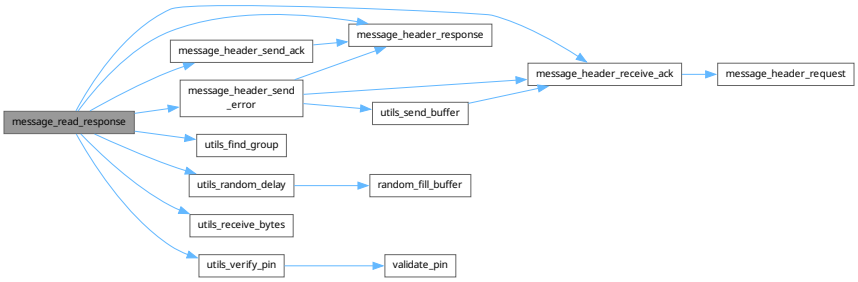
7.45.2 Function Documentation

7.45.2.1 message_read_response()

```
void message_read_response (
            message_header_t header)
```

Responds to Read Request.

The read command is a fairly simple command to process. We start by verifing that the metadata of the file is valid by verifing the signature that is stored alongside the metadata. After verifing the signature we generate the decryption AES secret using the read private key that the board should be provissioned with if it has valid permissions and the public key stored inside the signed metadata. The file is then decrypted in chunks and transfered over UART with the last chunk only being sent if the GCM tag is valid. Here is the call graph for this function:



Here is the caller graph for this function:

7.45.3 Variable Documentation

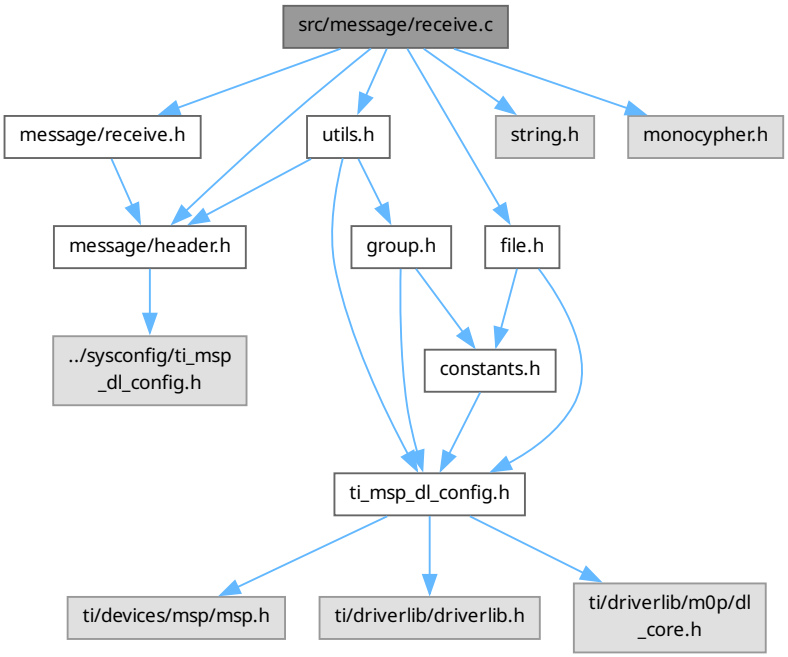
7.45.3.1 buffers

```
BufferNode buffers[2]
```

7.46 src/message/receive.c File Reference

```
#include "message/receive.h"
#include "message/header.h"
#include "file.h"
#include "utils.h"
#include <string.h>
#include <monocypher.h>
```

Include dependency graph for receive.c:



Functions

- void message_recieve (message_header_t header)

  Responds to Receive Request.

7.46.1 Function Documentation

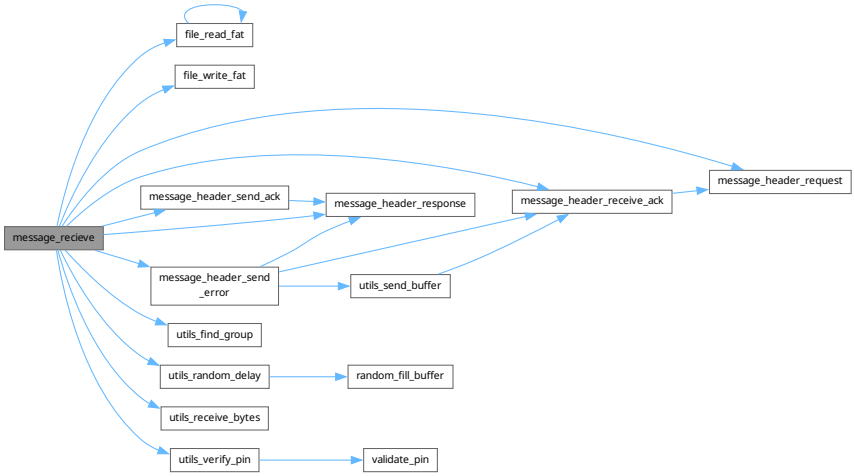
7.46.1.1 message_recieve()

```
void message_recieve (
            message_header_t header)
```

Responds to Receive Request.

Our recieve design uses asymetric encryption to ensure secrecy. Instead of trying to verify that an HSM can is allowed to decrypt a file we designed our system where we encrypt the file so it can only be decrypted by an HSM

that has the private transfer key for the file group. We accomplish this by generating the AES secret key from a randomly generated ED25519 keypair and the public transfer key that every HSM is provisioned with. We then send the public key from the generated keypair to the recieving HSM so that the secret can be regenerated with the private transfer key. We transfer the entire file_slot_entry_t as it is one less point of failure in our design.Here is the call graph for this function:

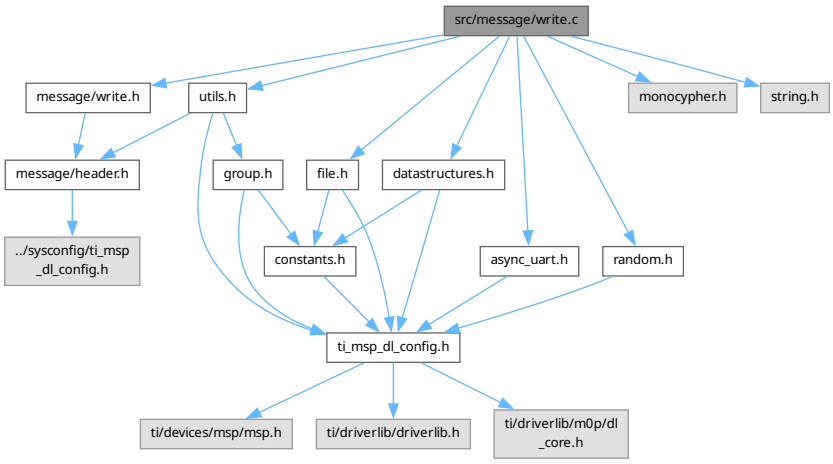


Here is the caller graph for this function:

7.47 src/message/write.c File Reference

```
#include "message/write.h"
#include "file.h"
#include "async_uart.h"
#include "datastructures.h"
#include "random.h"
#include "utils.h"
#include <monocypher.h>
#include <string.h>
```

Include dependency graph for write.c:



Macros

- #define DMA_CHANNEL 1

Functions

- message_write_header_t message_write_parse_header (UART_Regs ∗uart)

  Parses the header of the Write File command.
- void message_write_response (message_header_t header)

  Responds to Write Request.

Variables

- BufferNode buffers [2]

7.47.1 Macro Definition Documentation

7.47.1.1 DMA_CHANNEL

```
#define DMA_CHANNEL 1
```

7.47.2 Function Documentation

7.47.2.1 message_write_parse_header()

```
message_write_header_t message_write_parse_header (
            UART_Regs * uart)
```

Parses the header of the Write File command.

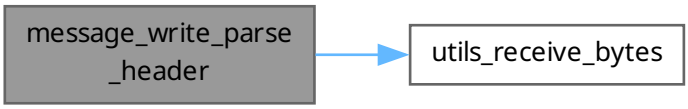
Parameters

in	*uart*	UART registers for communication

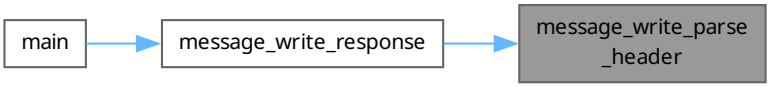
Returns

Write header structure containing parsed information

Here is the call graph for this function:



Here is the caller graph for this function:



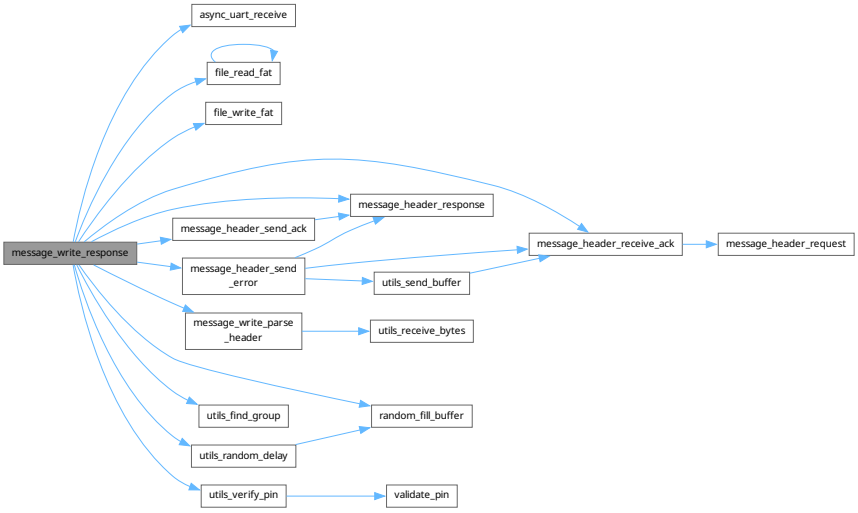
7.47.2.2 message_write_response()

```
void message_write_response (
            message_header_t header)
```

Responds to Write Request.

This was fun to write and sucked to debug. Our implementation works by concurrently reading bytes from the UART with DMA while the encryption keys are derived and calculated. The encryption calculations are interrupted by the DMA to send ACKs and to re-enable the DMA for each chunk. After the entire file is read into memory then the AES is used to encrypt the file using the generated secret. Each file gets a randomly generated key-pair that is used in combination with the read public key to generate the secret key for the AES encryption. We use AES-GCM so we can verify the file when being read back to the user. The public key from the random key-pair is stored in the

metadata that is then signed using the writer private key. This allows us to check that a file was created by a valid writer by verifing that the metadata signature is valid. Here is the call graph for this function:



Here is the caller graph for this function:

7.47.3 Variable Documentation

7.47.3.1 buffers

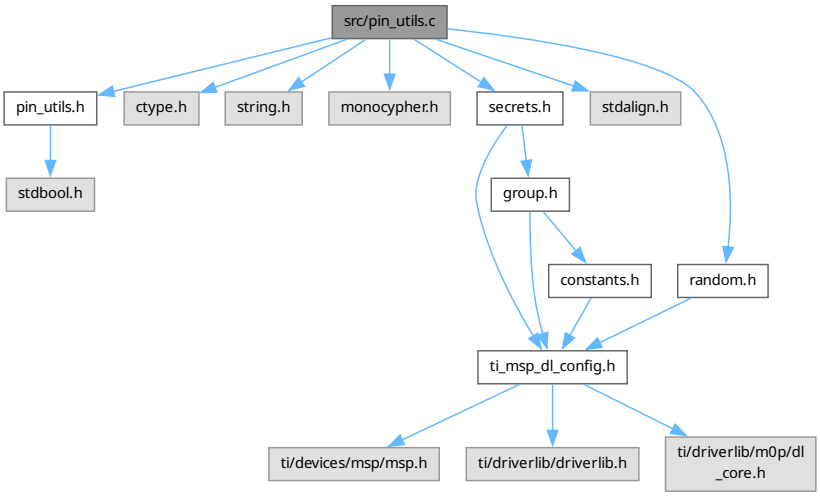
```
BufferNode buffers[2]
```

7.48 src/pin_utils.c File Reference

```
#include "pin_utils.h"
#include <ctype.h>
#include <string.h>
#include <monocypher.h>
#include "secrets.h"
#include <stdalign.h>
```

```
#include "random.h"
```

Include dependency graph for pin_utils.c:



Macros

- #define SEED (∗(int∗)&random_data[4∗__COUNTER__])
- #define COMPILER_FUNC_OPTS
- #define RANDOM_FUNC COMPILER_FUNC_OPTS[SEED % (sizeof(COMPILER_FUNC_OPTS) / sizeof(void∗))]

Functions

- bool validate_pin (const char pin[6])

  Validates PIN with 2-stage BLAKE2b hashing.
- void init_pin ()

  Initializes PIN validation system.

7.48.1 Macro Definition Documentation

7.48.1.1 COMPILER_FUNC_OPTS

```
#define COMPILER_FUNC_OPTS
```

Value:

```
   ((uint8_t*[]) {\
   (uint8_t*) &validate_pin,      \
   (uint8_t*) &apply_stage1,      \
   (uint8_t*) &apply_stage2,      \
   (uint8_t*) &_unused,      \
})
```

7.48.1.2 RANDOM_FUNC

```
#define RANDOM_FUNC COMPILER_FUNC_OPTS[SEED % (sizeof(COMPILER_FUNC_OPTS) / sizeof(void*))]
```

7.48.1.3 SEED

```
#define SEED ( *(int*)&random_data[4*__COUNTER__] )
```

7.48.2 Function Documentation

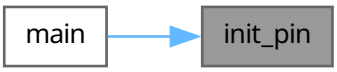
7.48.2.1 init_pin()

```
void init_pin ()
```

Initializes PIN validation system.

Here is the call graph for this function:

Here is the caller graph for this function:



7.48.2.2 validate_pin()

```
bool validate_pin (
            const char pin[6])
```

Validates PIN with 2-stage BLAKE2b hashing.

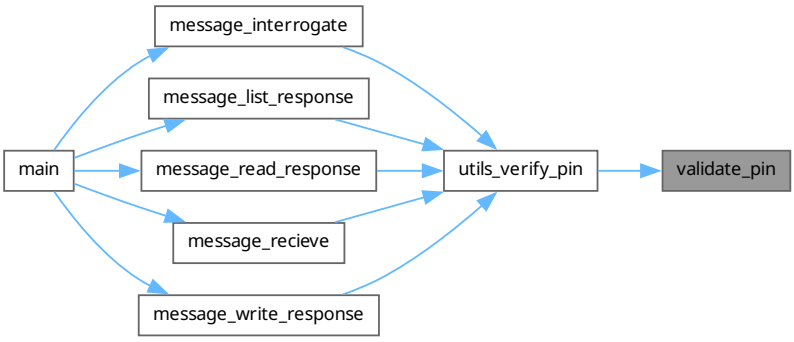
Parameters

in	*pin*	PIN string to validate (6 characters)

Returns

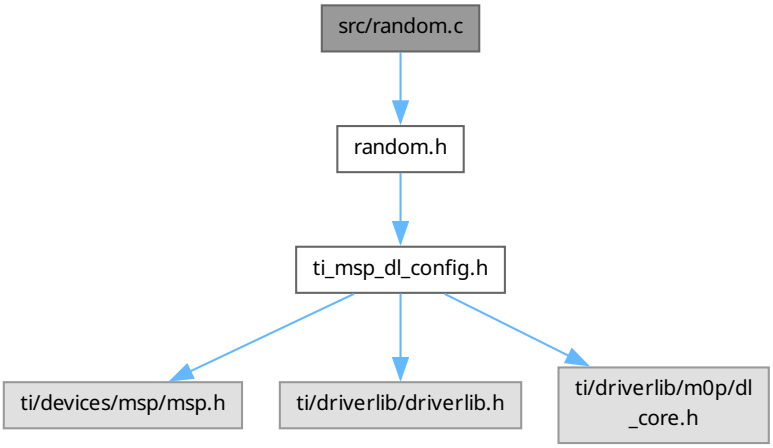
true if PIN is valid, false otherwise

Here is the caller graph for this function:



7.49 src/random.c File Reference

`#include "random.h"`
Include dependency graph for random.c:



Functions

- void random_fill_buffer (uint8_t *buffer, size_t length)

7.49.1 Function Documentation

7.49.1.1 random_fill_buffer()

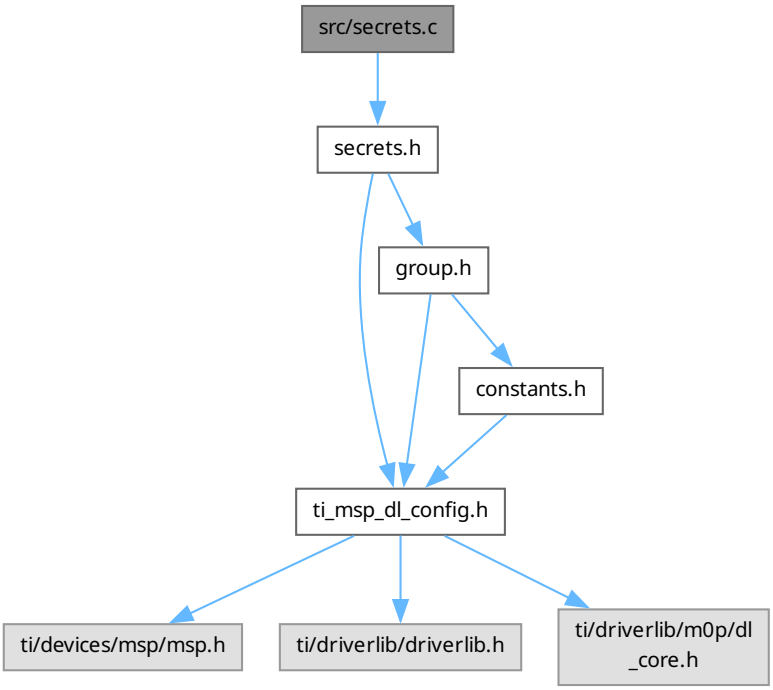
```
void random_fill_buffer (
            uint8_t * buffer,
            size_t length)
```

This function generates random bytes by repeatedly sampling the True Random Number Generator (TRNG) hardware module and converting the 32-bit outputs into individual bytes. The function blocks until the entire output buffer is filled.

7.50 src/secrets.c File Reference

```
#include "secrets.h"
```

Include dependency graph for secrets.c:



Variables

- const uint8_t TRANSFER_KEY [32] = { 0x96, 0x4a, 0x20, 0xbb, 0x48, 0x78, 0x3c, 0xa0, 0x71, 0xaa, 0x0d, 0xe5, 0xfe, 0x48, 0x0e, 0x41, }
- const group_t groups [1]

7.50.1 Variable Documentation

7.50.1.1 groups

```
const group_t groups[1]
```

Initial value:

```
= {

    {
        .group_id = 1,
        .private = {
            .permissions = {
                .read = false,
                .write = true,
                .transfer = true,
            },
            .read_key = {  },
            .write_key = { 0x96, 0x6d, 0x7d, 0x77, 0xd1, 0x32, 0xa2, 0xb6, 0x07, 0xd8, 0x49, 0x6c, 0x99,
      0x3e, 0x63, 0xaa, 0xa7, 0x64, 0xf5, 0x0c, 0x5b, 0x1e, 0x56, 0x8e, 0x08, 0x83, 0xa1, 0x10, 0xcd, 0x0d,
      0x8a, 0x25, 0xd3, 0x27, 0xe8, 0xcc, 0x5c, 0x9f, 0x4b, 0x5c, 0x8f, 0x09, 0x88, 0x0d, 0x33, 0x55, 0xef,
      0xc6, 0x83, 0xca, 0x45, 0xcc, 0xc3, 0xc1, 0x7b, 0xc1, 0xda, 0xa8, 0xa9, 0x41, 0xfb, 0x37, 0xe7, 0x6a,
      },
            .transfer_key = { 0xc4, 0x49, 0xff, 0xca, 0x1a, 0x8b, 0xad, 0xab, 0x88, 0x99, 0xb9, 0xf5, 0x01,
      0x9e, 0xd4, 0x1b, 0x77, 0xeb, 0x55, 0xa6, 0x1b, 0x32, 0x3b, 0xd2, 0xe0, 0x3b, 0xa1, 0xf6, 0xe1, 0xf8,
      0xc5, 0x22,  },
        },
        .public = {
            .read_key = { 0xe4, 0xf1, 0x54, 0x04, 0xa0, 0xf5, 0xf1, 0x62, 0x6b, 0x8d, 0x92, 0x02, 0x4e,
      0x28, 0x3f, 0xf9, 0xe8, 0x04, 0xed, 0x09, 0x69, 0x6a, 0x3e, 0x9c, 0x72, 0x28, 0x15, 0x53, 0xf1, 0xbb,
      0x90, 0x73,  },
            .write_key = { 0xd3, 0x27, 0xe8, 0xcc, 0x5c, 0x9f, 0x4b, 0x5c, 0x8f, 0x09, 0x88, 0x0d, 0x33,
      0x55, 0xef, 0xc6, 0x83, 0xca, 0x45, 0xcc, 0xc3, 0xc1, 0x7b, 0xc1, 0xda, 0xa8, 0xa9, 0x41, 0xfb, 0x37,
      0xe7, 0x6a,  },
            .transfer_key = { 0xd7, 0xfb, 0x07, 0x5f, 0xbf, 0xe6, 0xc2, 0x5f, 0x5c, 0x95, 0x02, 0xf0, 0x32,
      0xac, 0xaa, 0x29, 0x0b, 0xbc, 0xdf, 0x00, 0x39, 0xab, 0x30, 0x01, 0x2e, 0xa1, 0xd9, 0xda, 0x47, 0x9d,
      0x4a, 0x5b,  },
        }
    },

}
```

7.50.1.2 TRANSFER_KEY

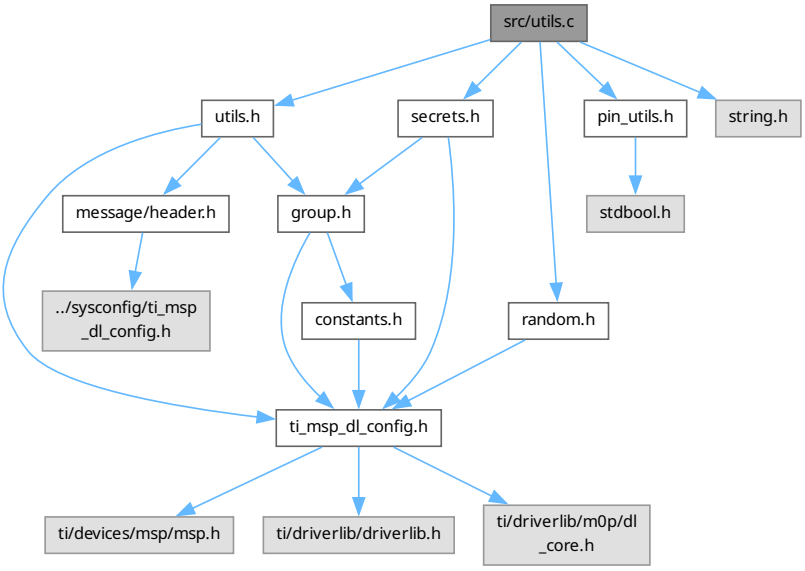
```
const uint8_t TRANSFER_KEY[32] = { 0x96, 0x4a, 0x20, 0xbb, 0x48, 0x78, 0x3c, 0xa0, 0x71, 0xaa,
0x0d, 0xe5, 0xfe, 0x48, 0x0e, 0x41, }
```

7.51 src/utils.c File Reference

```
#include "utils.h"
#include "secrets.h"
#include "random.h"
#include "pin_utils.h"
```

```
#include <string.h>
```

Include dependency graph for utils.c:



Functions

- bool utils_verify_pin (uint8_t ∗pin, size_t pin_length)

  Verifies host PIN.
- void utils_send_buffer (UART_Regs ∗uart, const void ∗buffer, size_t length, size_t bytes_sent)

  Sends buffer over UART with ACKing.
- void utils_send_buffer_no_ack (UART_Regs ∗uart, const void ∗buffer, size_t length)

  Sends buffer over UART without ACKing.
- void utils_receive_bytes (UART_Regs ∗uart, void ∗buffer, size_t num_bytes)

  Receives specified number of bytes from UART.
- const group_t ∗ utils_find_group (uint16_t group_id)

  Finds group by ID.
- void utils_random_delay ()

  Delay between 0 - 4096 cycles to mitigate timing attacks.

7.51.1 Function Documentation

7.51.1.1 utils_find_group()

```
const group_t * utils_find_group (
            uint16_t group_id)
```

Finds group by ID.

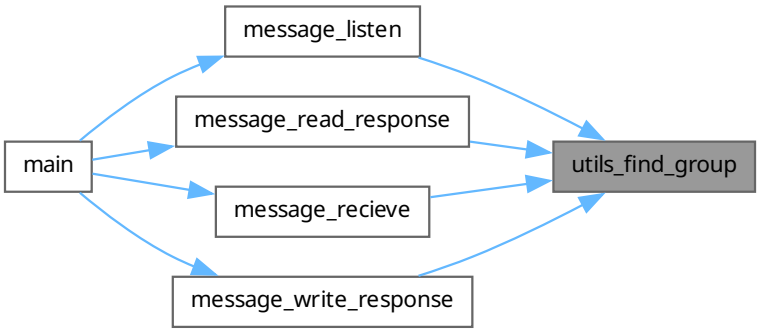
Parameters

in	*group↩ _id*	Group identifier to search for

Returns

Pointer to group if found, NULL otherwise

Here is the caller graph for this function:

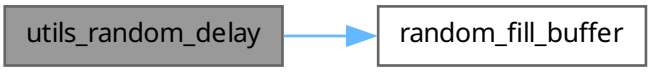


7.51.1.2 utils_random_delay()

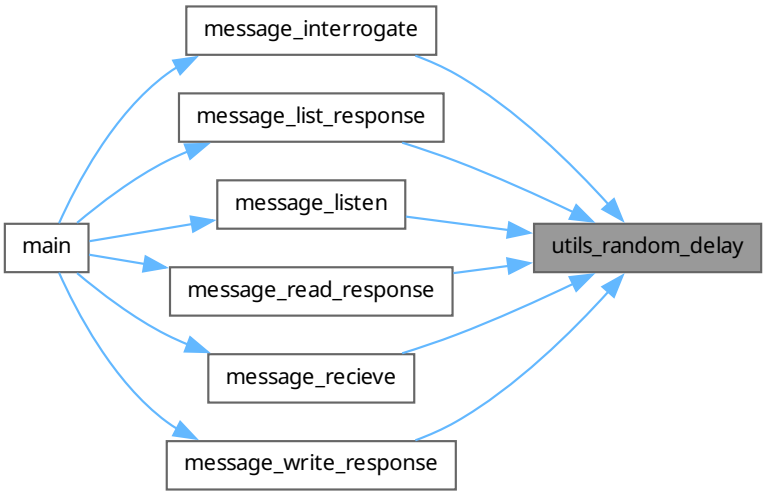
```
void utils_random_delay ()
```

Delay between 0 - 4096 cycles to mitigate timing attacks.

Here is the call graph for this function:



Here is the caller graph for this function:



7.51.1.3 utils_receive_bytes()

```
void utils_receive_bytes (
            UART_Regs * uart,
            void * buffer,
            size_t num_bytes)
```

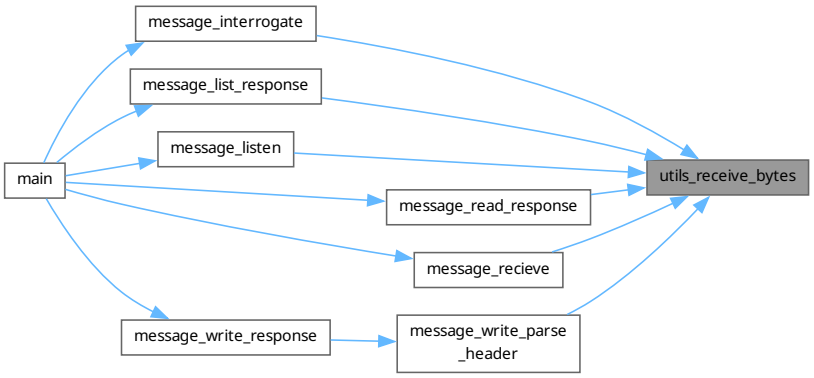
Receives specified number of bytes from UART.

Blocks until num_bytes have been read from the provided uart and stored into buffer.

Parameters

in	*uart*	UART peripheral
out	*buffer*	Destination buffer
in	*num_bytes*	Number of bytes to receive

Here is the caller graph for this function:



7.51.1.4 utils_send_buffer()

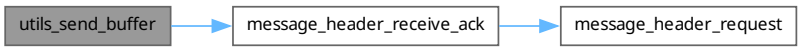
```
void utils_send_buffer (
            UART_Regs * uart,
            const void * buffer,
            size_t length,
            size_t bytes_sent)
```

Sends buffer over UART with ACKing.

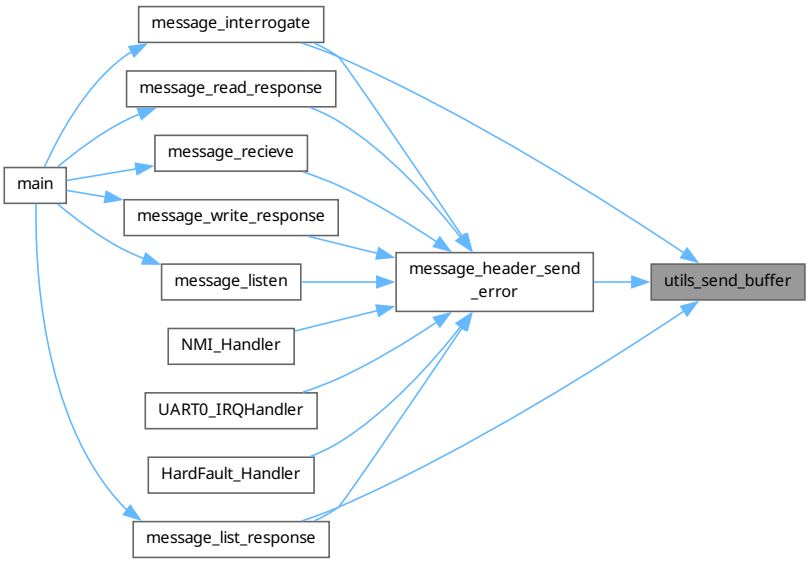
Parameters

in	*uart*	UART peripheral
in	*buffer*	Buffer to send
in	*length*	Buffer length
in	*bytes_sent*	Number of bytes already sent (for resumption)

Here is the call graph for this function:



Here is the caller graph for this function:



7.51.1.5 utils_send_buffer_no_ack()

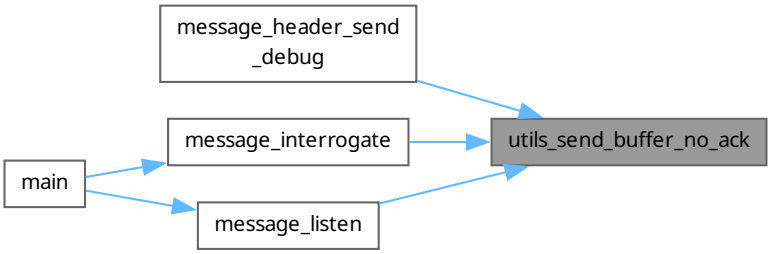
```
void utils_send_buffer_no_ack (
            UART_Regs * uart,
            const void * buffer,
            size_t length)
```

Sends buffer over UART without ACKing.

Parameters

in	*uart*	UART peripheral
in	*buffer*	Buffer to send
in	*length*	Buffer length

Here is the caller graph for this function:



7.51.1.6 utils_verify_pin()

```
bool utils_verify_pin (
            uint8_t * pin,
            size_t pin_length)
```

Verifies host PIN.

Parameters

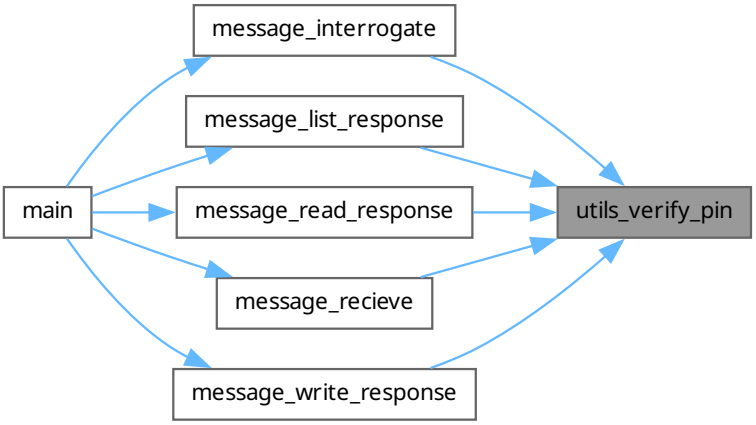
in	*pin*	PIN buffer
in	*pin_length*	Length of PIN

Returns

true if PIN is valid, false otherwise

Here is the call graph for this function:

Here is the caller graph for this function:



Index

/workspaces/2026-ectf-osu-design/docs/design.md, 45
/workspaces/2026-ectf-osu-design/docs/index.md, 45
/workspaces/2026-ectf-osu-design/docs/misc.md, 45

address
    file_fat_entry_t, 18
async_uart.c
    async_uart_receive, 103
    DMA_IRQHandler, 103
    transfers, 104
async_uart.h
    async_uart_receive, 46
async_uart_ctx, 13
    buffer, 14
    bytes_transfered, 14
    dma_channel, 14
    first_chunk_offset, 14
    stop, 14
    stopped, 15
    total_bytes, 15
    transfer_complete, 15
    uart, 15
async_uart_receive
    async_uart.c, 103
    async_uart.h, 46

buffer
    async_uart_ctx, 14
    BufferNode, 16
BufferNode, 15
    buffer, 16
    length, 16
    next, 16
buffers
    read.c, 124
    write.c, 129
bytes_transfered
    async_uart_ctx, 14

CHUNK_SIZE
    constants.h, 48
COMPILER_FUNC_OPTS
    pin_utils.c, 130
CONFIG_LP_MSPM0L2228
    ti_msp_dl_config.h, 89
CONFIG_MSPM0L2228
    ti_msp_dl_config.h, 89
constants.h
    CHUNK_SIZE, 48
    MAX_FILE_SIZE, 48
    MAX_GROUPS, 48
    NUM_SLOTS, 48
    PIN_DELAY, 48
    WRITE_HEADER_SIZE, 48
CPUCLK_FREQ
    ti_msp_dl_config.h, 89

DMA_CHANNEL
    listen.c, 121
    read.c, 123
    write.c, 127
dma_channel
    async_uart_ctx, 14
DMA_IRQHandler
    async_uart.c, 103

encrypted_file
    file_slot_entry_t, 24
encryption_public_key
    file_metadata_t, 21
entries
    file_address_table_t, 17
    message_list_response_t, 42

file.c
    FILE_ADDRESS_TABLE, 107
    file_read_fat, 105
    file_write_fat, 106
    SLOTS, 107
file.h
    FILE_ADDRESS_TABLE, 54
    file_read_fat, 52
    file_write_fat, 53
    FLASH_SECTOR_SIZE, 52
    FLASH_WORD_SIZE, 52
    MAX_PADDING_BACK, 52
    SLOTS, 54
FILE_ADDRESS_TABLE
    file.c, 107
    file.h, 54
file_address_table_t, 17
    entries, 17
file_fat_entry_t, 18
    address, 18
    length, 18
    padding, 19
    uuid, 19
file_info
    message_interrogate_file_info_encrypted_t, 34
file_length

message_write_header_t, 43
file_metadata_signed_t, 19
metadata, 20
writer_signature, 20
file_metadata_t, 20
encryption_public_key, 21
file_signature, 21
file_size, 22
group_id, 22
name, 22
uuid, 22
file_read_fat
file.c, 105
file.h, 52
file_signature
file_metadata_t, 21
file_size
file_metadata_t, 22
file_slot_entry_t, 23
encrypted_file, 24
signed_metadata, 24
file_uuid
message_write_header_t, 43
file_write_fat
file.c, 106
file.h, 53
files
message_interrogate_file_info_t, 35
first_chunk_offset
async_uart_ctx, 14
FLASH_SECTOR_SIZE
file.h, 52
FLASH_WORD_SIZE
file.h, 52

GPIO_HOST_IOMUX_RX
ti_msp_dl_config.h, 89
GPIO_HOST_IOMUX_RX_FUNC
ti_msp_dl_config.h, 89
GPIO_HOST_IOMUX_TX
ti_msp_dl_config.h, 89
GPIO_HOST_IOMUX_TX_FUNC
ti_msp_dl_config.h, 89
GPIO_HOST_RX_PIN
ti_msp_dl_config.h, 89
GPIO_HOST_RX_PORT
ti_msp_dl_config.h, 90
GPIO_HOST_TX_PIN
ti_msp_dl_config.h, 90
GPIO_HOST_TX_PORT
ti_msp_dl_config.h, 90
GPIO_HSM_IOMUX_RX
ti_msp_dl_config.h, 90
GPIO_HSM_IOMUX_RX_FUNC
ti_msp_dl_config.h, 90
GPIO_HSM_IOMUX_TX
ti_msp_dl_config.h, 90
GPIO_HSM_IOMUX_TX_FUNC
ti_msp_dl_config.h, 90
GPIO_HSM_RX_PIN
ti_msp_dl_config.h, 90
GPIO_HSM_RX_PORT
ti_msp_dl_config.h, 90
GPIO_HSM_TX_PIN
ti_msp_dl_config.h, 90
GPIO_HSM_TX_PORT
ti_msp_dl_config.h, 91
group_id
file_metadata_t, 22
group_t, 31
message_interrogate_file_metadata_t, 36
message_list_file_entry_t, 40
message_write_header_t, 43
group_ids
message_interrogate_group_list_t, 39
group_list
message_interrogate_group_list_encrypted_t, 38
group_permissions_t, 24
read, 25
transfer, 25
write, 25
group_private_t, 26
permissions, 27
read_key, 27
transfer_key, 27
write_key, 27
group_public_t, 28
read_key, 28
transfer_key, 28
write_key, 29
group_t, 29
group_id, 31
private, 31
public, 31
groups
secrets.c, 134
secrets.h, 86

HardFault_Handler
main.c, 108
header.c
message_header_receive_ack, 111
message_header_request, 112
message_header_response, 113
message_header_send_ack, 114
message_header_send_debug, 115
message_header_send_error, 115
header.h
MESSAGE_ACK, 58
MESSAGE_DEBUG, 58
MESSAGE_ERROR, 58
message_header_receive_ack, 59
message_header_request, 59
message_header_response, 60
message_header_send_ack, 61
message_header_send_debug, 62
message_header_send_error, 62
MESSAGE_INTERROGATE, 58

MESSAGE_LIST, 58
MESSAGE_LISTEN, 58
message_operation_t, 58
MESSAGE_READ, 58
MESSAGE_RECEIVE, 58
MESSAGE_WRITE, 58
High Level Design, 3
HOST_BAUD_RATE
ti_msp_dl_config.h, 91
HOST_FBRD_32_MHZ_115200_BAUD
ti_msp_dl_config.h, 91
HOST_IBRD_32_MHZ_115200_BAUD
ti_msp_dl_config.h, 91
HOST_INST
ti_msp_dl_config.h, 91
HOST_INST_FREQUENCY
ti_msp_dl_config.h, 91
HOST_INST_INT_IRQN
ti_msp_dl_config.h, 91
HOST_INST_IRQHandler
ti_msp_dl_config.h, 91
HSM_BAUD_RATE
ti_msp_dl_config.h, 91
HSM_FBRD_32_MHZ_115200_BAUD
ti_msp_dl_config.h, 91
HSM_IBRD_32_MHZ_115200_BAUD
ti_msp_dl_config.h, 92
HSM_INST
ti_msp_dl_config.h, 92
HSM_INST_FREQUENCY
ti_msp_dl_config.h, 92
HSM_INST_INT_IRQN
ti_msp_dl_config.h, 92
HSM_INST_IRQHandler
ti_msp_dl_config.h, 92

include/async_uart.h, 45, 47
include/constants.h, 47, 49
include/datastructures.h, 49, 50
include/file.h, 50, 55
include/group.h, 55, 56
include/message/header.h, 57, 64
include/message/interrogate.h, 64, 67
include/message/list.h, 68, 70
include/message/listen.h, 71, 72
include/message/read.h, 73, 74
include/message/receive.h, 75, 76
include/message/write.h, 77, 79
include/pin_utils.h, 80, 82
include/random.h, 83, 84
include/secrets.h, 84, 87
include/ti_msp_dl_config.h, 87, 94
include/utils.h, 96, 102
init_pin
pin_utils.c, 131
pin_utils.h, 81
interrogate.c
message_interrogate, 117
interrogate.h
message_interrogate, 66
iv
message_interrogate_file_info_encrypted_t, 34
message_interrogate_group_list_encrypted_t, 38

length
BufferNode, 16
file_fat_entry_t, 18
list.c
message_list_response, 120
list.h
message_list_response, 69
listen.c
DMA_CHANNEL, 121
message_listen, 121
listen.h
message_listen, 71

main
main.c, 108
main.c
HardFault_Handler, 108
main, 108
NMI_Handler, 109
UART0_IRQHandler, 110
MAX_FILE_SIZE
constants.h, 48
MAX_GROUPS
constants.h, 48
MAX_PADDING_BACK
file.h, 52
MESSAGE_ACK
header.h, 58
MESSAGE_DEBUG
header.h, 58
MESSAGE_ERROR
header.h, 58
message_header_receive_ack
header.c, 111
header.h, 59
message_header_request
header.c, 112
header.h, 59
message_header_response
header.c, 113
header.h, 60
message_header_send_ack
header.c, 114
header.h, 61
message_header_send_debug
header.c, 115
header.h, 62
message_header_send_error
header.c, 115
header.h, 62
message_header_t, 31
message_length, 32
operation, 32
MESSAGE_INTERROGATE

header.h, 58
message_interrogate
interrogate.c, 117
interrogate.h, 66
message_interrogate_file_info_encrypted_t, 32
file_info, 34
iv, 34
tag, 34
message_interrogate_file_info_t, 34
files, 35
num_files, 35
message_interrogate_file_metadata_t, 35
group_id, 36
name, 36
slot, 36
message_interrogate_group_list_encrypted_t, 37
group_list, 38
iv, 38
tag, 38
message_interrogate_group_list_t, 38
group_ids, 39
num_groups, 39
message_length
message_header_t, 32
MESSAGE_LIST
header.h, 58
message_list_file_entry_t, 39
group_id, 40
name, 40
slot, 40
message_list_response
list.c, 120
list.h, 69
message_list_response_t, 41
entries, 42
num_files, 42
MESSAGE_LISTEN
header.h, 58
message_listen
listen.c, 121
listen.h, 71
message_operation_t
header.h, 58
MESSAGE_READ
header.h, 58
message_read_response
read.c, 124
read.h, 73
MESSAGE_RECEIVE
header.h, 58
message_recieve
receive.c, 125
receive.h, 75
MESSAGE_WRITE
header.h, 58
message_write_header_t, 42
file_length, 43
file_uuid, 43
group_id, 43
name, 43
pin, 43
slot, 44
message_write_parse_header
write.c, 127
write.h, 78
message_write_response
write.c, 128
write.h, 78
metadata
file_metadata_signed_t, 20
Miscellaneous Details, 7

name
file_metadata_t, 22
message_interrogate_file_metadata_t, 36
message_list_file_entry_t, 40
message_write_header_t, 43
next
BufferNode, 16
NMI_Handler
main.c, 109
num_files
message_interrogate_file_info_t, 35
message_list_response_t, 42
num_groups
message_interrogate_group_list_t, 39
NUM_SLOTS
constants.h, 48

operation
message_header_t, 32
Overview, 1

padding
file_fat_entry_t, 19
permissions
group_private_t, 27
pin
message_write_header_t, 43
PIN_DELAY
constants.h, 48
pin_utils.c
COMPILER_FUNC_OPTS, 130
init_pin, 131
RANDOM_FUNC, 130
SEED, 131
validate_pin, 131
pin_utils.h
init_pin, 81
validate_pin, 81
POWER_STARTUP_DELAY
ti_msp_dl_config.h, 92
private
group_t, 31
public
group_t, 31

random.c
random_fill_buffer, 133
random.h
random_fill_buffer, 83
random_fill_buffer
random.c, 133
random.h, 83
RANDOM_FUNC
pin_utils.c, 130
read
group_permissions_t, 25
read.c
buffers, 124
DMA_CHANNEL, 123
message_read_response, 124
READ_HEADER_LENGTH, 123
read.h
message_read_response, 73
READ_HEADER_LENGTH
read.c, 123
read_key
group_private_t, 27
group_public_t, 28
receive.c
message_recieve, 125
receive.h
message_recieve, 75

secrets.c
groups, 134
TRANSFER_KEY, 134
secrets.h
groups, 86
STAGE1_KEY, 86
STAGE1_PADDING, 86
STAGE1_PIN_HASH, 86
STAGE1_PIN_ITERATIONS, 86
STAGE2_RAND_INT, 86
TRANSFER_KEY, 86
SEED
pin_utils.c, 131
signed_metadata
file_slot_entry_t, 24
slot
message_interrogate_file_metadata_t, 36
message_list_file_entry_t, 40
message_write_header_t, 44
SLOTS
file.c, 107
file.h, 54
src/async_uart.c, 102
src/file.c, 104
src/main.c, 108
src/message/header.c, 110
src/message/interrogate.c, 117
src/message/list.c, 119
src/message/listen.c, 120
src/message/read.c, 122
src/message/receive.c, 125
src/message/write.c, 126
src/pin_utils.c, 129
src/random.c, 132
src/secrets.c, 133
src/utils.c, 134
STAGE1_KEY
secrets.h, 86
STAGE1_PADDING
secrets.h, 86
STAGE1_PIN_HASH
secrets.h, 86
STAGE1_PIN_ITERATIONS
secrets.h, 86
STAGE2_RAND_INT
secrets.h, 86
stop
async_uart_ctx, 14
stopped
async_uart_ctx, 15
SYSCFG_DL_GPIO_init
ti_msp_dl_config.h, 93
SYSCFG_DL_HOST_init
ti_msp_dl_config.h, 93
SYSCFG_DL_HSM_init
ti_msp_dl_config.h, 93
SYSCFG_DL_init
ti_msp_dl_config.h, 93
SYSCFG_DL_initPower
ti_msp_dl_config.h, 93
SYSCFG_DL_SYSCTL_init
ti_msp_dl_config.h, 93
SYSCFG_DL_TRNG_init
ti_msp_dl_config.h, 94
SYSCFG_DL_WWDT0_init
ti_msp_dl_config.h, 94

tag
message_interrogate_file_info_encrypted_t, 34
message_interrogate_group_list_encrypted_t, 38
ti_msp_dl_config.h
CONFIG_LP_MSPM0L2228, 89
CONFIG_MSPM0L2228, 89
CPUCLK_FREQ, 89
GPIO_HOST_IOMUX_RX, 89
GPIO_HOST_IOMUX_RX_FUNC, 89
GPIO_HOST_IOMUX_TX, 89
GPIO_HOST_IOMUX_TX_FUNC, 89
GPIO_HOST_RX_PIN, 89
GPIO_HOST_RX_PORT, 90
GPIO_HOST_TX_PIN, 90
GPIO_HOST_TX_PORT, 90
GPIO_HSM_IOMUX_RX, 90
GPIO_HSM_IOMUX_RX_FUNC, 90
GPIO_HSM_IOMUX_TX, 90
GPIO_HSM_IOMUX_TX_FUNC, 90
GPIO_HSM_RX_PIN, 90
GPIO_HSM_RX_PORT, 90
GPIO_HSM_TX_PIN, 90
GPIO_HSM_TX_PORT, 91

HOST_BAUD_RATE, 91
HOST_FBRD_32_MHZ_115200_BAUD, 91
HOST_IBRD_32_MHZ_115200_BAUD, 91
HOST_INST, 91
HOST_INST_FREQUENCY, 91
HOST_INST_INT_IRQN, 91
HOST_INST_IRQHandler, 91
HSM_BAUD_RATE, 91
HSM_FBRD_32_MHZ_115200_BAUD, 91
HSM_IBRD_32_MHZ_115200_BAUD, 92
HSM_INST, 92
HSM_INST_FREQUENCY, 92
HSM_INST_INT_IRQN, 92
HSM_INST_IRQHandler, 92
POWER_STARTUP_DELAY, 92
SYSCFG_DL_GPIO_init, 93
SYSCFG_DL_HOST_init, 93
SYSCFG_DL_HSM_init, 93
SYSCFG_DL_init, 93
SYSCFG_DL_initPower, 93
SYSCFG_DL_SYSCTL_init, 93
SYSCFG_DL_TRNG_init, 94
SYSCFG_DL_WWDT0_init, 94
TRNG_INT_IIDX, 92
TRNG_INT_IRQN, 92
WWDT0_INST, 92
WWDT0_INT_IRQN, 92
total_bytes
async_uart_ctx, 15
transfer
group_permissions_t, 25
transfer_complete
async_uart_ctx, 15
TRANSFER_KEY
secrets.c, 134
secrets.h, 86
transfer_key
group_private_t, 27
group_public_t, 28
transfers
async_uart.c, 104
TRNG_INT_IIDX
ti_msp_dl_config.h, 92
TRNG_INT_IRQN
ti_msp_dl_config.h, 92

uart
async_uart_ctx, 15
UART0_IRQHandler
main.c, 110
utils.c
utils_find_group, 135
utils_random_delay, 136
utils_receive_bytes, 137
utils_send_buffer, 138
utils_send_buffer_no_ack, 139
utils_verify_pin, 139
utils.h
utils_find_group, 97
utils_random_delay, 97
utils_receive_bytes, 98
utils_send_buffer, 99
utils_send_buffer_no_ack, 100
utils_verify_pin, 100
utils_find_group
utils.c, 135
utils.h, 97
utils_random_delay
utils.c, 136
utils.h, 97
utils_receive_bytes
utils.c, 137
utils.h, 98
utils_send_buffer
utils.c, 138
utils.h, 99
utils_send_buffer_no_ack
utils.c, 139
utils.h, 100
utils_verify_pin
utils.c, 139
utils.h, 100
uuid
file_fat_entry_t, 19
file_metadata_t, 22

validate_pin
pin_utils.c, 131
pin_utils.h, 81

write
group_permissions_t, 25
write.c
buffers, 129
DMA_CHANNEL, 127
message_write_parse_header, 127
message_write_response, 128
write.h
message_write_parse_header, 78
message_write_response, 78
WRITE_HEADER_SIZE
constants.h, 48
write_key
group_private_t, 27
group_public_t, 29
writer_signature
file_metadata_signed_t, 20
WWDT0_INST
ti_msp_dl_config.h, 92
WWDT0_INT_IRQN
ti_msp_dl_config.h, 92